Covered User's Guide 0.7.10

Verilog Code Coverage Analyzer

Trevor Williams phase1geo@gmail.com

- 1 Overview
  - 1.1 Introduction
    - 1.1.1 What is Covered?
    - 1.1.2 What can code coverage do?
    - 1.1.3 What can't code coverage do?
    - 1.1.4 What does Covered do?
    - 1.1.5 What does Covered NOT do?
    - 1.1.6 What makes Covered different?
  - 1.2 Coverage Metrics
    - 1.2.1 Line Coverage
    - 1.2.2 Toggle Coverage
    - 1.2.3 Memory Coverage
    - 1.2.4 Combinational Logic Coverage
    - 1.2.5 Finite State Machine (FSM) Coverage
    - 1.2.6 Assertion Coverage
  - 1.3 Coverage Boundaries
    - 1.3.1 What logic can be analyzed?
    - 1.3.2 What logic cannot be currently analyzed?
    - 1.3.3 What is done with code blocks that contain code that cannot be analyzed?
  - 1.4 Race Condition Checking
    - 1.4.1 Checked Coding Guidelines
    - 1.4.2 Race Condition Handling
    - 1.4.3 Avoiding Race Condition Checking
    - 1.4.4 Reporting Race Conditions
  - 1.5 Inline Attributes
    - 1.5.1 What are inline attributes?
    - 1.5.2 Adding FSM attributes
- 2 Installation
  - 2.1 Installation
    - 2.1.1 Prerequisites
    - 2.1.2 Downloading source
    - 2.1.3 Configuring the build
    - 2.1.4 Compiling and Installing
- 3 Command-line Usage
  - 3.1 Getting Started
    - 3.1.1 Methods for scoring the design
    - 3.1.2 What is needed for dumpfile scoring?
    - 3.1.3 What is needed for VPI scoring?
  - 3.2 Using Covered
    - 3.2.1 Work Flow
    - 3.2.2 Covered usage
    - 3.2.3 Covered global options
  - 3.3 The score Command
    - 3.3.1 Usage
    - 3.3.2 Options
    - 3.3.3 Specifying What to Cover
    - 3.3.4 Specifying What Not to Cover (Coverage Exclusions)
    - 3.3.5 Overriding Parameters (-P option)
    - 3.3.6 Scoring FSMs
    - 3.3.7 The reason for the -conservative feature
    - 3.3.8 Specifying Plus Arguments When Scoring
    - 3.3.9 Other Notes
  - 3.4 The merge Command
    - 3.4.1 Usage
    - 3.4.2 Options
    - 3.4.3 Merging Rules
  - 3.5 The report Command
    - 3.5.1 Usage
    - 3.5.2 Options

        - 3.5.3 Summary Vs. Detailed Vs. Verbose
        - 3.5.4 Module Vs. Instance
        - 3.5.5 Covered Vs. Uncovered
    - 3.6 The rank Command
        - 3.6.1 Options
        - 3.6.2 Methods for specifying CDDs to rank
        - 3.6.3 Specifying metric weights
        - 3.6.4 Understanding the rank command output information
        - 3.6.5 Description of ranking algorithm
    - 3.7 The exclude Command
        - 3.7.1 Usage
        - 3.7.2 Options
    - 3.8 Reading the Report
        - 3.8.1 Reading Line Coverage
        - 3.8.2 Reading Toggle Coverage
        - 3.8.3 Reading Memory Coverage
        - 3.8.4 Reading Combinational Logic Coverage
        - 3.8.5 Reading FSM Coverage
        - 3.8.6 Reading Assertion Coverage
    - 3.9 Debugging
        - 3.9.1 Verbose Debug Output
        - 3.9.2 Command-Line Interface (CLI)
        - 3.9.3 Source Code Profiling
- 4 Graphical User Interface
    - 4.1 Introduction to the GUI
    - 4.2 Navigating the Main Window
        - 4.2.1 The Main Menu Bar
        - 4.2.2 The Coverage Type View Bar
        - 4.2.3 The Module/Instance Listbox
        - 4.2.4 The Coverage File Viewer
        - 4.2.5 The Information Bar
    - 4.3 Navigating the Line Coverage Window
        - 4.3.1 Viewing Uncovered Lines
        - 4.3.2 Excluding/Included Uncovered Lines from Line Coverage
        - 4.3.3 Viewing Covered Lines
    - 4.4 Navigating the Toggle Coverage Window
        - 4.4.1 Understanding the Layout
        - 4.4.2 Excluding/Including a Signal for Toggle Coverage
        - 4.4.3 Displaying a New Signal for Toggle Coverage
    - 4.5 Navigating the Memory Coverage Window
        - 4.5.1 Understanding the Layout
        - 4.5.2 Excluding/Including a Memory for Memory Coverage
        - 4.5.3 Displaying a New Memory for Memory Coverage
    - 4.6 Navigating the Combinational Logic Window
        - 4.6.1 Navigating a Statement
        - 4.6.2 Understanding the Coverage Information
        - 4.6.3 Excluding/Including a Subexpression
        - 4.6.4 Displaying a New Statement
    - 4.7 Navigating the FSM State/State Transition Coverage Window
        - 4.7.1 Navigating the FSM Window
        - 4.7.2 Excluding/Including a State Transition
        - 4.7.3 Displaying a New FSM
    - 4.8 Navigating the Assertion Coverage Window
        - 4.8.1 Navigating the Assertion Coverage Window
        - 4.8.2 Excluding/Including Assertion Coverage Points
        - 4.8.3 Displaying a New Assertion Instance
    - 4.9 Navigating the Assertion Source Code Window
        - 4.9.1 Navigating the Source Code
    - 4.10 Creating a New CDD
        - 4.10.1 Navigating the New CDD Creation Option Selection Page
        - 4.10.2 Navigating the New CDD Creation CDD Name Page
        - 4.10.3 CDD Creation Type Page
        - 4.10.4 CDD Creation Options1 Page
        - 4.10.5 CDD Creation Options2 Page
        - 4.10.6 CDD Creation Output Pane
    - 4.11 Creating an ASCII report file
        - 4.11.1 Navigating the Report Generator Selection Page

        - 4.11.2 Report Generation Options Page
    - 4.12 Creating a CDD Ranking Report
        - 4.12.1 CDD Ranking Option Selection Page
        - 4.12.2 CDD Ranking Option Page
        - 4.12.3 CDD Ranking Input File Selection Page
        - 4.12.4 CDD ranking Command Output Page
    - 4.13 Navigating the Preferences Window
        - 4.13.1 Option Categories
        - 4.13.2 Preference Buttons
        - 4.13.3 Preferences Configuration File
        - 4.13.4 Preferences General Pane
        - 4.13.5 Preferences Color Pane
        - 4.13.6 Preference Coverage Goals Pane
        - 4.13.7 Preferences Syntax Highlighting Pane
        - 4.13.8 Preferences Exclusion Pane
        - 4.13.9 Preference Merging Pane
    - 4.14 The Exclusion Reason Popup Window
    - 4.15 Navigating the Wizard Window
- 5 FAQ
    - 5.1 FAQ
- 6 Epilogue
    - 6.1 Epilogue
        - 6.1.1 Author
        - 6.1.2 Special Thanks
        - 6.1.3 Reporting Bugs
        - 6.1.4 Mailing List
        - 6.1.5 Homepage
        - 6.1.6 Copyright and Licensing

1 Overview

1.1 Introduction

1.1.1 What is Covered?

Covered is a Verilog code coverage analysis tool that can be useful for determining how well a diagnostic test suite is covering the design under test (DUT). It is command-line based with an optional Tcl/Tk GUI report viewer, making it portable across almost all platforms.

1.1.2 What can code coverage do?

Typically, in the design verification work flow, a design verification engineer will develop a self-checking test suite to verify design elements/functions specified by a design's specification document. When the test suite contains all of the tests required by the design specification, the test writer may ask:

- "How much logic in the design is actually being exercised?"
- "Does my test suite cover all of the DUT?"
- "Am I done writing tests for the logic?"
- "Do I have any redundant diagnostics in my test suite?"

When the design verification gets to this point, it is often useful to get some metrics for determining logic coverage. This is where a code coverage utility, such as Covered, is very useful.

The metrics obtained by using a code coverage analysis tool can be very useful for determining the following about a design and the test suite testing that design:

1. Completeness of the test suite in terms of logic coverage.
2. Unexercised logic in the design (useful in helping to determine what types of tests need to be added to the test suite).
3. Corner cases in design that are untestable.

1.1.3 What can't code coverage do?

It is important to note that any code coverage tool is only useful in indicating how much logic is being covered by a test suite. It does not indicate that the covered logic works appropriately. This, of course, can only be verified by the diagnostics themselves.

Additionally, it is possible that two or more diagnostics can achieve the same coverage and yet be functionally testing different characteristics of a design. Since the coverage metrics are not improved in this case, one may conclude that the second test is unnecessary. This may or may not be true depending on what is being tested; it is always up to the test writer to determine the necessity of a diagnostic. Using the code coverage tool results as the sole means of making this determination is not recommended. Use common sense in these areas.

1.1.4 What does Covered do?

Covered is a tool that uses your design files along with standard VCD, LXT2 or FST dump files to analyze the code coverage of the DUT. The code coverage information is stored in a special database file that can be retrieved and "merged" with new coverage information to create a summed coverage total for several tests. After a database file has been created, the user may generate various ASCII reports that summarize the coverage information or run Covered's GUI to interactively analyze the coverage information. Additionally, multiple CDD files from the same design can be ranked for the purposes of running regressions and understanding which CDD files do not add coverage information and can be excluded from regressions runs, if needed.

1.1.5 What does Covered NOT do?

Though Covered does perform some resimulation of the design to derive line, combinational logic, memory, and assertion coverage, Covered is NOT a full-fledged compiler/simulator and should not be used for such purposes. Additionally, Covered is NOT a linting tool. Many syntax/semantic issues that are not allowed by the LRM are allowed by Covered for purposes of making Covered's core more generic and/or simplistic. Please make sure that your code is properly linted in your design flow using an appropriate tool.

1.1.6 What makes Covered different?

Most Verilog code coverage tools perform a pre-compilation procedure known as instrumenting. During this procedure, the coverage tool will read in the DUT and generate its own version of the DUT with additional code added to aid in calculating coverage after the simulation is complete. The benefits to this style are higher performance of the coverage tool when it is run after the simulation. The drawbacks to this approach are that it slows down simulation speed and a trust issue that the coverage tool did not alter your simulation model when it performed the instrumentation.

Covered, on the other hand, omits the instrumenting procedure all-together and only performs its analysis on a pre-simulated design. This means that Covered cannot make a mistake to your simulation and the simulation is allowed to run faster since there is no additional code that must be run to get coverage data. The drawback is that there is some overhead in post-simulation for extracting coverage data but this is typically less than the simulation overhead in the other method.

Additionally, unlike all other commercial coverage tools, Covered is free! No license managers to invoke and maintain. No licensing fees and negotiations. The only thing the developers of Covered want in return are bug reports and user input (and perhaps a donation if you find the software useful and want to see its development continue). Isn't the open source life grand?!

1.2 Coverage Metrics

Covered currently generates six types of code coverage metrics.

1.2.1 Line Coverage

1.2.1.1 Description

Line coverage simply answers the question, "Was this line of code executed during simulation?"

Covered will display the number of logical lines of code that exist in a particular file with the number of logical lines that were executed during the simulation along with a percentage indicating the percentage of lines executed. If verbose mode is selected for a report, Covered will display the lines of logic that were not executed during the simulation run.

1.2.1.2 Example

Suppose that the following module was specified in your DUT that is being analyzed:

```
module test;

  reg a, b, c;

  initial begin
    a = 0;    // Line 1
    b = 1;    // Line 2
```

```
    if( a )
      c = b;  // Line 3
    else
      c = ~b; // Line 4
  end

endmodule
```

If this code were simulated as stated, the simulator would execute lines 1 and 2, and due to the value assigned to a, line 4 would be executed. This means that during execution, out of a possibility of 4 lines, only 3 were executed. Therefore, Covered would state that for this module the line coverage percentage would be 75% (or 3 out of 4).

1.2.1.3 Recommendations

For a design to pass full coverage, it is recommended that the line coverage for all modules in a design receive 100% coverage. If a line of logic is not executed during simulation, the design has not been fully exercised. Line coverage is useful for determining holes in the test suite.

1.2.2 Toggle Coverage

1.2.2.1 Description

Toggle coverage answers the question, "Did this bit of this wire/register change from a value of zero (0) to one (1) and back from one (1) to zero (0) during simulation?"

A bit is said to be fully covered when it toggles back and forth at least once. This metric does not indicate to the user that every value of a multi-bit vector was seen. For example, if we have a two bit vector called "foo", toggle coverage will not tell you that the value of foo was set to the values of 0, 1, 2 and 3. However, it will tell you that all bits in that vector were toggled back and forth.

1.2.2.2 Example

Suppose that a DUT was comprised of the following module:

```
module test;

  reg [2:0] a;

  initial begin
    a = 3'b0;
    #10;
    a = 3'b110;
    #10;
    a = 3'b010;
    #10;
  end

endmodule
```

The register called "a", after being simulated, would have achieved a total toggle percentage of 50% (or 3 out of 6). Can you see which toggles are missing?

Bit 0 has never toggled to 1 (and has never been at a value of 1 and toggled to 0); therefore, bit-0 has toggled 0% (0 out of 2). Bit 1 has toggled from a value of 0 to 1, but has not toggled from a value of 1 to 0; therefore, bit-1 has toggled 50% (0 out of 2). Bit 2 is the only bit that has fully toggled, achieving a toggle percentage of 100% (2 out of 2). If you add all of the possible toggles to the number of achieved toggles, you end up with 3 out of 6 (or 50% total toggle coverage).

1.2.2.3 Recommendations

For a design to pass full coverage, it is recommended that the toggle coverage for all modules in a design received 100% coverage. If a bit is never changes value, it is usually an indication that a mode is not being exercised in the design or a datapath has a stuck-at issue.

1.2.3 Memory Coverage

1.2.3.1 Description

Memory coverage answers a number of questions about memories/multi-dimensional arrays being used in your design, including the following:

1. Did all bits of each addressable memory element toggle from 0 to 1?
2. Did all bits of each addressable memory element toggle from 1 to 0?
3. Did every addressable memory element get written?
4. Did every addressable memory element get read?

Memories/multi-dimensional arrays can have two types of dimensions, packed and unpacked. Packed dimensional information is specified to the left of an array declaration while unpacked dimensional information is specified to the right of an array declaration. Consider the following example:

```
reg [3:0][2:0] foo[0:15];
```

The memory "foo" consists of three dimensions. The first dimension is an unpacked dimension which specifies that there are 16 addressable memory elements (AME). The number of AMEs are determined by multiplying the unpacked dimension of an array. The second two dimensions are packed dimensions, creating a total of 12 bits for each AME.

Other than getting toggle coverage information for each AME, memory coverage also allows us to easily see which AMEs were written and read during simulation. If all entries of a memory have not been written during a regression run, it could indicate several shortcomings of either the testbench or a logical problem with the write control logic, including, but not limited to, the following:

1. If no AMEs for a given memory were written, it could indicate that the testsuite was never able to get out of bypass mode for that memory.
2. If fewer AMEs for a given memory were written than were read, it could indicate that there is a read controller issue leading to an underflow issue with the memory.
3. For memories that act like stacks or use lowest/highest address first write selection scheme, indicates that the memory was never completely filled (may miss out on things like buffer overflow errors).
4. For memories that are written in a round-robin fashion, indicates that either not enough entries were written (could miss a write pointer wrap bug) or that there is a logical issue by which the write pointer did not properly increment in a round-robin fashion.
5. If there are "holes" in the write coverage of the AMEs (that is an AME is found to not have been written while those above and below it have been), this could indicate a corner case logic bug or other testsuite issue that needs to be addressed.

If all entries of a memory have not been read during a regression run, it could indicate several other shortcomings of either the testbench or a logical problem with the read control logic, including, but not limited to, the following:

1. If no AMEs for a given memory were read, it could indicate that the testsuite was never able to get out of bypass mode for that memory.
2. If fewer AMEs for a given memory were read than written, it could indicate that a resource allocation issue.
3. For memories that are written in a round-robin fashion, indicates that either not enough entries were read (could miss a read pointer wrap bug) or that there is a logical issue by which the read pointer did not properly increment in a round-robin fashion.
4. If there are "holes" in the read coverage of the AMEs (that is an AME is found to not have been read while those above and below it have been), this could indicate a corner case logic bug or other testsuite issue that needs to be addressed.

Currently, Covered simulates all memories/multi-dimensional arrays and its coverage is automatically accumulated during the score command. To see its report information, either specify the -m m option to the report command or bring up the GUI for interactive analysis.

1.2.3.2 Recommendations

For a design to pass full coverage, it is recommended that the write/read memory coverage for all modules in a design receive 100% coverage. If every AME was not both written and read during simulation, the design has not been fully exercised. Toggle coverage per AME need not receive 100% since a memory element that is written once will not cause toggle coverage to change.

1.2.4 Combinational Logic Coverage

1.2.4.1 Description

Combinational logic coverage answers the question, "What values did an expression (or subexpression) evaluate to (or not evaluate to) during the course of the simulation?"

This type of coverage is extremely useful in determining logical combinations of signals that were not tried during simulation, exposing potential holes in verification.

1.2.4.2 Example

Suppose the DUT comprised of the following module:

```
module test;

  reg  a;
  reg  b;
  wire c;

  initial begin
    a = 1'b0;
    b = 1'b1;
    #10;
    a = 1'b1;
    #10;
  end

  assign c = a | b;

endmodule
```

This module comprises of only one expression: a | b. If this module were simulated it would result in a combinational coverage value of 50% (2 out of 4). Can you determine which cases are not being covered?

The expression "a | b" can result in two values, 0 and 1, but can do so in four combinations:

1. a = 0, b = 0, c = 0
2. a = 0, b = 1, c = 1
3. a = 1, b = 0, c = 1
4. a = 1, b = 1, c = 1

Noticing the values assigned to a and b during simulation, shows that combinations (2) and (4) were hit during execution while combinations (1) and (3) were not (2 out of 4 - 50%). This also shows us that the value of c was never 0 during simulation.

1.2.4.3 Recommendations

For a design to pass full coverage, it is recommended that the combinational logic coverage for all modules be 80% or higher. If the expression coverage for an expression is not 100%, it is recommended that the verification engineer closely examine these missed cases to determine if more testing is required. Sometimes certain combinations of signals are unachievable due to design constraints, keeping the expression coverage from ever reaching a value of 100% but still can be considered fully covered.

1.2.5 Finite State Machine (FSM) Coverage

1.2.5.1 Description

Finite state machine (FSM) coverage answers the question, "Did I reach all of the states and traverse all possible paths through a given state machine?"

There are two types of coverage detail for FSMs that Covered can handle:

1. State coverage - answers the question "Were all states of an FSM hit during simulation?"
2. State transition coverage - answers the question "Did the FSM transition between all states (that are achievable) in simulation?"

1.2.5.2 Example

Suppose the DUT comprised of the following finite state machine:

```
module test( clock );

  input clock;
```

```
    reg [1:0] state;
    reg req, gnt;

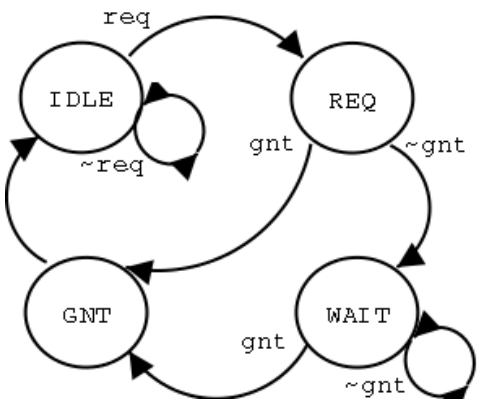
    parameter IDLE 2'b00;
    parameter REQ  2'b01;
    parameter WAIT 2'b10;
    parameter GNT  2'b11;

    initial begin
      req = 1'b0;
      gnt = 1'b0;
      repeat(4) @(posedge clock);
      req <= 1'b1;
      @(posedge clock);
      req <= 1'b0;
      gnt <= 1'b1;
      @(posedge clock);
      gnt <= 1'b0;
      repeat(2) @(posedge clock);
    end

    always @(posedge clock)
      case( state )
        IDLE : state <= req ? REQ : IDLE;
        REQ  : state <= gnt ? GNT : WAIT;
        WAIT : state <= gnt ? GNT : WAIT;
        GNT  : state <= IDLE;
        default : state <= 2'bx;
    endcase

  endmodule
```

To help visualize this state machine, let's also display it as a digraph.



Digraph Version of Example FSM

This state machine has four states: IDLE, REQ, WAIT, GNT. During simulation, 75% of the states were hit (3 out of 4) including IDLE, REQ and GNT. You can see that the WAIT state was never hit during simulation since the gnt signal was asserted the clock period after the req signal asserted.

Additionally, this state machine contains 7 state "arcs" which are represented in the digraph as arrowed lines. They are the following:

1. `IDLE -> IDLE  (hit)`
2. `IDLE -> REQ   (hit)`
3. `REQ  -> WAIT  (missed)`
4. `REQ  -> GNT   (hit)`
5. `WAIT -> WAIT  (missed)`
6. `WAIT -> GNT   (missed)`
7. `GNT  -> IDLE  (hit)`

Counting the number of traversed arcs during simulation, we can see that we have covered 4 out of 7 (or 57%) of the possible arcs in this state machine.

1.2.5.3 Recommendations

For a design to pass full coverage, it is recommended that the FSM coverage for all finite state machines in the design to receive 100% coverage for the state coverage and 100% for all achievable state transitions. Since Covered will not determine which state transitions are achievable, it is up to the verification engineer to examine the executed state transitions to determine if 100% of possible transitions occurred.

1.2.6 Assertion Coverage

1.2.6.1 Description

Assertion coverage answers the question, "Did I hit all of the possible coverage points of the assertions that I included in the design?"

Currently, Covered is capable of finding all OVL (Open Verification Language) assertion modules instantiated within the design (when the **-A ovl** option has been specified to the score command). Within most OVL assertion modules are one or more built-in coverage points. When the user has specified a particular assertion instance to check for a coverage point, Covered simulates this assertion module, keeping track of which coverage points within the assertion have been hit during simulation (and how many times each coverage point has been hit) and which have not. This allows the user to check for more complex coverage scenarios within the design.

Currently, only OVL version 1.6 (and newer) are supported. Older versions of the OVL may not work and are not planned to be supported.

1.2.6.2 Recommendations

For a design to pass full coverage, it is recommended that the assertion coverage for all modules in a design receive 100% coverage. If an assertion coverage point is not hit during simulation, the design has not been fully exercised. Assertion coverage is useful for determining more specific corner case holes in the test suite.

1.3 Coverage Boundaries

To properly evaluate the effectiveness of any coverage tool, it is necessary to understand what kinds of logic are able to be analyzed and which kinds of logic are unable to be analyzed (along with a reason for why the logic cannot be adequately covered). Understanding the limitations of tools up front can often guide the user of those tools to make better decisions in their usage. In the case of Covered and the open source community, it may also help drive ideas to how to correct or get around these limitations.

For these reasons, a list of what type of logic can be analyzed and what type of logic cannot be currently analyzed is listed below. Note that both of these lists may not be exhaustive and are subject to change between releases of the tool. Also note that it is the goal of Covered to be able to parse all of the Verilog-1995, Verilog-2001, and a subset of SystemVerilog constructs though some portions of these may be ignored for coverage purposes.

1.3.1 What logic can be analyzed?

1.3.1.1 Verilog-1995 constructs

1. All expressional logic which is comprised of the following data types and operators which exist in an assign block, always block, initial block, task, function or named begin/end block:
    - Nets: `wire, tri, tri0, tri1, wor, wand, trior, triand, supply0, suppl1`
    - Triregs: `trireg`
    - Registers: `reg`
    - Parameter values
    - String values
    - Integer values
    - Unary operators: `+, -, ~, &, |, ^, ~&, ~|, ~^`
    - Arithmetic operators: `+, -, *, /, %`
    - Logical operators: `&&, ||, !`
    - Relational operators: `>, <, >=, <=`
    - Equality operators: `==, ===, !=, !==`

- Bitwise operators: `&, |, ^, ~&, ~|, ~^`
- Shift operators: `<<, >>`
- Concatentation/replication operators: `{}, {{...}}`
- Bit select operators: `[], [:]`
- Conditional operator: `?:`

2. case, casex, casez statements
3. if, if/else statements
4. Delays when delay value is integer
5. Delayed procedural assignments (i.e., "`a = #5 b;`" or "`a = @(c) b;`")
6. Event waits and triggers: `wait, @(posedge), @(negedge), @(), ->`
7. Repetition statements: `for, while, repeat`
8. Fork/join statements
9. Memories
10. $finish and $stop system calls
11. $time and $random system functions
12. Real numbers
13. Real number system tasks: $rtoi, $itor, $realtobits, $bitstoreal
14. $test$plusargs system function
15. The following system tasks are allowed within logic blocks being considered for coverage:

    $display, $dumpall, $dumpfile, $dumpflush, $dumplimit, $dumpvars, $dumpoff, $dumpon, $fdisplay, $fmonitor, $fstrobe, $fwrite, $history, $key, $list, $log, $monitor, $monitoroff, $monitoron, $nokey, $nolog, $printtimescale, $showscopes, $showvariables, $showvars, $strobe, $timeformat, $write

1.3.1.2 Verilog-2001 constructs

1. `ifndef and `elsif directives
2. `file and `line directives
3. Constant functions calls
4. Inline parameter passing by name
5. Localparam support
6. Implicit event sensitivity lists: `@*`
7. Comma-separated event lists (alternative to the "`or`" keyword)
8. Signed value support
9. Immediate register assignment (i.e., `reg a = 1'b0`)
10. Variable multi-bit selects (i.e., `a[b+:3]`, `a[b-:3]`)
11. Exponential power operator: `a**b`
12. Arithmetic shift operators: `<<<, >>>`
13. Pre-port parameter listing (i.e., `module foo #(parameter a=0) (...);`)

14. Inline port listing (i.e., `module foo (input wire a, output reg b);`)
15. Attributes (note: Covered only performs attributes that it recognizes)
16. Arrays of instances
17. Generate blocks
18. "Automatic" or reentrant tasks/recursive functions
19. $value$plusargs system function call
20. The following system tasks are allowed within logic blocks being considered for coverage:

    $dumpports, $dumpportsall, $dumpportsoff, $dumpportson, $dumpportslimit, $dumpportsflush, $fclose, $ferror, $fflush, $swrite, $swriteb, $swriteh, $swriteo, $ungetc

1.3.1.3 SystemVerilog constructs

1. New integer types: `char, byte, bit, logic, int, shortint, longint`
2. Op-and-assign statements: `+=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, <<<=, >>>=, ++, --`
3. Proper support for `always_comb, always_latch, always_ff, final` blocks
4. `do..while` loops
5. Enumerations
6. Typedefs
7. $root scope support
8. Unnamed scope support
9. Multi-dimensional arrays including packed/unpacked dimensions and bit-select, part-select and slicing
10. Parsing support for `unique` and `priority` if/case labels
11. Parsing support for implicit .name ports (i.e., `foo f( .a, .b, .c );`)
12. Parsing support for implicit .* ports (i.e., `foo f( .* );`)
13. Parsing support for assertions, properties and sequences
14. Functional support for $urandom, $urandom_range, $srandom, $shortrealtobits, $bitstoshortreal

1.3.2 What logic cannot be currently analyzed?

1.3.2.1 Verilog-1995 constructs

1. All gate types (i.e., `buf, not, or, and,` etc.)
2. All user defined primitives (UDPs)
3. System task calls (or any $... call) that are not mentioned in the supported list above

1.3.2.2 Verilog-2001 constructs

1. Configuration statements

1.3.3 What is done with code blocks that contain code that cannot be analyzed?

When logic is encountered that cannot be analyzed within an always, initial, task, function or named block, Covered will disregard the entire always block. If the entire block cannot be analyzed, it does not try to get coverage statistics as these will most likely be incorrect (i.e., disregarded logic could affect the state of the logic that is being covered).

1.4 Race Condition Checking

Due to Covered's method of abstracting coverage from a dumpfile, a partial "resimulation" of the design is needed to obtain proper statistics for line, combinational logic and FSM metrics. Therefore, accurately resimulating what occurred in the actual simulation is a requirement for obtaining correct results. This can be easily achieved so long as the design doesn't contain any code that could result in potential race conditions.

1.4.1 Checked Coding Guidelines

To avoid this problem Covered performs automatic race condition checking on the specified design after parsing has been completed but before simulation/scoring is performed. All statement blocks in the design that don't adhere to certain coding guidelines are removed from coverage consideration by Covered. By following these coding guidelines, a design should be void of race condition scenarios that would lead to faulty simulation results. The following coding guidelines are applied by Covered when checking for race conditions:

1. All sequential logic and latches use non-blocking assignments.
2. All combinational logic in an always block uses blocking assignments.
3. All mixed sequential and combinational logic in the same always block uses non-blocking assignments.
4. Blocking and non-blocking assignments should not be used in the same always block.
5. Assignments made to a variable should only be done within one always block.
6. The $strobe system call should only be used to display variables that were assigned using non-blocking assignments. (Not currently implemented)
7. No #0 procedural assignments should exist. (Not currently implemented)

1.4.2 Race Condition Handling

If Covered detects a block as violating any one of the above mentioned coding guidelines, it will do one of two things depending on options specified to the score command by the user.

1. If the **-rS**, **-rW** or none of the -r options are specified, Covered will simply remove the non-conforming always block from coverage consideration and continue with scoring. **-rW** will display a message describing detected race conditions during the scoring command while the **-rS** option will display no messages.
2. If the **-rE** option is specified, Covered will halt the score command after all race condition checking has been performed with a message to the user specifying how many always blocks were found to be in non-compliance to the race-condition-free coding guidelines.

The first mode of operation is the recommended mode of operation as this will allow Covered to continue calculating coverage information for the design while still providing accurate coverage information for the logic that is still under consideration. The second mode of operation is meant to allow the user to use Covered as a race-condition checking tool.

1.4.3 Avoiding Race Condition Checking

Covered has three means of avoiding race condition checking: (1) skip race condition checking for the entire design altogether, (2) skip race condition checking for one or more specified modules, or (3) place inline pragmas around specific logic blocks to avoid checking. The main advantage for using either of these methods is to allow Covered to score more of the design that would otherwise be excluded due to failing a static race condition checkpoint. The main disadvantage to avoiding race condition checking using either of these methods is that you could potentially be ignoring code that contains an actual race condition, causing Covered to potentially generate incorrect coverage information without warning. These types of "errors" are not assumed to be Covered bugs.

To exclude the entire design from race condition checking, simply specify the **-rI** option on the score command-line. This will cause Covered to skip the race condition checking phase entirely, outputting a warning message (unless the global -Q option has been specified). The use of this command-line option is highly discouraged due to its general assumption by the user that the entire design is void of race conditions.

To exclude an entire module from race condition checking, simply specify the **-rI=*module name*** option on the score command-line. This option may be specified one or more times on the score command-line (once for each module to be skipped).

To exclude portions of the design from race condition checking, the user may also place inline comment-based pragmas in the design and specify the **-rP** option to the score command-line. The inline pragmas look like the following:

```
module foobar;

  ...

  // racecheck off
  always @(posedge clock) begin
    ...
  end
  // racecheck on

  ...

endmodule
```

This example will cause the initial block surrounded by the `racecheck` pragmas to be ignored from race condition checking, thus making it available for coverage consideration. If there are any race condition rules that this block contains or is a part of, they will not be reported.

The keyword `racecheck` is the default pragma keyword for race condition checking avoidance; however, if this name needs to be changed to something else by the user (i.e., if `racecheck` is used by another tool and has a different meaning or format), the user may make this keyword anything they please as long as the meaning is preserved (i.e., `keyword off` means that race condition checking should be turned off for all subsequent code until its matching `keyword on` pragma is seen). The new name of this keyword is passed to the -rP option by specifying **`-rP=keyword`**. It should also be noted that race condition pragmas can be embedded within each other (all code within the outside pair are removed from race checking) and race condition avoidance pragmas (`racecheck off`) are in effect on a file-basis -- that is, they are in effect even beyond their current scope but are no longer in effect after the current file.

1.4.4 Reporting Race Conditions

Even though race condition information is displayed (assuming the -rS option was not specified in the score command-line) during the scoring process, it may be convenient to view this information in a generated report as well. Because of this need, Covered saves all race condition information to the CDD file for use in using the report command. By specifying 'r' in the -m option to the report command (race condition report output is not turned on by default), the statement blocks which were eliminated from coverage consideration will be output to the coverage report file. Summary coverage will contain the total number of statement blocks eliminated for each module. Verbose coverage will contain each eliminated statement block, organized by module, specifying both the starting line of the eliminated statement block and the reason for why the statement block was removed.

1.5 Inline Attributes

1.5.1 What are inline attributes?

In the IEEE Verilog 1364-2001 standard, an attribute is a way to add information to a Verilog object, statement or groups of statements that is tool-specific and does not affect simulation of that design. All Verilog-2001 attributes begin with the token `(*` and end with the token `*)`. An attribute can be multi-line and is "attached" to the Verilog object, statement, or group of statements that is specified immediately beneath the attribute.

Covered uses the Verilog-2001 attribute for allowing users to specify coverage-specific information about embedded objects within a particular design. When an attribute is found, it is interrogated to see if it is a Covered attribute. If the attribute is a Covered attribute, its contents are parsed. If the attribute is not found to be a Covered attribute, it is ignored and parsing continues normally.

The rest of this chapter specifies the attributes that Covered is capable of handling, along with their use and syntax.

1.5.2 Adding FSM attributes

There are two ways that Covered currently allows the user to specify the location of and information about FSMs embedded in a particular design. The first way to specify an FSM is on the score command-line. The benefit to specifying the location of a state machine this way is that the source code does not need to be modified. The potential disadvantage to this method is that the FSM location and coverage information can get lost if the FSM is used in a different testbench (or even a different project if the FSM code is reused in a later project). For more information on specifying an FSM on the score command-line, please refer to Scoring FSMs.

The second way that an FSM can be specified to Covered is through the use of the Verilog-2001 attribute. The advantages to using this method are that the FSM information specified in an attribute stays embedded in the design (for ease of reusing the FSM and still retaining information relevent to coverage). Additionally, at the current release of Covered, using attributes to specify an FSM is the only way to tell Covered what all of the valid states and state-transitions are for a specific FSM (the command-line specification does not allow for this). This provides a unique advantage of this method over the command-line method. The potential disadvantage of this method for specifying FSM information is that source code needs to be modified.

To learn how to specify an FSM attribute within a design, let's use an example of an FSM that is embedded in a design.

```
module foo (
  input clk,
  input reset,
  input head,
  input tail,
  input valid
);

  parameter STATE_IDLE = 2'b00,
            STATE_HEAD = 2'b01,
            STATE_DATA = 2'b10,
            STATE_TAIL = 2'b11;

  reg [1:0] state;
  reg [1:0] next_state;

  always @(posedge clock)
    state <= reset ? STATE_IDLE : next_state;

  always @(reset or state or head or valid or tail)
    begin
      case( state )
        STATE_IDLE: next_state = (valid & head) ?
                                   STATE_HEAD : STATE_IDLE;
        STATE_HEAD: next_state = (valid & tail) ?
                                   STATE_TAIL : STATE_DATA;
        STATE_DATA: next_state = (valid & tail) ?
                                   STATE_TAIL : STATE_DATA;
        STATE_TAIL: next_state = (valid & head) ?
                                   STATE_HEAD : STATE_IDLE;
      endcase
    end

endmodule
```

This example shows an FSM that has an input FSM variable called "state" and an output FSM variable called "next_state". There are four states in the state machine that are represented with the parameters located in this module (STATE_IDLE, STATE_HEAD, STATE_DATA, STATE_TAIL). There are a total of eight (8) state transitions that this FSM can take. They are the following:

1. `STATE_IDLE -> STATE_IDLE` (loopback)
2. `STATE_IDLE -> STATE_HEAD`
3. `STATE_HEAD -> STATE_DATA`
4. `STATE_HEAD -> STATE_TAIL`
5. `STATE_DATA -> STATE_DATA` (loopback)
6. `STATE_DATA -> STATE_TAIL`
7. `STATE_TAIL -> STATE_HEAD`
8. `STATE_TAIL -> STATE_IDLE`

All attributes that specify information for an FSM are a comma-separated list of values that contain the following information:

1. "covered_fsm" attribute keyword
   - MUST be first value in the attribute list
   - Specifies to Covered that this attribute contains information for an FSM that Covered needs to handle.
2. FSM identifier
   - MUST be second value in the attribute list
   - Specifies a alphanumeric name for this FSM.
   - The name will eventually be used to tie individual attributes that specify information for the same FSM.
3. Input state expression (optional)
   - Syntax: `is="expression"`

- If this is specified, MUST be specified third in the list.
- Specifies the input state expression.
- Can be a combination of signal names, signal bit selects, and concatenation operators.
- See Scoring FSMs for more information on the specification of an input state expression.

4. Output state expression
   - Syntax: `os="expression"`
   - If the input state expression is specified, MUST be fourth value in list; otherwise, MUST be third value in list.
   - Specifies the output state expression of the FSM.
   - Can be a combination of signal names, signal bit selects, and concatenation operators.
   - See Scoring FSMs for more information on the specification of an output state expression.
5. State-transition specifiers (optional)
   - Syntax: `trans="from_state->to_state"`
   - MUST be specified after the above has been specified in the list.
   - Arguments MUST be constant values (parameters; numerical values -- binary, octal, decimal, hexidecimal; and defines that equate to one of these two types of values).
   - Each transition that is specified is a unique value in the attribute list.
   - You may optionally add characters after the "trans" keyword and before the '=' character (with the exception of the '=' character) to make each transition parameter name unique. Covered will ignore these extra characters, but this can be useful to avoid warning messages that may get emitted from your Verilog compiler when these attributes are parsed (some compilers expect each parameter name in an attribute list to have a unique name). This feature was put in the 0.7.8 stable release.

To specify the FSM attribute in the above example, including input state, output state and all state transitions, the code would be modified to look like:

```
(* covered_fsm, channel, is="state", os="next_state",
                         trans="STATE_IDLE->STATE_IDLE",
                         trans="STATE_IDLE->STATE_HEAD",
                         trans="STATE_HEAD->STATE_DATA",
                         trans="STATE_HEAD->STATE_TAIL",
                         trans="STATE_DATA->STATE_DATA",
                         trans="STATE_DATA->STATE_TAIL",
                         trans="STATE_TAIL->STATE_HEAD",
                         trans="STATE_TAIL->STATE_IDLE" *)
always @(reset or state or head or tail or valid)
  ...
```

2 Installation

2.1 Installation

2.1.1 Prerequisites

The goal of Covered was to create a tool that has very few requirements to allow it to be compiled and run on all platforms. The following is a list of all utilities/resources required to compile Covered. However, Covered is primarily developed on RedHat/Fedora Core Linux and Mac OS X and as such is primarily tested by the developers on these platforms.

- gcc (Version 2.96 or higher)
- flex (Version 2.5.4 or higher)
- bison (Version 1.28 or higher)
- GNU make (Version 3.27.1 or higher)

- GNU gperf (Version 2.7.2 or higher)
- Tcl/Tk (Versions 8.4 or higher)
    - tablelist Tcl/Tk widget (Version 4.9 or higher)

Covered is primarily developed on Fedora Core but has been known to compile with both 32-bit and 64-bit compilers that contain the general GNU library of tools. It has also been compiled and run on Cygwin and Mac OS X.

2.1.2 Downloading source

To download the source files in .tar.gz format, go to the following URL:

http://covered.sourceforge.net/downloads/

Double-click on the .tar.gz version that you wish to install. The browser should immediately bring up a window asking if you would like to save to disk or open immediately. Choose the "Save to Disk" option. Another window will pop up, allowing you to choose the directory to store the tarball in. After selecting a directory, click the "Save" button. The download utility will begin downloading the tarball to the selected directory.

Once you have downloaded a tarball, enter the following to unzip and untar the package:

```
> gzip -dc tarball_filename | tar xvf -
```

This will create a directory called covered-*version*, where *version* will be of the form *major version*.*minor version* for stable releases or *date* for development releases. Enter that directory to begin configuring, building and compiling Covered from source.

2.1.3 Configuring the build

At this point, you are ready to configure the build environment to create the executables, libraries and/or documentation for Covered. However, before you configure the build environment, you will need to make a few decisions which will lead to different options given to the configure script. These questions are the following:

1. Do you need to debug Covered? (for most users, the answer to this will be "no")

   If the answer to this question is yes, configure the build environment with the **--enable-debug** option to the configure script. Doing so will allow you to use the global -D option and/or the -cli (Command-Line Interface (CLI)) score command option for debugging; however, it will also have the side effect of causing the score command to run noticably slower. For normal usage of Covered, this option should not be specified when configuring.

2. Do you want to profile Covered's internal functionality? (for most users, the answer to this will be "no")

   If the answer to this question is yes, configure the build environment with the **--enable-profiling** option to the configure script. Doing so will allow you to use the global -P option to generate an output file containing internal profiling information for the run command. See Source Code Profiling for more information on the -P option.

3. Will you being using Covered as a VPI module and, if so, what simulator(s) will you need VPI modules built for?

   If the answer to this question is yes and you need to build a VPI module for the Icarus Verilog simulation tool, simply add **--with-iv** on the configure command-line. If a VPI shared object module is needed for the Cver simulator, simply add **--with-cver=cver_include_dir** on the configure command line, where *cver_include_dir* specifies the directory that contains the vpi_user.h file. If a VPI shared object module is needed for the VCS simulator, simply add **--with-vcs=vcs_include_dir** on the configure command line, where *vcs_include_dir* specifies the directory that contains the vpi_user.h file.

4. Do you want to use Covered's Tcl/Tk GUI for interactive coverage analysis or just the command-line created reports?

   If the answer to this question is yes and you have a Tcl/Tk distribution that the configure script cannot find (i.e., the tcl-config and tk-config scripts exist in a non-standard directory location), please specify the **--with-tcl-config=tcl-config_pathname** and **--with-tk-config=tk-config_pathname** options to the configure script. If the answer to this question is no, specify the **--with-tcltk=no** option to the configure script to cause the GUI to not be built.

Once you have determined which configuration options you need, simply enter the following command:

```
> ./configure options
```

This command will create the Makefiles/include files necessary for your machine to compile the source code correctly. If any errors are generated during this command run, it is usually an indication that some program or library is missing from your computer that is necessary for Covered to compile/run. Please install any missing programs/libraries and type this command again.

2.1.4 Compiling and Installing

If the configure script completes successfully, you may compile the source with the following command:

```
> make
```

This will begin the process of compiling/linking the source code for Covered. This stage may take a while. When the source files have been compiled, the covered executable file will have been created. To install this executable in your `/usr/local/bin` directory, enter the following command:

```
> make install
```

This will install the covered executable and manpage help files. Once covered is installed, make sure that `/usr/local/bin` is in your environment path. You are now ready to run Covered.

3 Command-line Usage

3.1 Getting Started

3.1.1 Methods for scoring the design

Before you are ready to being scoring, merging and generating coverage reports from the design, you will first need to decide whether you want to post-process VCD/LXT2/FST -formatted dumpfiles generated from simulation, co-process VCD/LXT2/FST -formatted dumpfiles using two processes (one for the simulation process and one for the Covered process) and a Unix FIFO, or whether you want to accummulate coverage information alongside the simulator using the simulator's VPI interface. There are advantages and disadvantages to these methods, so select the method that will best work for you.

3.1.1.1 Post-processing VCD/LXT2/FST dumpfiles

Generating post-process coverage information from VCD/LXT2/FST -formatted dumpfiles requires three steps:

1. Compile the simulator to dump in the specified format.
2. Run the simulation.
3. Run Covered's score command using the dumpfile as input along with the design.

Performing these steps will generate the needed CDD file that can be used for merging or reporting. The advantages to this approach is that the steps are fairly simple and are compiler/simulator -independent. However, there are two main drawbacks. First, compiling with dumping enabled often slows simulation by some significant factor. Second, the dumpfiles generated from simulation can often be quite large (especially for VCD dumping), consuming a lot of disk space. Additionally, if your simulator dumps in a format different from VCD, LXT2 or FST, you will need to perform an additional step in transforming the original dumpfile into one of these formats. This is often a time-consuming task if the design and/or dumpfile is significantly large. If you are generating lots of dumpfiles for coverage (say from running a regression), the amount of disk space needed to store these files can be tremendous, making this approach almost not feasible.

3.1.1.2 Co-processing VCD/LXT2/FST dumpfiles via FIFO

Generating co-process coverage information from VCD/LXT2/FST -formatted dumpfiles requires four steps:

1. Compile the simulator to dump in the specified format.
2. Create a Unix FIFO via the `mkfifo fifoname` command.
3. Create an empty CDD file for the design using Covered's score command.
4. Run the simulation and Covered's score command simultaneously. The following is an example of a testbench and command-line commands:

```
# Module listing for file "test.v"
module tb;
  ...
  initial begin
          $dumpfile( "dump_fifo" );
          $dumpvars( 0, tb );
          ...
  end
```

```
  ...
endmodule

# Command-line output
> mkfifo dump_fifo
> vcs test.v
> covered score -t tb -v test.v -o test.cdd
> ./simv &
> covered score -cdd test.cdd -vcd dump_fifo
```

The end result of these steps is the same CDD file that results from using the dumpfile method described above. The primary advantages of this approach are that no dumpfiles are actually created, saving on disk space, and it does not significantly slow down the simulation process. Because both simulation and coverage scoring occur simultaneously, there should be a wall clock time improvement with this method.

The drawbacks of this approach are basically that the time improvement won't be significant if you are running on a single processor/core system and that it is restricted to machines that support the Unix fifo (this may not be an actual restriction, however).

3.1.1.3 Using the VPI interface to gather coverage information

Generating coverage information using the VPI interface of the simulator requires three main steps:

1. Create a base CDD file from the design along with a top-level Verilog module and PLI table file (this second file is needed for the commercial VCS compiler).

2. Compile the simulator using the Covered VPI module (or shared object).

3. Run the simulation.

The end result of these steps is the same CDD file that results from using the dumpfile method described above. The primary advantage of this approach is that no dumpfiles are required, saving on disk space and dumpfile processing time. The drawbacks of this approach are the following:

1. Simulation runs much slower using the VPI and Covered (most likely slower than creating dumpfiles from the simulator).

2. Covered VPI modules are only available for certain compilers (Icarus Verilog, Cver and VCS currently); however, adding support for other compilers should not be a difficult thing to do.

3. The VPI modules will only work for compilers that support the VPI interface (a Verilog-2001 feature).

3.1.2 What is needed for dumpfile scoring?

Before Covered can be invoked for dumpfile scoring, you must have a simulatable Verilog design and a VCD, LXT2 or FST dumpfile containing information from a simulation run of the design that dumps the module(s) that you want to check for coverage. The VCD dumpfile style was chosen due to its universal support by Verilog simulators while the LXT2 and FST dumpfile styles were chosen due to their compactness and growing support by other open source simulators and dumpfile readers. Once you have these two parts, you are ready to begin generating coverage results.

3.1.2.1 Selecting What to Dump

If coverage is being scored via a VCD/LXT2/FST dumpfile, it is necessary for the user to setup $dumpfile and $dumpvars calls to dump the various modules that are being scored. Covered allows the user to do this in one of two ways.

First, the user may hand create these calls within their testbench. If this approach is taken, it is necessary that all modules being scored by Covered be included in the $dumpvars calls. Eliminating scored modules from the dumpfile will result in inaccurate coverage information. Additionally, it is mandatory that the dumpfile start at time 0 and that all dump information be included in the dumpfile (i.e., using the $dumplimit system task could cause dump information to be lost that will result in inaccurate coverage results).

Second, the user may take advantage of Covered's -dumpvars option to the score command. Adding this parameter (and the optional -top_ts option) will create a top-level module file that can be compiled along with the testbench that will dump all of the parts of the Covered design that are needed by Covered. Using this approach will result in the best overall performance for dumpfile coverage accumulation as all unnecessary dump information will be stripped, resulting in both faster simulation and faster coverage scoring.

If the second approach is taken, simply use the score command to parse the design and output an unscored CDD and the dumpvars module, using a command similar to the following:

```
covered score -t top -v dut.v -dumpvars dut_dump.v -o dut.cdd
```

If `timescale is specified at the top of dut.v, it may be necessary for compiling purposes to have the timescale specified at the top of dut_dump.v. To do this, simply specify the -top_ts option in the score command. Note that the example command does not list either the -vcd, -lxt or -fst options. If either of these options are present on the score command-line, the dut_dump.v file will not be created and the score command will attempt to score the design with the specified dumpfile.

After the dut_dump.v file has been created, simply compile it along with your DUT as a top-level module. For example,

```
iverilog dut.v dut_dump.v
```

After the simulation has been compiled and run, a dumpfile called "dut_dump.vcd" (using the above example) will exist in the run directory. This dumpfile can then be used in a second call to the score command via the -vcd, -lxt or -fst options along with the unscored CDD file to create a scored version of the CDD file.

3.1.2.2 Creating a VCD dumpfile

By default, the $dumpfile system task call will create a file and populate it with VCD-style dumpfile information. Simply use the resulting VCD dumpfile in the -vcd option to the score command to obtain coverage for that design.

3.1.2.3 Creating an LXT2 dumpfile

An LXT2 dumpfile can be created in several different ways depending on the Verilog simulation tool that you are using. If you are using the Icarus Verilog open source simulator, you can simply generate an LXT2-style dumpfile by specifying the -lxt2 option to the simulator command-line. For example, if you had a file called "foo.v" that contained the same $dumpfile and $dumpvars commands used for VCD dumping and compiled it with Icarus Verilog into a VVP file called "a.out", you could cause Icarus Verilog to generate an LXT2 dumpfile (instead of a VCD dumpfile) by calling "vvp a.out -lxt2". This will cause an LXT2 style dumpfile instead of the standard VCD style dumpfile.

You can also transform many different dumpfile formats into an LXT2 style dumpfile with the helper programs that come with the GtkWave waveform viewer.

3.1.2.4 Creating an FST dumpfile

An FST dumpfile can be created in several different ways depending on the Verilog simulation tool that you are using. If you are using the Icarus Verilog open source simulator, you can simply generate an FST-style dumpfile by specifying the -fst option to the simulator command-line. For example, if you had a file called "foo.v" that contained the same $dumpfile and $dumpvars commands used for VCD dumping and compiled it with Icarus Verilog into a VVP file called "a.out", you could cause Icarus Verilog to generate an FST dumpfile (instead of a VCD dumpfile) by calling "vvp a.out -fst". This will cause an FST style dumpfile instead of the standard VCD style dumpfile.

You can also transform many different dumpfile formats into an FST style dumpfile with the helper programs that come with the GtkWave waveform viewer.

3.1.3 What is needed for VPI scoring?

After Covered has been configured and built to include the creation of a VPI loadable module or shared object file (see Configuring the build for details), Covered is capable of scoring during a simulation run by using the VPI (Verilog Procedural Interface) access mechanism. This is done by loading the Covered VPI module (or shared object) into the simulator executable prior to running (depending on the simulator being used). The following steps should be taken to create a scored CDD file using this method.

1. Create the Verilog file that will be compiled as a top-level module in the design (alongside the actual top-level(s)).

   This is done by running the score command with the -vpi option. The following example command-line shows how this step is done, creating a file called "covered_vpi.v".

   ```
   > covered score -t top -vpi
   ```

2. Compile the simulator executable, including the previously generated file and the Covered VPI module. See the compile instructions for the Icarus Verilog, Cver or VCS simulators below.

3. Run the simulation. Once simulation is complete, the resulting CDD file will be a completely scored database, ready to be merged with other CDD files from the same design or reported on.

3.1.3.1 Compiling for Icarus Verilog

If you are compiling an Icarus Verilog simulation, simply add **`-m tool_install_root/libexec/covered.vpi covered_vpi.v`** to the command-line.

3.1.3.2 Compiling for Cver

If you are compiling a Cver simulation, simply add **`+loadvpi=tool_install_root/libexec/covered.cver.so:vpi_compat_bootstrap covered_vpi.v`** to the command-line.

3.1.3.3 Compiling for VCS

If you are compiling a VCS simulation, simply add **`+vpi -load tool_install_root/libexec/covered.vcs.so:covered_register covered_vpi.v`** to the command-line.

3.1.3.4 Compiling for NC-Verilog

If you are compiling a NC-Verilog simulation, switch to NC-Verilog's irun command to load the covered shared object: **`-loadvpi tool_install_root/libexec/covered.ncv.so:covered_register`** and enable all access with **`-access +rwc`**.

You can hardcode the $covered_sim call into your RTL or you can run it dynamically using the CLI, by adding the **`-input input.tcl`** switch to irun. Where the input.tcl file looks like the following and tb.dut is the coverage instance:

```
call -systf {$covered_sim} {"scored.cdd"} tb.dut
run
```

3.2 Using Covered

3.2.1 Work Flow

Covered has four basic commands that it uses for gaining coverage information about a design and displaying that information to the user.

- score
- merge
- report
- rank

3.2.1.1 Scoring the Design

To begin using Covered, you will need to create a special database file called a coverage description database (CDD). Covered uses this file to store coverage-specific information about the DUT. This file is a text-based file that is written in a specific format (the format of this file is not discussed in this document but can be found in the developer's document) that Covered understands.

To create the CDD and populate it with the simulation dumpfile results, you must use the score (see The score Command for details) command. This command parses in the DUT files and generates an initial CDD for the design. This CDD does not contain any coverage details but only the design elements that Covered needs to gain coverage results.

Once the initial CDD has been created, the score command reads in the specified VCD/LXT2 dumpfile from the given DUT. If Covered recognizes that the VCD/LXT2 dumpfile does not correspond to the DUT that it parses (dumpfile was not generated from the DUT specified), an error message is supplied to the user telling them this. Once the dumpfile has been parsed, Covered generates a new version of the CDD containing a populated database. This CDD may be merged with another CDD from the same DUT and/or reports may be generated from this CDD.

3.2.1.2 Merging CDDs

When two or more CDDs have been generated from the same DUT. It is often desirable to merge their contents to see what the total coverage is for both simulation runs. The reasons why merging is desirable is that often several tests are written to exercise the entire design (not just one). This makes tests easier to write and maintain. Since, typically, only one test is run per simulation run and that test does not fully test the DUT, it is necessary to combine the results of the multiple simulations to the see the combined effect of the tests on the design. This is where merging comes into play.

Merging multiple CDDs from the same design is easy with Covered, and is accomplished with the merge (see The merge Command for details) command. This command reads in two or more CDDs, merges their results and outputs the merged CDD to a different file or replaces the first CDD with the results of the merge.

3.2.1.3 Creating Coverage Reports

After a CDD is created or merged, the contents of the CDD can be extracted and transformed into human-readable coverage reports with the report (see The report Command for details) command. A summary report containing the percentage of metrics covered is generated for each module. Additionally, to aid in understanding what and why something was not covered, a verbose reporting mechanism exists. Verbose reporting will explicitly point to and describe why something was not fully covered for a specific metric.

Reports are text-based output that by default are directed toward standard output but may, with the use of an option flag, be output to a specific file for storage purposes.

3.2.1.4 Ranking CDDs

After a collection of CDDs have been generated from a single design (most likely due to a regression run), it is desirable to know which CDDs have an effect on coverage and which CDDs do not add any new coverage information. The CDDs that do not provide additional coverage may be unnecessary for regression and removing them may significantly improve a regression runs time (which can help in a projects schedule).

Additionally, it also good to know an order that simulations should be run in to produce the maximum amount of coverage the quickest. This information can be useful for creating a quick (or "sanity check" or "confidence") regression that runs in a relatively short period of time and, when run successfully, provides the engineer a level of confidence that recent changes have not severely broke the functionality of the design.

This is the purpose of the rank (see The rank Command for details) command. It produces a report that shows the diagnostics that add to the overall coverage of the suite, the order that those diagnostics should run, and each diagnostic's overall effect on coverage when it is run.

3.2.1.5 Excluding CDD Coverage Points

Not all unhit coverage points reported by Covered will be valid for your design. Some coverage points may be extraneous, some may be due to obsolete logic that is not pulled from the design for various reasons, and some coverage points may be hit in other verification environments but not in the current environment.

Whatever the reason may be (and some reasons may be much more specific that these suggestions), Covered allows the user to exclude those coverage points from coverage reporting (they will still be reported but they will be considered "covered" points instead of "uncovered" points). Additionally, each excluded coverage point can have an option message attached to it that explains the reason for exclusion. This message is stored with the coverage information in the CDD file and can be displayed in a standard coverage report or in a stand-alone exclusion report. This information helps to document the coverage analysis process.

In addition to being able to exclude coverage points, the user can also include previously excluded coverage points (in the event that a mistake was made or if the design has changed). Coverage exclusion can be accomplished via the command-line exclude command or within the GUI coverage analyzer.

3.2.2 Covered usage

Covered is initiated with the following command:

```
covered (-h | -v | [-P [filename]] [-B] [-D|-T|-Q]) (score|merge|report|rank) command_options)
```

The -v command (described below) is mutually exclusive from the rest of the commands and has the highest priority. If Covered is called with this option and any other options, the version will only be displayed.

The -h command (described below) is mutually exclusive from the rest of the commands and has the second highest priority. If Covered is called with this option and the -v option is not specified, the usage information will be displayed to the screen.

The -Q option suppresses all normal user output from standard output. The only output that will not be suppressed are messages to standard error. This option may be used with any of the Covered commands to achieve this effect. By default, normal user output is sent to standard output.

The -T option suppresses all normal user output from standard output with the exception of the Covered header banner, warning messages. This type of output is a bit more informative than output generated with the -Q option but not as much as running without any output limiters.

The -D option outputs debug information to standard output. Its effect overrides the -Q option. This option may be used in conjunction with any other Covered command for the purposes of debugging tool failures. By default, debug mode is turned off and must be enabled in the build of Covered (using the --enable-debug option to configure). This command should only be used for debugging as it generates an immense amount of output which may hinder performance.

The -B option obfuscates all design-sensitive signal, module, task, function, instance, file and parameter names when outputting them to either stdout, stderr or a file (with the exception of CDD file output). This mode is intended to be used for sharing output information with others (including the developer's of Covered) which is sensitive information for the purposes of debugging.

The -P option causes Covered to enable internal code profiling and generates a profiling report that is output to either the given filename or the default filename of "covered.prof". This option is only available if the --enable-profiling configuration option is specified and will cause performance degradation. This option should only be used by Covered developers or those interested in finding performance bottlenecks in Covered's code. It is not recommended that the --enable-profiling configuration option and the -P global option be specified for normal users of Covered. See Source Code Profiling for more information on profiling and understanding the profiling report file.

3.2.3 Covered global options

The following table describes options that Covered will accept apart from its normal commands which are described in the next several chapters.

Global Options to Covered

Option	Description
-Q	Quiet mode. Suppresses user output to screen. Use in conjunction with a command.
-T	Terse mode. Causes all output to be suppressed with the exception of warning messages and the Covered header information. Use in conjunction with a command.
-D	Debug mode. Outputs code debug information to screen. Use in conjunction with commands.
-B	Obfuscate. Obfuscates all design sensitive names before outputting them.
-P [*filename*]	Profiling mode. Outputs profiling report to either the given *filename* or "covered.prof" if no *filename* was specified.
-h	Generates usage output to standard output.
-v	Outputs current version of Covered.

3.3 The score Command

3.3.1 Usage

The score command is initiated with the following call:

```
covered [global_options] score –t top_module [options]
```

The *top_module* parameter is required for the scoring purposes and represents the top-most module that will be covered. It is important to note that Covered does not need to parse the entire DUT. It only parses that part of the design which will receive coverage results. More on this will be described Specifying What to Cover below.

3.3.2 Options

The following table describes the additional options that can be passed to the score command. Note that it is optional in how you tell Covered where to find the Verilog design files; however, it is not optional to exclude the Verilog files. If no Verilog can be found that contains at least the top instance, Covered will generate an error message to the user specifying that it was unable to find the required Verilog modules.

Options to score Command

Option	Description
-A ovl	Causes OVL assertions to be used for assertion coverage. This flag must be given to the score command if assertion coverage metrics are needed in the report command. See Assertion Coverage.
-cli [*filename*]	Causes the command-line debugger to be used during VCD/LXT2/FST dumpfile scoring. If *filename* is specified, this file contains information saved in a previous call to savehist on the CLI and causes the history contained in this file to be replayed prior to the CLI command prompt. If *filename* is not specified, the CLI prompt will be immediately available at the start of simulation. This option is only available when Covered is configured with the --enable-debug option. See Command-Line Interface (CLI).
-conservative	If this option is specified, any logic blocks that contain code that would cause coverage discrepancies leading to potentially inaccurate coverage results are removed from coverage consideration. See The reason for the -conservative feature for more information.s
-D *name*[=*value*]	Defines the specified name to 1 or the specified value.
-dumpvars [*filename*]	If this option is specified without the -vcd, -lxt or -fst options, the design is parsed, a CDD file is created and a top-level Verilog module named *filename* (if this value is specified) or "covered_dump.v" (if filename is not specified) is created. If compiled as a top-level module along with your design and no other $dumpfile/$dumpvars calls are made, this dump module will provide additional performance gains. If either the -vcd, -lxt or -fst options are specified, this option has no effect. See Selecting What to Dump.

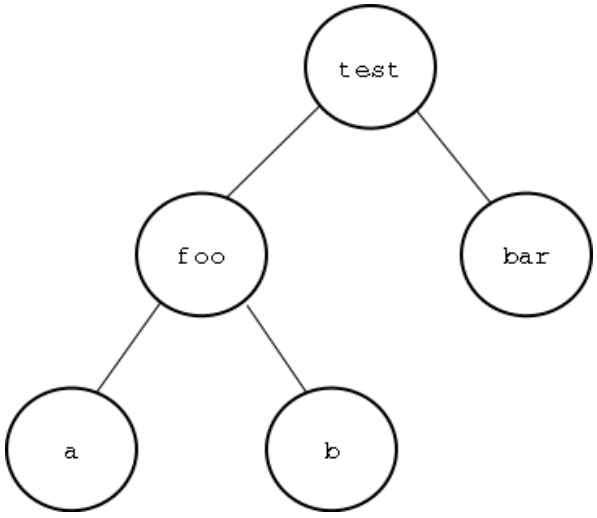
Option	Description
-e *module*	Name of module, task, function or named begin/end block to not score. You may specify this option more than once on a command-line; one for each block to remove from coverage. See Specifying What Not to Cover (Coverage Exclusions).
-ea	Specifies that all always blocks in the design should be excluded from coverage consideration. This option may help to increase performance of the score command if all always blocks in the design do not need to be checked for coverage. By default, always blocks are considered for coverage. See Specifying What Not to Cover (Coverage Exclusions).
-ec	Specifies that all continuous assignments in the design should be excluded from coverage consideration. This option may help to increase performance of the score command if all assign blocks in the design do not need to be checked for coverage. By default, continuous assignments are considered for coverage. See Specifying What Not to Cover (Coverage Exclusions).
-ef	Specifies that all final blocks in the design should be excluded from coverage consideration. This option may help to increase performance of the score command if all final blocks in the design do not need to be checked for coverage. By default, final blocks are considered for coverage. See Specifying What Not to Cover (Coverage Exclusions).
-ei	Specifies that all initial blocks in the design should be excluded from coverage consideration. This option may help to increase performance of the score command if all initial blocks in the design do not need to be checked for coverage. By default, initial blocks are considered for coverage. See Specifying What Not to Cover (Coverage Exclusions).
-ep [*name*]	Causes any code that is inbetween inline coverage pragmas (eg. '// coverage off' ... '//coverage on') to be excluded from coverage consideration. The code specified between these pragmas is not removed from resimulation but is marked as being excluded from coverage for reporting purposes. These exclusions may be undone via the GUI, if necessary, without requiring the design to be resimulated by Covered. If *name* is specified, its value will be used by Covered as the pragma keyword to search for in the design (instead of the default value of 'coverage'). This feature can be useful in the event that other tools being used on the design use a similar pragma name and there are tool conflicts. See Specifying What Not to Cover (Coverage Exclusions).
-F *module*=[*in_expr*,]*out_expr*	Specifies the location to find an FSM to score where *module* is the Verilog module containing the FSM, *in_expr* is the input state expression and *out_expr* is the output state expression. If *in_expr* is not specified, out_expr is used as both the input and output state expression.
-f *filename*	Name of file containing additional arguments to parse. You may specify this option more than once on a command-line.
-fst *dumpfile*	Name of FST dumpfile to score design with. If -vcd, -lxt or this option is not used, Covered will only create an initial CDD file from the design and will not attempt to score the design.
-g [*module*=](1|2|3)	Specifies what generation of Verilog to use for parsing the design. If *module* is specified, uses the given generation value for that module only. If *module* is not specified, applies that generation to the entire design. This option may be specified multiple times on the command-line, where options to the left take precedence over the values to the right. 1=Verilog-1995, 2=Verilog-2001, 3=SystemVerilog.
-h	Outputs usage information for the score command.
-I *directory*	Directory to find included Verilog files. You may use this option as many times as necessary on the command-line, one for each directory.
-i *instance*	Instance name of top-level module. Necessary if module to verify coverage is not the top-level module in the design. If not specified, -t value is used. See Specifying What to Cover.
-lxt *dumpfile*	Name of LXT2 dumpfile to score design with. If -vcd, -fst or this option is not used, Covered will only create an initial CDD file from the design and will not attempt to score the design.
-m *message*	Allows the user to specify information about this CDD file. This information can be anything (messages with whitespace should be surrounded by double-quotation marks), but may include something about the simulation arguments to more easily link the CDD file to its simulation for purposes of recreating the CDD file.
-o *filename*	Name of CDD to write coverage information to.
-p *filename*	Overrides filename used for outputting intermediate preprocessor output. See Other Notes.
-P *parameter_scope=value*	Performs a defparam on the specified parameter with value. See Overriding Parameters (-P option).
-rS	Performs race condition checking and removes any offending logic blocks from coverage consideration without reporting the information. See Race Condition Handling.
-rW	Performs race condition checking, removes any offending logic blocks from coverage consideration, and displays this information to standard output. This is the default race condition checking behavior. See Race Condition Handling.
-rE	Performs race condition checking, reports any offending logic and immediately stops scoring. This option allows Covered to be used as a static race condition checking tool. See Race Condition Handling.

Option	Description
-rI[=*module name*]	If =*module name* is not specified, race condition checking is not performed for the entire design. If =*module name* is specified, race condition checking is not performed on the specified module only. This option may be specified more than once. See Avoiding Race Condition Checking.
-rP[=*name*]	Skip race condition checking for all code surrounded by "// racecheck off/on" embedded pragmas. The "racecheck" keyword can be changed by specifying =*name* where *name* is the new name for the race condition pragma keyword. See Avoiding Race Condition Checking.
-S	Outputs simulation statistics after simulation has completed. This information is currently only useful for the developers of Covered.
-top_ts (1|10|100) (s|ms|us|ns|ps|fs)/(1|10|100) (s|ms|us|ns|ps|fs)	This option is only valid when the -vpi or -dumpvars options have been specified. This option allows the user to specify a timescale for the generated Verilog module created with the -vpi or -dumpvars options. If this option is not specified, no timescale will be created for the generated module. If whitespace is needed between the various values, place the entire contents of timescale in double quotes.
-ts *number*	When scoring occurs, this option allows the user to see how far the simulator has progressed by outputting the current timestep to standard output. The value of *number* specifies how many timesteps are allowed to be simulated before outputting the current timestep (results in less calls to output stream).
-T (min|typ|max)	Specifies which value to use in min:typ:max delay expressions. Default is typ.
-v *filename*	Name of specific Verilog file to score. You may specify this option more than once on a command-line; one for each filename to parse.
-vcd *dumpfile*	Name of VCD dumpfile to score design with. If -lxt, -fst or this option is not used, Covered will only create an initial CDD file from the design and will not attempt to score the design.
-vpi [*filename*]	If this option is specified without the -vcd, -lxt or -fst options, the design is parsed, a CDD file is created and a top-level Verilog module named *filename* (if this value is specified) or "covered_vpi.v" (if filename is not specified) is created along with a PLI table file called *filename*.tab or "covered_vpi.v.tab". Both of these files are used in the compilation of the simulator to use Covered as a VPI module. If either the -vcd, -lxt or -fst options are specified, this option has no effect. See Creating an LXT2 dumpfile.
-Wignore	Suppresses any warning output during code parsing and/or simulation.
-y *directory*	Directory to find unspecified Verilog files. You may specify this option more than once on a command-line; one for each directory required.
+libext+.extension[+.extension]*+	Extensions of Verilog files to allow in scoring.

3.3.3 Specifying What to Cover

Covered is not a logic simulator and, therefore, is not required to parse the entire DUT. True simulation is performed by the Verilog simulator used to generate the VCD/LXT2/FST dumpfile. Covered, therefore, is given the ability to parse only the part of the DUT that is to be scored for coverage. Having this ability to ignore parts of the DUT has several advantages in the way of performance. Less memory is necessary for coverage simulation/ calculation. Additionally, less processing is also required which leads to faster turnaround. On the other hand, since Covered ignores the parts of the design not being scored, some information is not automatically extracted by Covered that is required for scoring the specified part of the DUT. The *defparam* statement is one such statement. More on this is mentioned in the following section.

The following example describes a complete DUT and how to cause Covered to score various parts of the DUT.



Hierarchical Tree View of Example DUT

Assume that the following Verilog modules are used to create this tree.

- module "test" instantiated as "test" in the above figure
- module "foomod" instantiated as "foo"
- module "barmod" instantiated as "bar"
- module "amod" instantiated as "a"
- module "bmod" instantiated as "b"

To get coverage for the entire design, the covered command line would be:

```
covered score -t test -i test (rest of command-line)
```

Because both the -t and -i options are the same value (the module name is the same as the instance name), only the -t option needs to be specified. If the -i option is not specified, Covered will assume that the instance name of the top module is the same name as the top module itself. Note that the -t option always needs to be specified for the score command.

To get coverage for the instance foo and all modules below it, the covered command line would be:

```
covered score -t foomod -i test.foo (rest of command-line)
```

In this case, we do not wish to score the module test (the top-level file for the design) but rather wish to gain coverage information for module foomod and all instantiated modules underneath it. Since foomod is not the top-level design file, the -i option must be specified to tell Covered what the hierarchical scope of the top-level score file is in relation to the entire design. The main reason why this is necessary is so that Covered will be able to locate the dumpfile information for the modules/instances that need to be scored.

To get coverage for only the instance foo (and none of the modules instantiated below it in the hierarchy), Covered command-line would look like the following:

```
covered score -t foomod -i test.foo -e amod -e bmod
```

Here we see the use of the -e option, it takes a module name as its value. When a module with the matching name is found in the hierarchy, it and all of its submodules are removed from coverage. You may specify as many -e arguments as necessary to select the appropriate modules to cover.

To get coverage for the instance foo and all modules below it, in addition to the instance bar, you would have to run Covered twice (once for each top instance).

The -vcd *vcd_dumpfile*, -lxt *lxt_dumpfile* or -fst *fst_dumpfile* parameter is simply the name of the VCD/LXT2/FST dumpfile that contains all of the module instances selected by the -t parameter (in combination with the -e option -- see below).

3.3.4 Specifying What Not to Cover (Coverage Exclusions)

Besides the -e option (scope exclusion), Covered comes equipped with several other means of excluding codes from coverage consideration. The -ea, - ec, -ei and -ef exclusion options cause Covered to not simulate always blocks, continuous assignment blocks, initial blocks and final blocks, respectively, allowing for additional score command performance enhancement opportunities. These options affect all code that is under coverage consideration for the current score command, and cannot be turned on/off on an individual basis.

3.3.4.1 Using the -ep Option for Exclusion

The -ep exclusion option allows Covered to exclude all code that has user-specified coverage pragmas embedded inline in the code being considered for coverage. By default, these pragmas look like the following:

```
module foo;

  // coverage off
  ...
  // coverage on

endmodule;
```

Any registers, nets, variables, blocks, expressions, FSMs or assertions found within these pragmas within the design being parsed by Covered will be excluded from coverage consideration when the -ep option is specified. If the -ep option is not specified, these pragmas are ignored by Covered.

The code that is found between these pragmas is not removed from resimulation, however. Therefore, if the user decides that any code found within these pragmas needs to be considered, they may change the exclusion properties via the GUI without needing to rerun Covered's score command. Coverage numbers in generated reports will treat excluded code as though it has been hit (i.e., the coverage points will be a part of the total number and the hit counts will be increased as though these coverage points were hit during simulation).

The coverage pragmas may use either the single-line C++ comment style (//) or the multi-line C comment style (/* */). By default, the pragma is identified with the keyword 'coverage' following the comment characters. This value can be changed by the user, if needed, by specifying a string value to the right of the -ep option in the score command. This can be useful if there are other programs that use the same pragma keyword and causes conflicts with Covered's use of this pragma name. The value specified after the keyword is either 'off' (meaning that code should be excluded from coverage consideration after this comment) or 'on' (meaning that code should be considered for coverage after this comment). The values of '`off`' and '`on`' have the same meanings regardless of the pragma keyword so be sure to rename the pragma keyword to something that is appropriate.

Coverage pragmas may be embedded within each other, if desired. That is the following code example is allowed and handled properly by Covered' parser:

```
module foobar;

  reg b, c, d, e;

  // coverage off

  wire a;

  // coverage off
  assign a = b & c;
  // coverage on

  assign e = ~d;
  // coverage on

endmodule
```

The net effect of this example is that all code between the outer '`coverage off`' and '`coverage on`' pragmas are excluded from coverage.

One last note about the -ep option is that coverage exclusion is restricted on a per file basis. That is, if the user specifies a '`// coverage off`' pragma within a file and does not specify a '`// coverage on`' pragma later in the file, the next file to be parsed by Covered will be handled as if coverage is on. Coverage pragmas can, however, span across multiple modules or other entities within the same file.

3.3.5 Overriding Parameters (-P option)

The *defparam* statement can override any parameter specified in the DUT from any module in the DUT. Since parameter information is not typically specified in VCD/LXT2/FST dumpfiles, Covered needs to know about all parameter values at score time. Since all defparam statements are ignored by the Covered parser (even those in the DUT being scored), the user must inform Covered of any parameters that need to be overridden. This is accomplished with the -P option to the score command.

The first value in the -P option is the hierarchical reference to the parameter that needs to be overridden. The second value (following the '=' sign) specifies the value to assign the parameter to. This will override any default value assigned to that parameter. The user may not override the same parameter value twice on the command-line. Additionally, if the parameter is already being overridden in the design via an inline parameter override (parameter overrides specified in the '#(...)' part of the module instantiation).

As an example of how to override a parameter value on the score command-line, let's assume that there is a parameter called 'google' in the module called 'amod' used in the example in Section 9.3. The parameter definition in this module looks like the following:

```
parameter google = 4'hf;
```

To override this value to a new value of 18, use the following option in the score command:

```
-P test.foo.a.google=18
```

In this example, notice that we have not only changed the value of google, but we have also changed the bit-width of google from 4 bits to 32 bits (the size of an integer). This is considered legal behavior by Covered.

It is also possible to specify non-integer values to the parameter override option. Decimal, binary, octal and hexidecimal values may also be specified, using the same syntax as they are specified in Verilog. The following examples would also be other ways to override the value to 18.

```
-P test.foo.a.google=5'b10010
-P test.foo.a.google=4'o22
-P test.foo.a.google=12'h12
```

```
-P test.foo.a.google='d18
```

In fact, you may specify any value (with the exception of real numbers) to override a parameter, including the following value types: 'x', 'X', 'z', 'Z', '_', '?'. This means that unknowns may be specified in a parameter. Note that this may generate user reported errors if parameters overridden to unknown values are used in mathematical equations or are used to specify the width of wires, regs, etc.

If a defparam statement is encountered in the DUT being scored, a warning message will be output to the user specifying that it has encountered a defparam statement. The defparam statement is ignored by Covered; however, if the parameter that it is overridding is used in the DUT being scored, the results of Covered may differ from actual simulation results. As such, please review all of these warning types to determine if the parameter should be overridden or not. If defparams are not parsed (seen by Covered), no such warning will be output. Please review the DUT for defparams which affect the DUT being scored.

3.3.6 Scoring FSMs

At this time Covered does not have the ability to automatically extract FSM information from the design. As a result, it is up to the user to specify FSM information for Covered to use in scoring FSMs. The ability to automatically extract FSM information is planned to be supported in the future; however, it is always a good idea to allow the user to specify and potentially override Covered's automatic extraction in the case where Covered either misses an FSM or incorrectly determines logic to be an FSM in cases where none exists. The information specified in this section describes how to tell Covered where an FSM exists within the design as of the current development version of Covered.

First, we need to make a few assumptions about the type of FSMs that Covered will be able to handle and, in the future, identify.

1. The FSM is a synchronous design with edge-triggered flip-flops.
2. There are no FSMs embedded in another FSM.
3. The FSM is not a pipe-lined system.
4. The entire FSM must exist within the same module.
5. All state registers/variables must be less than 2^16 bits in width.

There is currently only one way, via the command-line, to specify to Covered where to find FSMs within the design. In the future, the inline Verilog-2001 "$attribute" function will be used to also allow the user to specify to Covered where an FSM is located. Using the command-line method, no changes need to be to source Verilog code for Covered's purposes.

To specify where an FSM exists within the design, the -F option must be specified to Covered's score command. Each -F option specifies one FSM within the design. Multiple -F options to the score command are allowed, allowing more than one FSM to be scored simultaneously. The -F option takes one argument that is subdivided into two or three parts:

```
-F module=(input_state_expression,)output_state_expression
```

The *module* specifies the Verilog module where the FSM state variables exist. The *input_state_expression* specifies the name of the variable(s) that represent the input state of the state machine. The *output_state_expression* specifies the name of the variable(s) that represent the output state of the state machine. Most FSMs have one variable as the input state (in most cases a registered version of the output state) such that the input variable is different than the output variable of the state machine. Specifying both variables in the -F option provides the user to more accurately specify to Covered the FSM. In the case that an FSM has the same variable for the input and output state, the user must specify only the output state variable (omitting the *input_state_expression* and comma character in the argument).

The input and output expression to the -F command can represent any signals, parts of signals, or combinations of signals. Constant values or parameters in the specification of the input/output expression is currently not supported (in fact, there are no future plans to support this behavior unless user-feedback demands it). The following examples show various legal ways to specify input/output expressions. The syntax is limited to what it shown in these examples.

```
-F foobar=a
```

Example 1 specifies that the entire vector called "a" should be considered both the input and output state variable.

```
-F foobar=a[2]
```

Example 2 specifies that bit 2 of the vector called "a" should be considered both the input and output state variable.

```
-F foobar=a[3:1]
```

Example 3 specifies that bits 3:1 of the vector called "a" should be considered both the input and output state variable.

```
-F foobar={a,b}
```

Example 4 specifies that vectors "a" and "b" should be concatenated to form the input/output state variable. Only one set of curly brackets may be used to specify a concatenated state expression; however, 2 or more signals may be specified within those brackets.

Any combination of the above four examples can be used to formalate the input state expression or the output state expression.

If the user specifies an FSM that Covered is not able to find (either the module was not included in the design that Covered is scoring or the FSM uses Verilog code that Covered is not able to handle), an error message is displayed to the user. It is then up to the user to determine if the FSM was incorrectly specified to Covered or otherwise. This warning is not considered to be a bug in Covered (unless of course, it can be proven that the FSM should have been found).

At the current time, Covered does not have the ability to extract the states and state transitions of the specified FSM from the design. Currently, it also doesn't have the ability to specify this information in other ways. To provide meaningful coverage information for the FSM, Covered outputs all hit states and state transitions in the report when all achievable states and state transitions are not known. This information will be able to be specified by the user in future versions of Covered, with the ability to automatically extract this information also in the works.

3.3.7 The reason for the -conservative feature

Though Covered has support for many Verilog-1995, Verilog-2001 and SystemVerilog constructs there are handful of supported constructs that are dependent on correct simulation ordering to provide accurate coverage information. In certain cases, Covered will be able to accurately determine the order of simulation, but in some cases, it may not. In these cases, it is best for Covered to run in a "conservative" mode of operation whereby logic blocks that contain these code issues are removed from coverage consideration. Removing logic blocks does not have a negative impact (in terms of coverage accuracy) on the rest of the design. The only negative consequence is that no coverage results can be obtained for a given logic block.

For example, consider the following code:

```
module main;
  foo a();
  bar b();
endmodule

module foo;
  integer a;
  initial begin
    #10;
    a = $random;
  end
endmodule

module bar
  integer a;
  initial begin
    #5;
    a = $random;
  end
endmodule
```

If only module "foo" is chosen to gather coverage information, the call to the $random function will result in a value that will not match the actual simulation if the **-conservative** option is not specified. This is due to the ordering importance of $random system task calls. The first call to $random will return a value of X while the second call will return a value of Y. If we remove the code that calls $random the first time, the code that still calls X will receive the incorrect value. This discrepancy in simulation results will result in incorrect coverage results.

If the **-conservative** option is specified to the score command, Covered will automatically remove any coverage blocks that make a call to the $random system function from coverage consideration and any assigned values within these removed blocks will get their value from the dumpfile instead of being calculated by Covered directly. This will result in correct coverage information for the module "foo".

The constructs that can cause issues with Covered simulation correctness (in some situations) are the following:

1. $random
2. $urandom
3. $urandom_range

It is currently up to the user to determine whether liberal or conservative coverage accumulation should occur and use the **-conservative** option to the score command as needed.

3.3.8 Specifying Plus Arguments When Scoring

Covered supports the $test$plusargs and $value$plusargs system function calls and, as such, runtime command-line options are allowed to be specified when scoring the design. To specify plusargs to the score command, simply specify them anywhere after the score command keyword just as you would specify them to a simulator. Any plusargs specified to the score command which are not recognized for compilation purposes are used for $test$plusargs and $value$plusargs comparisons,

Note that specifying plusargs to the score command is only necessary when the score command is given either the -vcd, -lxt or -fst options. If the VPI method of coverage scoring is used, specify the plusargs to the simulation runtime as required. Covered will automatically extract the plusargs from the simulator runtime for its purposes.

3.3.9 Other Notes

When the Verilog files are parsed, a preprocessor is run on each file that is to be read in by Covered. The preprocessor generates an intermediate form of each file, resolving defined values and included files. The intermediate form is output to a file which is called "tmpXXXXXX" where XXXXXX represents some alphanumeric sequence chosen by the OS via the mkstemp() function. The file is guaranteed to be unique to all other files in the directory and it is automatically removed by Covered when parsing is completed.

If the user does not want Covered to use this naming convention for specifying this file, the user may use the -p option (see Options section above) to override the default name. Covered will then use this filename instead of generating a unique name. Make sure that the file specified does not contain any useful data since Covered will overwrite this file when the score command is run.

3.4 The merge Command

3.4.1 Usage

The merge command is initiated with the following call:

```
covered merge [options] existing_CDD CDD_to_merge+
```

The *existing_CDD* specifier relates to any existing CDD. If the -o option is not specified (see below for details), this file will be replaced with the merged CDD created by the merge. This is important to remember since merging is an irreversible operation (apart from generating a brand-new CDD from the design and dumpfile using the score command).

The *CDD_to_merge* specifier relates to any existing CDD that is not the *existing_CDD*. Though it is not illegal to merge the same CDD to itself, the result is the same as the original CDD. Note that all files treated as *CDD_to_merge* are guaranteed not to be modified by the merge command. These files are only read by Covered and then merged into the information contained in the *existing_CDD*.

3.4.2 Options

Options to merge Command

Option	Description
-d *directory*	Directory to search for CDD files to include. This option is used in conjunction with the -ext option which specifies the file extension to use for determining which files in the directory are CDD files.
-er (first|last|all|new|old)	Specifies how to handle exclusion reason resolution. If two or more CDD files being merged have exclusion reasons specified for the same coverage point, the exclusion reason needs to be resolved (unless it is the same string value). If this option is not specified and a conflict is found, Covered will interactively request input for each exclusion as to how to handle it. If this option is specified, it tells Covered how to handle all exclusion reason conflicts. The values are as follows: **Value** — **Description** first — The CDD file that contained the first occurence of the exclusion should use its reason in the merged CDD file. last — The last CDD file that conflicts with a previous CDD file should use its reason in the merged CDD file. all — All conflicting coverage points will merge their exclusion reason information into a single exclusion reason. new — The newest exclusion reason should be used (all exclusion reasons are timestamped). old — The oldest exclusion reason should be used (all exclusion reasons are timestamped).
-ext *extension*	Used in conjunction with the -d option. If no -ext options are specified on the command-line, the default value of '.cdd' is used. Note that a period (.) should be specified.
-f *filename*	Name of file containing additional arguments to parse. You may specify this option more than once on a command-line.
-h	Outputs usage information for the merge command.
-m *message*	Allows the user to specify information about this CDD file which can be most anything. All messages found in merged CDDs will not be included in the merged CDD file.

Option	Description
-o *filename*	Name of CDD to write merged coverage information to (if not specified, the *existing_CDD* is overwritten with merged CDD).

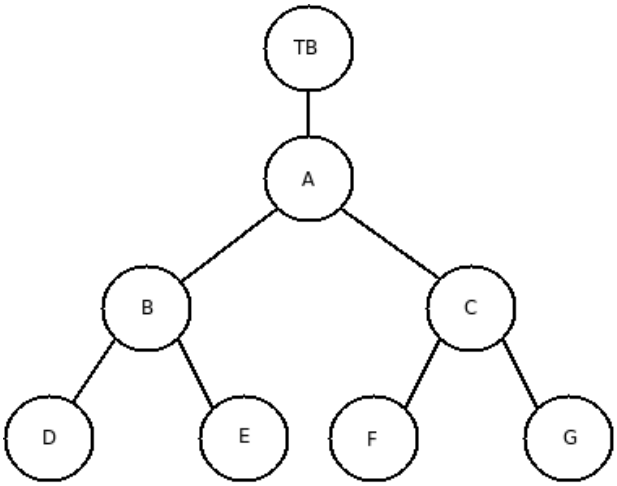
3.4.3 Merging Rules

In previous versions of Covered, merging could only be accomplished when the top-most scored modules of each CDD were the same the module. Typically, the user would score the entire DUT in each diagnostic run, creating a set of CDD files that could be merged together to see the overall coverage of the diagnostic runs. While this capability still exists, Covered has recently been enhanced to allow more complex merging which provides some new capabilities.

3.4.3.1 Block Merging

Covered's merging algorithms have now been enhanced to allow merging CDDs that are generated for different parts of the same DUT. That is to say that each instance or sub-tree in the design can be scored individually and can be merged together. This ability allows for the DUT to be split up into smaller parts, scored in parallel and then merged back together to create a single CDD containing the scored results for the entire DUT. This parallel scoring capability can now be used to speed up scoring of large designs by taking advantage of multiple CPUs or CPU cores.

The following is an example of a DUT (top-most module of the DUT is block A) and the generic covered calls to score and create a single CDD file containing coverage for the entire design.



Example of Verilog hierarchy to be scored in parallel

The following commands could be used to score the DUT comprised of blocks A-G in parallel (each score command below could occur in parallel with all other score commands).

```
covered score -t A -i MAIN.A -e B -e C -o A.cdd ...
covered score -t B -i MAIN.A.B -e D -e E -o B.cdd ...
covered score -t C -i MAIN.A.C -e F -e G -o C.cdd ...
covered score -t D -i MAIN.A.C.D -o D.cdd ...
covered score -t E -i MAIN.A.C.E -o E.cdd ...
covered score -t F -i MAIN.A.B.F -o F.cdd ...
covered score -t G -i MAIN.A.B.G -o G.cdd ...

covered merge -o total.cdd A.cdd B.cdd C.cdd D.cdd E.cdd F.cdd G.cdd
```

In addition to allowing for parallel scoring, this merge method allows users to gain coverage results for smaller parts of the design and merge them into a single CDD containing coverage results for the entire design. This can be advantageous for those that are focused on increasing coverage for a specific portion of the design. Scoring can be done on that small part and the coverage gained can be added into an existing CDD containing coverage information for the entire design.

Using Example of Verilog hierarchy to be scored in parallel this type of merging would be performed as such:

```
covered score -t A -i MAIN.A total.cdd   // The initial coverage file for the entire design

covered score -t C -i MAIN.A.C -o CDE.cdd ...   // Coverage obtained for modules C, D and E only

covered merge total.cdd CDE.cdd
```

3.4.3.2 Single Module Merging

Along with the capability to merge different portions of the same large design, it is also possible to merge CDDs that were generated from different testbenches but have modules that are in common with each other. This allows different teams to work together to enhance coverage for shared design blocks. It is important to note that when this type of merging is being performed that the top-most covered module in the CDDs labeled *CDD_to_merge* exist exactly once in the CDD labeled *existing_CDD* for instance coverage information to be reported correctly. If the module is instantiated more than once in the *existing_CDD*, the CDD will still be merged but only the module coverage information will be correct.

3.5 The report Command

3.5.1 Usage

The report command is initiated with the following call:

```
covered report (–h | –view [CDD_filename] | [options] CDD_filename)
```

The *CDD_filename* refers to the name of the CDD to generate the report for. This CDD may be either the result of a score or the result of merging CDDs.

If the -view option is specified, Covered's GUI is run. You may specify the CDD filename to initially load with the -view option. See Introduction to the GUI for instructions on using the GUI for interactive coverage analysis.

3.5.2 Options

The following table lists the options available for use with the report command.

Options to report Command

Option	Description
-b	If combinational logic verbose output is reported and the expression is a vector operation, this option outputs the coverage information on a bitwise basis.
-c	Used in conjunction with the '-d d' or '-d v' option. If set, causes covered logic to be reported instead of default behavior of reporting uncovered logic. See Covered Vs. Uncovered
-d (sldlv)	Level of report detail (s=summary, d=detailed, v=verbose). Default is to display summary coverage information. See Summary Vs. Detailed Vs. Verbose.
-e	Outputs exclusion cases when the -d v or -d d options are specified on the command-line. The exclusion cases are displayed in the verbose format along with an exclusion reason (if one is associated with the excluded case).
-f *filename*	Name of file containing additional arguments to parse. You may specify this option more than once on a command-line.
-h	Outputs usage information for the report command.
-i	Generates report information for each instance (default is to generate per module). See Module Vs. Instance
-m [l][t][c][f][r][a][m]	Specifies type of coverage information to report (l=line, t=toggle, c=combinational logic, f=FSM state/arc, r=race condition, a=assertion, m=memory). Default is ltcf.
-o *filename*	Specifies output file to stream report to (default is standard output).
-s	Suppresses modules/instances that do not contain any coverage information from being output to the coverage report. This may eliminate potentially meaningless information from the report.
-view	Starts the interactive Covered report viewer GUI.
-w (*number*)	Specifies the maximum line width (in characters) that can be used to output Verilog information. If this option is not specified, all Verilog code in the report will retain the same formatting as was specified in the original Verilog code. If this option is specified, Verilog code will be formatted to use as much of the current line as possible, wrapping text when the line reaches the maximum line width. The default maximum line width is 115 characters (this value is used if no number is specified with the -w option). If a number is specified with the -w option, this value is used for the maximum line width.
-x	When used in conjunction with the -d v or -d d option, causes all uncovered or excluded coverage points to display an associated "exclusion ID" to the left of the verbose output within parenthesis. This exclusion ID can be used in calls to the exclude command for the purposes of excluding or including coverage points.

3.5.3 Summary Vs. Detailed Vs. Verbose

There are three forms of reports that can be generated by the Covered report function: summary, detailed, and verbose. The three forms are described below.

3.5.3.1 Summary Report

A summary report shows at a very high-level what the coverage for a specific module (or instance) is in terms of line, toggle, combinational and/or FSM coverage (depending on which types are selected on the command line). Each coverage metric for the module is given a percentage of items covered for that metric as well as the total number of items per metric, the number of items "hit" during simulation and the number of items "missed" during execution. See the chapter on "Reading the Report" for more information on understanding the formats of line, toggle, combinational logic, and FSM summary information.

The summary report is useful for understanding exactly which modules are missing coverage (and what type(s) of coverage are missing) as well as what modules are fully covered. This can help guide you more easily in understanding where you need to improve your code coverage without getting lost in all of the details that the verbose reporting provides.

3.5.3.2 Detailed Report

The detailed report is useful for understanding where logic was found to be uncovered in the design along with some higher-level information to understand why it was considered to be uncovered. This amount of detail is between the minimum (summary coverage) and the maximum (verbose coverage) and should be used as the first detailed report to be looked at since it is easier to read and comprehend the coverage results.

3.5.3.3 Verbose Report

The verbose report contains all of the data that the summary report contains; however, in addition to the summary information for a module (or instance), a more in-depth look at the exact cases that were "missed" during simulation are provided. This report outputting option is useful for combinational logic report information when a more in-depth look at why certain expressions did not reach full coverage is needed. This option allows the user to look at all subexpressions of an expression that were not fully covered.

3.5.4 Module Vs. Instance

Any report can be calculated by module or by instance. Both reports are of interest for verification purposes and the differences are described as follows.

Module reporting combines the results of all instances that come from the same module. That is, when a module is instantiated multiple times in a design, the coverage results from all covered instances are merged together and output as one combined module. This reporting format allows a test writer to see if any logic within a module has not been touched during simulation.

Instance reporting displays each instance in the covered design separately (no combining occurs). This reporting format is useful for determining if certain instances within a design are being neglected. For instance, if a module is instantiated four times (i.e., four instances of the same buffer), it may be that the first buffer is used more often than the other three buffers. This could indicate controller errors or just an indication that there was not enough activity during simulation to fill the other buffers (need to bolster diagnostics or possibly some buffers could be removed?) This type of information would not be viewable if only module reporting were performed.

3.5.5 Covered Vs. Uncovered

Covered has the ability the generate reports from any given CDD file that displays either uncovered logic (the default behavior) or covered logic. It is understandable why one would want to generate reports displaying uncovered logic (this is probably the reason why you are interested in this tool in the first place). However, why would anyone be interested in reporting covered logic?

The reason for having this option is two-fold (and maybe there are other reasons). First, this option is useful in debugging the report command since it let's the user evaluate whether a particular signal or portion of logic was actually fully covered or not. Second, it may be useful for user's of the tool to understand what logic is being evaluated for coverage and what logic is not. If only uncovered logic was supplied for evaluation of the tool, one could not evaluate the effectiveness of the tool. However, by allowing the user to see what logic is covered and uncovered, a more full picture of the tool's capability can be understood.

To generate a report that specifies what logic is not covered (output only available in verbose reporting mode), no further options are needed. To generate a report that specifies what logic is being covered, simply specify the -c option along with the -v option when calling the report command.

3.6 The rank Command

The rank command allows the user to specify a number of CDD files that have been created from the same DUT (presumably in a regression run -- but not necessarily) and rank them in order of coverage importance. This information can be useful in accomplishing the following tasks:

- Understand which CDD files (simulations) can be eliminated from regression due to their inability to add useful coverage information that selected CDD files already hit.
- Understand the best order to run simulations in full regression.

- It can help in creating a quick (confidence) regression which is meant to hit a good deal of the overall coverage points in a short period of time to verify that recent changes don't massively break a previously working DUT. This can be accomplished by selecting the first N simulation CDD files in the ranked list. The selection can easily be done by either using the simulation order, accumulated hit, hit percent or timestep column values as a means of selection.

3.6.1 Options

Options to rank Command

Option	Description
-depth *number*	Specifies the minimum number of CDD files to hit each coverage point. The value of *number* should be a value of 1 or more. Default is 1.
-d *directory*	Directory to search for CDD files to include. This option is used in conjunction with the -ext option which specifies the file extension to use for determining which files in the directory are CDD files.
-ext *extension*	Used in conjunction with the -d option. If no -ext options are specified on the command-line, the default value of '.cdd' is used. Note that a period (.) should be specified.
-f *filename*	Name of file containing additional arguments to parse.
-h	Displays help information for the rank command.
-names-only	If specified, outputs only the needed CDD filenames that need to be run in the order they need to be run. If this option is not set, a report-style output is provided with additional information.
-o *filename*	Name of file to output ranking information to. Default is stdout.
-require-cdd *filename*	Name of CDD file to use as a required CDD file for ranking, regardless of whether it is necessary to achieve full coverage or not.
-require-list *filename*	Name of file containing list of CDD files which are required to be in the list of ranked CDDs to be run, regardless of whether they are necessary to achieve full coverage or not.
-v	Outputs verbose information during the run of the rank command. Useful for selection process understanding and command performance.
-weight-assert *number*	Specifies a relative weighting for assertion coverage used to rank non-unique coverage points.
-weight-comb *number*	Specifies a relative weighting for combinational logic coverage used to rank non-unique coverage points.
-weight-fsm *number*	Specifies a relative weighting for FSM state/state transition coverage used to rank non-unique coverage points.
-weight-line *number*	Specifies a relative weighting for line coverage used to rank non-unique coverage points.
-weight-memory *number*	Specifies a relative weighting for memory coverage used to rank non-unique coverage points.
-weight-toggle *number*	Specifies a relative weighting for toggle coverage used to rank non-unique coverage points.

3.6.2 Methods for specifying CDDs to rank

The rank command provides several mechanisms for specifying CDD files to rank. Feel free to use any combination of methods for providing this information to Covered as it makes sense to do in your environment. The following is the list of input methods:

- Specify each CDD file after the rank command options have been specified. This method is best when there are a relatively small number of CDD files to rank as the length of the command-line in a shell is typically limited to a certain number of characters.

- Create a file containing the list of CDD files to rank and pass this file to the rank command's **-f** option. Using this method, one can selectively specify a large number of CDD files. You may specify as many "-f *filename*" options as is necessary.

- Use shell commands to create a list of CDD files and pass them to Covered's standard input via command-line piping and the **-f -** command-line option (the - means use standard input). The following line is an example of this usage:

  ```
  ls foo*.cdd | covered rank -f -
  ```

  You may specify only one "-f -" option to the rank command.

- Use the **-d *directory*** and **-ext *extension*** options to tell Covered to retrieve all files found in the directory specified by *directory* that contain the file extension of *extension* (if the **-ext** option is not specified, the default extension of ".cdd" is used). The following is an example of this usage:

  ```
  covered rank -d /home/bubba/cdd_dir -ext .mycdd
  ```

You may specify as many -d options on the command-line as is necessary; however, only one -ext option is allowed.

- Specify a single CDD file that MUST be included in the final ranking list via the **-require-cdd** option. You can specify as many -require-cdd options to the rank command as necessary.
- Create a file containing a list of CDDs that MUST be included in the final ranking list where each CDD filename is separated by a newline or whitespace. Pass this file to the **-require-list** option. You can specify as many -require-list options to the rank command as necessary.

3.6.3 Specifying metric weights

The rank command has several options that allow the user to weight any and all of the coverage metrics that Covered supplies. Weights are used to ascribe the importance of its metric in ranking the value of a CDD file.

Weight values are non-negative numbers whose values are relative to each other. A value of 0 effectively disables the associated weight from consideration in ranking. All other values are considered relative to each other. In other words, if the line weight is set to a value of 5 while the toggle weight is set to 1, line coverage is considered 5 times more important than toggle coverage in ranking.

By default, each metric is given an equal weight of 1. Adjust each of the metrics as necessary to potentially modify which CDD files are chosen and what order those CDD files are given in the ranked list of diagnostics to run.

3.6.4 Understanding the rank command output information

After the rank command has been executed, the output will either be displayed to standard output (if no **-o** option was specified) or it will be output to a file. If the **-names_only** option was specified, the output will be a list of CDD filenames (one per line) which are output in the order in which they should be run to achieve full coverage in the shortest amount of time. If the -names_only option was not specified, the output will be in the form of a table (similar to reports generated with the report command) which describes not only the order that CDD files should be run but also various statistical data and the names of CDD files which are considered redundant in terms of coverage information. The following example shows an example of this report output.

```
::::::::::::::::::::::::::::::::::::::::::::::::::
::                                              ::
::     Covered -- Simulation Ranked Run Order   ::
::                                              ::
::::::::::::::::::::::::::::::::::::::::::::::::::


* Reduced 3 CDD files down to 2 needed to maintain coverage ( 33% reduction,   1.5x improvement)
* Reduced 9 timesteps down to 6 needed to maintain coverage ( 33% reduction,   1.5x improvement)

-----------+-----------------------------------------+-----------------------------------------------------------
           |                ACCUMULATIVE             |                          CDD
Simulation |-----------------------------------------+-----------------------------------------------------------
Order      |        Hit /      Total     %   Timesteps |  R  Name              Hit /      Total     %   Timesteps
-----------+-----------------------------------------+-----------------------------------------------------------

         1           9           17   53%           3       rank1c.cdd          9           17   53%           3
         2          13           17   76%           6       rank1b.cdd          8           17   47%           3

--------------------------------------  The following CDD files add no additional coverage  ---------------------
-------------------------

         3          13           17   76%           9       rank1a.cdd          8           17   47%           3
```

After the header information, there is a summary of the ranking reduction (if one occurred) specified in both number of files and number of simulation timesteps. example_title shows what this information looks like when no file reduction has occurred.

After the summary information, there is a rank output table which displays both the individual CDD information and the accumulative coverage information. Each row of the table contains one CDD file where the first row contains the first CDD that should be run in regression for maximum coverage impact. Each successive row contains CDD files that add less and less coverage to the overall coverage gained. If there were CDD files that were found to not add any coverage information to the above CDD files, a line is drawn and these CDD files displayed below that line. These CDD files can be excluded from regression because they don't add any useful coverage information over-and-above the CDD files that are selected to be run in regression.

The first major column is the simulation order. This value increments by one for each successive row. If you would like to create a shorter running regression that is limited to a certain number of simulations, this row information can be useful for figuring out which simulations to include in these type of regressions.

The second major column is the accumulative coverage statistics which include the following:

- **Hit**

  The first row contains the number of coverage points that the first CDD file hit. All successive rows add the unique coverage points hit by the CDD file listed on that line to the above hit total.

  For example, in example_title "rank1c.cdd" contains 9 hit coverage points (accumulative hit count = 9). The second highest ranked file, "rank1b.cdd", contains 8 hit coverage points but only 4 of those coverage points are different than the coverage points hit by "rank1c.cdd"; therefore, the accumulative coverage hit count is 13. The third coverage file, "rank1c.cdd", contains 8 coverage points but none of them add to the coverage points hit by the first two coverage files; therefore, it is considered unnecessary for regression.

- **Total**

  The "accumulative" total represents the total number of coverage points in each CDD file. Since this number must be the same for each CDD file, this value will be the same for each row.

- ***%***

  This column represents the hit percentage and is a simple calculation of `hit / total` for each row.

- **Timesteps**

  This column represents the number of accumulated simulation timesteps. This value represents the number of timesteps that need to transpire to gain the amount of accumulated coverage. One of the goals of the ranking algorithm is to gain the most amount of coverage in the least amount of simulation time.

The third major column is the individual coverage information for the CDD file listed in the row which includes the following information:

- **R**

  This column represents whether the current CDD file was a required CDD file by the user (a file listed via the -require-list or -require-cdd options). If the file is a required CDD file, an asterisk (*) is listed in this column. If the file was not required, nothing will be listed in this column. example_title provides an example of what this output looks like.

- **Name**

  Filename of the CDD file that was selected for this row.

- **Hit**

  This column represents the number of coverage points hit by the CDD file.

- **Total**

  This column represents the total number of coverage points within the CDD file that could be hit.

- ***%***

  This column represents the hit percentage (a calculation of `hit / total`.

- **Timesteps**

  This column represents the number of simulation timesteps that the CDD file took to run during simulation. This value does not necessarily represent the ending simulation time.

```
::::::::::::::::::::::::::::::::::::::::::::::::::
::                                              ::
::     Covered -- Simulation Ranked Run Order   ::
::                                              ::
::::::::::::::::::::::::::::::::::::::::::::::::::
```

```
No reduction occurred

-----------+-----------------------------------------+-------------------------------------------------------
--
           |                ACCUMULATIVE             |                              CDD
Simulation |-----------------------------------------+-------------------------------------------------------
--
Order      |        Hit /      Total     %   Timesteps |  R  Name                Hit /      Total     %
Timesteps
-----------+-----------------------------------------+-------------------------------------------------------
```

```
--

        1           9          17   53%          3       rank1.1c.cdd          9          17   53%
3
        2          13          17   76%          6    *  rank1.1b.cdd          8          17   47%
3
        3          13          17   76%          9    *  rank1.1a.cdd          8          17   47%
3
```

3.6.5 Description of ranking algorithm

This section describes the overall algorithm that is used to rank the specified CDD files. The algorithm is being presented in this document for the purposes of providing an explanation for how diagnostics are selected for ranking and how they are ranked. Knowing this information is not necessary for using the rank command or understanding its results.

The algorithm makes use of three different types of structures. The first structure is a bit-compressed structure where each bit corresponds to one coverage point in the associated CDD file. The purpose of this structure is to store a large amount of information as concisely and immediately available as possible. In addition to coverage point information, each structure also contains the following information:

- Name of CDD file that the structure is associated with (for reporting purposes only).
- The number of timesteps that occurred during the CDD simulation. This is used for calculating the average number of coverage points hit during a single clock period -- used for ranking selection.
- The total number of coverage points hit by the CDD. This is used for calculating the average number of coverage points hit during a single clock period -- used for ranking selection.
- The total number of coverage points that were hit by this CDD that were not hit by any other CDD listed in the rank command-line. This information is used to figure out which CDDs are absolutely necessary to be included in the ranked list.
- The current score associated with the CDD file. This value is updated for each selection iteration.

Each of these structures are maintained in an array that is iterated on during the ranking process.

The second structure is an array of 16-bit values where each value corresponds to a single coverage point. Its value is a sum of the number of unranked CDD files that have hit that coverage point. Each value is decremented by one when a CDD file is moved from the unranked list to the ranked list and that CDD file hits the given coverage point. If a coverage point value is a value of 1 and a CDD file has its corresponding coverage bit set, we know that the CDD is the only one that hits the coverage point.

The third structure is similar to the second and works similarly, except that it stores the number of CDD files in the ranked list that hit a particular coverage point.

The specified CDD files are ranked in a series of steps, some of which are reiterated for each CDD that is selected to be ranked. Here is the basic algorithm (in pseudo-code):

```
calculate the size of the compressed coverage point structure

foreach CDD in CDD_list {
  convert CDD to compressed coverage point structure
}

foreach CDD in CDD_list {
  if this CDD contains uniquely hit coverage points {
    move the CDD file into the ranked list
  }
}

foreach CDD in unranked list {
  foreach CDD in unranked list {
    calculate score (((total_num_cov_points_hit / num_timesteps) * 100) * metric_weight_value)
    if any unranked elements are less than the -depth option value and are hit by this CDD {
      set unique_found to true
    }
    if our score > highest score and a "unique" coverage point was found {
      set highest_score to our score
    }
  }
  move the CDD file indexed by highest_score from the unranked list to the ranked list
}

resort ranked list from most unique to least unique (if there is a tie, select the CDD that hit the most
```

```
coverage
    points in the least amount of time)
```

3.7 The exclude Command

The exclude command allows the user to exclude uncovered coverage points from coverage (effectively turning the uncovered points into covered points) or include coverage points that were previously excluded. Additionally, the exclude command allows the user to specify a reason why the coverage point is being excluded. This capability allows for good documentation of these coverage points for later evaluation and can be output into various report formats to go along with a verification testplan.

3.7.1 Usage

The exclude command is specified with the given call:

```
covered [global options] exclude [options] exclusion ID+ CDD file
```

The exclusion ID information can be obtained via a Covered report command call with the **-d (dlv)** and **-e** options. Exclusion IDs consist of an alphanumeric value that will be specified with parenthesis prior to a coverage point.

You may specify one or more exclusion IDs per call to the exclude command. If the **-m** option is specified, a prompt will be specified for each exclusion ID, individually. If a coverage point is currently included in the CDD prior to an exclude call, the coverage point will be toggled to be excluded and a user message can be attached to the coverage point, specifying the reason for exclusion. If a coverage point is currently excluded in the CDD prior to an exclude call, the coverage point will be toggled to be included and any previouly assigned user message will be forgotten.

3.7.2 Options

Options to the exclude command

Option	Description
-f *filename*	Allows the user to specify a file that contains additional exclude command options to parse. This option may be specified more than once for a single call to the exclude command.
-h	Displays help information for the exclude command.
-m	Allows a user message specifying the reason for excluding any coverage points being excluded on the command-line. An interactive prompt will be specified in the shell. Simply enter your reason at the prompt, hit return and specify a single period (.) character followed by a return to end the user input for a coverage point. You may use as many lines of text that you would like; however, all formatting characters (i.e., newlines, tabs, extra spaces, etc.) will be removed when reporting the information.
-p	Outputs the current exclusion status of the specified exclusion ID(s) and displays any exclusion reason messages associated with the specified exclusion ID(s) to standard output.

3.8 Reading the Report

This chapter describes the meanings of the textual report output that Covered generates when the report command is issued. It is well understood by the developers of Covered that a code coverage tool is only as good as the output that it generates, and this, of course, includes the ability to discern the information provided by the report.

To help describe the various sections of a Covered report, we will use a relatively small Verilog file containing our DUT, generating the two main types of reports (module and instance) based on the CDD generated for this file. This example file is provided here

After being compiled and run, this module is scored with the following Covered command:

```
covered score -t main -v example.v -o example.cdd -vcd example.vcd
```

The module-based verbose report can be viewed in its entirety and is generated with the following command:

```
covered report -d v example.cdd
```

The instance-based verbose report can be viewed in its entirety here and is generated with the following command:

```
covered report -i -d v example.cdd
```

The instructions for analyzing each of the six types of coverage information from the report can be found on the following pages.

1. Reading Line Coverage

2. Reading Toggle Coverage
3. Reading Memory Coverage
4. Reading Combinational Logic Coverage
5. Reading FSM Coverage
6. Reading Assertion Coverage

For information on reading race condition information in the report, please see the user guide page detailing race conditions and viewing the reports.

3.8.1 Reading Line Coverage

3.8.1.1 Summary Information Description - Module-based

For module-based reports, the summary table for line metrics includes information for the name of each module that was covered and the name of the file in which the module is described in. Lines 15 through 24 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.1.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for line metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand-side of each row. Lines of the instance-based report show what this information looks like in the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1" and "main.fsm2".

3.8.1.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the line coverage, followed by a calculated percent of the lines that were hit (calculated by taking the number of lines hit during simulation divided by the total number of lines that Covered simulated). The hit value indicates how many lines were executed during the simulation; the miss value indicates the number of lines not executed during simulation; and the total value indicates the total number of lines within the specified module/instance that Covered was able to simulate.

If the percentage value in the far left of the summary table is 100%, this indicates that all lines that Covered was capable of simulating (for the module/instance of this row) were executed. If the value of the percentage is less than 100%, this indicates that some number of lines were not executed and full coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any lines in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.1.4 Verbose Information Description - Both

If the miss value for a particular module was not 0 (indicating that lines were missed during simulation) and the '-d v' option was specified on the command-line (specifying to output verbose reporting information), the line that was missed during simulation will be output below the summary information per module that contained missed lines. In our sample report, there was one line that was missed during simulation (according to the summary output) for the module called "fsmb" (instance "main.fsmb"). The line number and Verilog line that was missed is output in lines 25 through 29 of the module-based report and in lines 25 through 29 of the instance-based report.

If a module does not contain any missed lines and the '-d v' option was specified, no verbose output will be displayed for this module. Likewise, if a module/instance does not contain any lines that Covered could execute, no verbose output will be displayed.

The definition of what Covered considers a line is a statement (ex. assignment, conditional, etc.). If a statement is written in such a way that it consumes multiple file lines, Covered only counts the statement as one line. The user can assume that if the beginning of a statement is labeled as missed, the entire expression was missed (not just the first line of the expression). For example, consider the following code:

```
always @(posedge b)
  a <= (b & c) |
       (d & e);
```

If the expression

```
a <= (b & c) | (d & e);
```

is not executed during simulation, Covered will indicate that only one line was missed during execution (not two), and it will display the missing line (if verbose reporting is specified with the '-d v' option) as the following:

```
2:      assign a <= (b & c) ...
```

The "..." notation at the end of the Verilog output indicates that the statement was a multiline statement in the source code but is only considered one line.

3.8.2 Reading Toggle Coverage

3.8.2.1 Summary Information Description - Module-based

For module-based reports, the summary table for toggle metrics includes information for the name of each module that was covered and the name of the file in which the module is described in. Lines 41 through 51 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.2.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for toggle metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand-side of each row. Lines 41 through 51 of the instance-based report show what this information looks like in the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1" and "main.fsm2".

3.8.2.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the toggle 0->1 coverage, followed by a calculated percent of the signal bits that toggled from a value of 0 to a value of 1 (calculated by taking the number of bits that toggled from 0 to 1 during simulation divided by the total number of signal bits). The hit value indicates how many signal bits were toggle from a value of 0 to 1 during simulation; the miss value indicates the number of signal bits that were not toggled from a value of 0 to 1 during simulation; and the total value indicates the total number of signal bits within the specified module/instance that Covered was able to simulate.

To the right of this information is the hit, miss and total statistics for signal bits that toggled from a value of 1 to a value of 0.

If the percentage values in the right of the summary table are 100%, this indicates that all signal bits toggled from a value of 0 to 1 and back to 0 during simulating (for the module/instance of this row). If the values of the percentage are less than 100%, this indicates that some signal bits did not fully toggle during simulation and full toggle coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any signals in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.2.4 Verbose Information Description - Both

If the miss value for a particular module/instance was not 0 (indicating that one or more signal bits did not fully toggle during simulation) and the '-d v' option was specified on the command-line (specifying to output verbose reporting information), the signals that were missed during simulation will be output below the summary information per module/instance that contained missed signal bit toggles. In our sample module-based report, there were three signals in module "main" ("go", "state", and "error"), three signals in module "fsma" ("go", "state" and "next_state"), and three signals in module "fsmb" ("go", "next_state" and "state") that were not fully toggled (see lines 53 through 90).

For each signal in the verbose toggle table, the format is the following:

```
        <signal_name>          0->1: <hexidecimal_value>
        ......................1->0: <hexidecimal_value>
```

The name of the signal is specified in the upper left corner. The bitwise toggle information for the signal from a value of 0 to a value of 1 is specified in the upper right corner. The hexidecimal value represents bits in the signal, each bit corresponding to the matching bit of the signal. If a bit value is set to the value 1, this indicates that this bit in the signal toggled from a value of 0 to a value of 1. If a bit value is set to the value 0, this indicates that this bit in the signal did NOT toggle from a value of 0 to a value of 1; therefore, full toggle coverage was not achieved for this signal. For example, for the signal called "go" in the module/instance "main", the verbose toggle information looks like:

```
go         0->1: 1'b1
...........1->0: 1'b0
```

This is indicating that bit 0 of the signal called "go" successfully toggled from a value of 0 to 1 during simulation (because the bit value at bit location 0 is a value of 1). However, the signal did not toggle from a value of 1 to a value of 0 during simulation, thus the signal was not considered fully toggled.

It is important to note that the hexidecimal values are displayed with the least-significant bit of the signal being output on the far right of the value while the most-significant bit of the signal is output on the far left of the value.

3.8.3 Reading Memory Coverage

3.8.3.1 Summary Information Description - Module-based

For module-based reports, the summary table for memory metrics includes information for the name of each module, instantiating one or more memories, that was analyzed and the name of the file in which the module is described in. Lines 93 through 111 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.3.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for memory metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand-side of each row. Lines 93 through 111 of the instance-based report show what this information looks like in the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1" and "main.fsm2".

3.8.3.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the various memory coverage metrics, followed by a calculated percent of each memory coverage metric that was hit (calculated by taking the number hit during simulation divided by the total number that Covered could have simulated). The hit value indicates how many were executed during the simulation; the miss value indicates the number not executed during simulation; and the total value indicates the total number within the specified module/instance that Covered can simulate. All four metrics (AME toggle 0->1, AME toggle 1->0, AME write and AME read) are listed in the summary section.

If the percentage value in the far left of the summary table is 100%, this indicates that all coverage points that Covered was capable of simulating (for the module/instance of this row) were executed. If the value of the percentage is less than 100%, this indicates that some number of coverage points were not executed and full coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any coverage points in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.3.4 Verbose Information Description - Both

When a module/instance is found to be not fully covered (i.e., the number of each hit coverage points is not equal to the associated number of attainable coverage points), the missed coverage points for each AME in the memory are output to the report as follows:

```
<AME>  Written: <0 or 1>  0->1: <hexidecimal_value>
.....  Read:    <0 or 1>  1->0: <hexidecimal_value> ...
```

The answers to each of the questions listed above are specified with a 0 (meaning that the answer is "NO") or a 1 (meaning that the answer is "YES"). Note that in the toggle coverage information, like toggle coverage verbose information, the individual bits are grouped together and displayed in hexidecimal form to conserve space and make the task of figuring out exactly which bit positions did not toggle easier. See lines 117 through 151 for an example of this output.

If the -c option is used, Covered outputs the names of the memories that were fully covered during simulation only.

3.8.4 Reading Combinational Logic Coverage

3.8.4.1 Summary Information Description - Module-based

For module-based reports, the summary table for combinational logic metrics includes information for the name of each module that was covered and the name of the file in which the module is described in. Lines 154 through 164 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma, and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.4.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for combinational logic metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand-side of each row. Lines 154 through 164 of the instance-based report show what this information looks like in

the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1", and "main.fsm2".

3.8.4.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the combinational logic coverage, followed by a calculated percent of the combinational logic expression values that were hit (calculated by taking the number of combinational logic expression values hit during simulation divided by the total number of expression values that Covered could have simulated). The hit value indicates how many expression values were achieved during the simulation; the miss value indicates the number of expression values not achieved during simulation; and the total value indicates the total number of combinational logic expression values that could have been achieved in the specified module/instance.

If the percentage value in the far left of the summary table is 100%, this indicates that all combinational logic expression values that Covered was capable of achieving (for the module/instance of this row) were hit. If the value of the percentage is less than 100%, this indicates that some number of expression values were not achieved and full coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any combinational logic expressions in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.4.4 Verbose Information Description - Both

The verbose output for combinational coverage is a bit more sophisticated than the line or toggle. Basically, for each expression that was found to not be fully covered (either the expression itself or some sub-expression in the tree was not fully covered), the expression is output with various sub-expressions underlined and identified with a number. Each expression is also supplied with the line number that the expression started at in its module file. In our example, there is an uncovered expression in the module called "main" (see lines 170 through 205 (numbered 5) which contains four uncovered subexpressions (numbered 1 through 4).

Below each expression, there will be one or more tables which specify a particular sub-expression that was not fully covered along with information describing what cases were not covered for that sub-expression.

In the report output above, we see that there was one expression in module "main" that did not achieve 100% coverage that starts on line 14 of the Verilog source. The expression is output to the report with certain sub-expressions underlined and labeled with an integer value for reference.

For this one expression, there were found to be four subexpressions (labeled 1, 2, 3, 4) that were found to not be 100% covered. Taking a look at subexpression 1 report output, the numbers to the right of the string "Expression 1" tell us the number of logical combinations of this subexpression that were hit (in this case 2 combinations were hit) and the total number of logical combinations that this subexpression can have (in this case 3 combinations were achievable). The character string below this information tells us the type of subexpression that we are examining. For subexpression 1, the type is the bitwise AND operator.

The table below this information for subexpression 1, lists the potential combinations that this subexpression could have reached with a '*' character placed underneath the combination(s) that were missed. In the case of subexpression 1, the left and right subexpressions did not evaluate to values of 1 simultaneously. The letter 'L' above each possible combination indicates the value that the left subexpression achieved at the same time as the value under the right subexpression indicated with the letter 'R'. Legal values for a subexpression are '0', '1' or '-' (which indicates that this subexpression value could either be 0 or 1).

3.8.5 Reading FSM Coverage

3.8.5.1 Summary Information Description - Module-based

For module-based reports, the summary table for FSM metrics includes information for the name of each module that was covered and the name of the file in which the module is described in. Lines 344 through 354 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.5.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for FSM metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand-side of each row. Lines 344 through 354 of the instance-based report show what this information looks like in the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1" and "main.fsm2".

3.8.5.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the FSM state coverage, followed by a calculated percent of the FSM states that were hit (calculated by taking the number of FSM states hit during simulation divided by the total number of FSM states that Covered could have simulated). The hit value indicates how many FSM states were executed during the simulation; the

miss value indicates the number of FSM states not executed during simulation; and the total value indicates the total number of FSM states within the specified module/instance that Covered can simulate.

To the right of the FSM state summary information is the FSM state-transition hit, miss, total and percentage hit summary information for each module/instance.

If the percentage value in the far left of the summary table is 100%, this indicates that all FSM states and state-transitions that Covered was capable of simulating (for the module/instance of this row) were executed. If the value of the percentage is less than 100%, this indicates that some number of FSM states and/or FSM state-transitions were not executed and full coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any FSMs in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.5.4 Verbose Information Description - Both

The verbose output for FSM coverage is split into the state coverage information and the state transition coverage information. When an FSM is found to be not fully covered (i.e., the number of hit states/state transitions is not equal to the number of attainable states/state transitions), the missed states are output to the report as a list of state values in hexidecimal format as follows:

```
<hexidecimal value>
```

When an FSM is found to be not fully covered, the missed state transitions are output to the report as a list of state transitions in the format below:

```
<hexidecimal input state value> -> <hexidecimal output state value>
```

If the -c option is used or the number of attainable state/state transitions are unknown for the specified FSM, Covered outputs this information in the same way as the missed cases except that the title of the output is "Hit cases". If the number of attainable states/state transitions is unknown, providing the cases that were hit during simulation is useful in aiding the user in determining coverage. In this case, the summary report will output question marks in the missed and total categories, showing the user that this information was not known by Covered.

3.8.6 Reading Assertion Coverage

3.8.6.1 Summary Information Description - Module-based

For module-based reports, the summary table for assertion metrics includes information for the name of each module, instantiating one or more assertions, that was covered and the name of the file in which the module is described in. Lines 397 through 405 of the module-based report show what this information looks like in the report. We have three modules that were scored within our DUT: main, fsma and fsmb. The table shows that all three modules were described in the file "example.v".

3.8.6.2 Summary Information Description - Instance-based

For instance-based reports, the summary table for assertion metrics includes information for the Verilog hierarchy pertaining to each instance on the left-hand side of each row. Lines 397 through 405 of the instance-based report show what this information looks like in the report. In our DUT example, there are three instances within the design with the Verilog hierarchies of "main", "main.fsm1" and "main.fsm2".

3.8.6.3 Summary Information Description - Both

On the right-hand side of each row in the table are the hit, miss and total numbers for the assertion coverage points (ACP), followed by a calculated percent of the ACPs that were hit (calculated by taking the number of ACPs hit during simulation divided by the total number of ACPs that Covered could have simulated). The hit value indicates how many ACPs were executed during the simulation; the miss value indicates the number of ACPs not executed during simulation; and the total value indicates the total number of ACPs within the specified module/instance that Covered can simulate. Note that more than one coverage point may exist within a single assertion coverage module.

If the percentage value in the far left of the summary table is 100%, this indicates that all ACPs that Covered was capable of simulating (for the module/instance of this row) were executed. If the value of the percentage is less than 100%, this indicates that some number of ACPs were not executed and full coverage was not achieved for that module/instance. Note that for a module/instance which does not contain any ACPs in which Covered was able to simulate, the values of hit, miss, and total will be 0 while the hit percentage value will indicate 100%.

3.8.6.4 Verbose Information Description - Both

When a module/instance is found to be not fully covered (i.e., the number of hit ACPs is not equal to the number of attainable ACPs), the missed coverage points are output to the report as follows:

```
<Instance Name>   <Assertion Name>    <Coverage Point Description>
```

If the -c option is used, Covered outputs this information in the same way as the missed cases except that the title of the output is "Hit cases" and the number of times each ACP was hit is indicated.

3.9 Debugging

As much as it pains the developers of Covered to admit, there will be times when a user of Covered will encounter a problem in the resimulation process or will experience a performance issue during certain runs and will want to investigate the issue a bit themselves before posting a bug report. There may also be people out there who just might want to learn more about how Covered operates "under the hood". For those people whom I have just mentioned, this section is for you.

3.9.1 Verbose Debug Output

The brute force method of understanding Covered's internals is to generate verbose debug output using the global "-D" option to any of Covered's commands. The user of this method should take care to note that using this option may generate an *extremely* large amount of output and will definitely cause Covered's runtime performance to degrade considerably. If this option is used, its output should probably be redirected to a file for post-processing purposes. It should be noted that the verbose debug output is most useful for the developer's of Covered and is not meant to be of much help to the normal user.

3.9.1.1 Building Covered for Verbose Debug Outputting

Because generating verbose debug output can have a tremendous degradation in performance when it is even compiled, Covered requires the user to specify an option to the configuration/build process to enable this feature. To enable verbose debug outputting, simply specify the **`--enable-debug`** option to the configure script. This will cause Covered to compile in verbose debug output support and enable the use of the -D global option.

If the user has not specified the --enable-debug option to the configuration script but proceeds to specify the -D global option, Covered will emit a warning message that it is unable to output verbose debug information due to the lack of the --enable-debug option being specified. It will then proceed to ignore the -D option.

3.9.2 Command-Line Interface (CLI)

As an alternative to simply outputting the verbose debug output from Covered, the score command comes equipped with its own interactive debugging mechanism called the CLI (Command-Line Interface). This interface is a command-line driven utility that starts at the beginning of the resimulation. Its purpose is to allow the user to step through simulation execution and view the contents of signals, time, scopes, code and internal thread queues for the purposes of debugging and/or understanding Covered. This utility works much in the same way as simulator CLIs and the GDB debugging utility.

3.9.2.1 Building Covered for the CLI

Because interactive command-line debugging has a small performance penalty when compiled in and is closely associated with the verbose debug output option, Covered requires the user to specify an option to the configuation/build process to enable this feature. To enable the CLI, simply specify the **`--enable-debug`** option to the configure script. This will cause Covered to compile in CLI capabilities and enable the user of the -cli score command option.

3.9.2.2 Using the CLI

To allow the score command to use the CLI for interactive resimulation, simply specify the **`-cli [filename]`** option to the score command. If filename is specified to this option, the contents of this file will be parsed and executed once the score command has reached the resimulation process. The contents of this file are a series of CLI commands (one command per line) in ASCII format and can be automatically produced with the "`savehist`" CLI command (see below). This feature allows a user to quickly save off the CLI commands made during one score run and reuse them in a subsequent score run without having to remember and manually input the same commands.

Once the -cli option is invoked, Covered will run the score command until it gets to the beginning of the resimulation process. At this time, if the optional filename is specified, it will parse the contents of this file and execute the CLI commands found there until all commands are executed. If filename was not specified, a prompt will be displayed and the resimulation execution will halt until the user has specified otherwise via command-line options. After entering a command-line option, simply press return to invoke that command. In the event that the command was a step, next or continue option, resimulation will resume until it reaches the next stop point.

During resimulation, if the -cli option is specified to the score command, the user may stop simulation at any time by hitting **Ctrl-C**. This will stop the simulation and display a CLI prompt where the user may continue to interact with the simulation.

3.9.2.3 CLI commands

The following is a list of all of the CLI options available to the user:

CLI Command-line Options

Option	Description
step [*num*]	Advances to the next statement if *num* is not specified; otherwise, advances *num* statements before returning to the CLI prompt.
next [*num*]	Advances to the next timestep if *num* is not specified; otherwise, advances *num* timesteps before returning to the CLI prompt.
goto time *num*	Simulates to the given timestep (or the next timestep after the given value if the timestep is not executed) specified by *num*.
goto line [*filename*:]*num*	Simulates until the specified line is the next line to be executed. If *filename* is specified, the line specified by *num* within that file is the line to stop on. If *filename* if not specified, the current filename is used.
goto expr *name bool_op value*	Simulates until the given expression is evaluated to true where *name* is the name of a signal in the design to evaluate, *bool_op* is a boolean operation (i.e., ==, !=, ===, !==, <, <=, >, >=), and *value* is a legal Verilog value (i.e., 30, 4'h0, etc.).
run	Runs the simulation to completion.
continue	Continues running the simulation to completion.
thread active	Displays the current state of the internal active simulation queue. This queue consists of process threads that will be executed in the current timestep.
thread delayed	Displays the current state of the internal delayed simulation queue. This queue consists of process threads that will be executed at a later time as dictated by a delay statement.
thread waiting	Displays the current state of the internal waiting simulation queue. This queue consists of process threads that are not active or delayed but may be activated by a value change in one of the signals in its expression tree.
thread all	Displays the current state of all process threads in the simulator.
current	Displays the current scope, block, filename and line number of the statement that is about to be executed in the active queue.
time	Displays the current simulation time.
signal *name*	Displays the current value of the given net/variable.
expr *num*	Displays the given expression and its current value where *num* is the ID of the expression to output.
debug [less | more | off]	Specifies the amount of debug information that is output during simulation. The value of 'less' only outputs the current statement that is being executed (along with line number). The value of 'more' outputs verbose output that is really only useful for the developer's of Covered. The value of 'off' turns off all debug output during simulation. If 'less', 'more', or 'off' is not specified, displays the current debug mode.
list [*num*]	Lists the contents of the file where the current statement is to be executed. If *num* is specified, outputs the given number of lines; otherwise, outputs 10 lines.
savehist *filename*	Saves the current history to the specified file.
history [(*num* | all)]	Displays the last 10 lines of command-line history. If 'all' is specified, the entire history contents will be displayed. If *num* is specified, the last *num* commands will be displayed.
!*num*	Executes the command at the *num* position in history.
!!	Executes the last valid command.
help	Displays the available command-line options.
quit	Ends simulation immediately.

3.9.3 Source Code Profiling

The built-in source code profiling facility allows the user to run any Covered command and understand performance metrics about the internal Covered source code, such as how many times a given function was called, how much wall-clock time was spent executing a given function, how many times a function allocated and deallocated memory from the heap, etc. This information can be useful in understanding where performance bottlenecks are during a given Covered command run. This feature does not provide the end-user any useful information about the design that they are testing.

3.9.3.1 Building Covered for Profiling

Because internal source code profiling is not a function of obtaining Verilog code coverage and adds a performance penalty for being compiled in and used, Covered requires the user to specify an option to the configuration/build process to enable this feature. To enable internal source code profiling, simply specify the **`--enable-profiling`** option to the configure script. This will cause Covered to compile in profiling capabilities and enable the use of the -P global option.

If the user has not specified the --enable-profiling option to the configuration script but proceeds to specify the -P global option, Covered will emit a warning message that it is unable to do profiling due to the lack of the --enable-profiling option being specified. It will then proceed to ignore the -P option.

3.9.3.2 Using Source Code Profiling

Once Covered has had profiling functionality configured and built into it, the user may optionally enable profiling for any command by specifying the **`-P [filename]`** global option. If the -P option is specified without a filename associated with it, the profiling report generated from the command run will be output to a file called "covered.prof" in the same directory where Covered was run. If a filename is associated with the -P option, the contents of the profiling report will be output to that file.

The specified filename can be anything except "score", "merge" or "report" as this name will be interpreted by Covered's command-line parser to indicate the type of command being run. If one of these command keywords are specified after the -P option, it is assumed that the user has not specified a filename and the contents of the profiling report will be sent to the default file of "covered.prof".

3.9.3.3 Understanding the Profiling Report

After a command has been run with the global -P option specified, a report file will contain several profiling statistics, including the following:

1. The amount of time spent running the entire command
2. The number of times each executed function was called
3. The total amount of wall clock time spent executing each function
4. The average amount of time spent executing each function
5. The number of times each function allocated heap memory
6. The number of times each function deallocated heap memory

In addition to providing this information on a per function basis, the profiling report also provides the information in three different sections with each function ordered from most to least by the following statistic columns:

1. The number of times each executed function was called
2. The total amount of wall clock time spent executing each function
3. The average amount of time spent executing each function

By ordering this information, it will help the users and developers determine where the performance bottlenecks in execution are for a given command run.

4 Graphical User Interface

4.1 Introduction to the GUI

The Covered report viewer GUI allows a user to view the coverage results from one or more CDD (Covered Design Database) files that have been previously generated using the Covered score and/or merge commands. The advantages of using the GUI versus using an ASCII file to view coverage results are the following:

1. View code that is covered/uncovered within the actual source file text (more context)
2. Quickly see which modules/instances were uncovered for each coverage metric.
3. Ability to hide/unhide verbose information for uncovered code.
4. Quickly find expressions that caused combinational logic cases to be uncovered.
5. Allows user to focus on a particular expression/signal/module/instance that is uncovered.
6. Non-useful coverage cases can be interactively excluded/included and summary information automatically recalculated.

Because the nature of a GUI interface is much more interactive than a generated text file report, the Covered report viewer is meant to provide the user a more in-depth look at the logic to identify the reasons for not achieving full coverage. This is the ultimate goal of the GUI

interface -- to be an interactive tool for identifying and understanding why full code coverage was not achieved and how to write tests that will cover those areas.

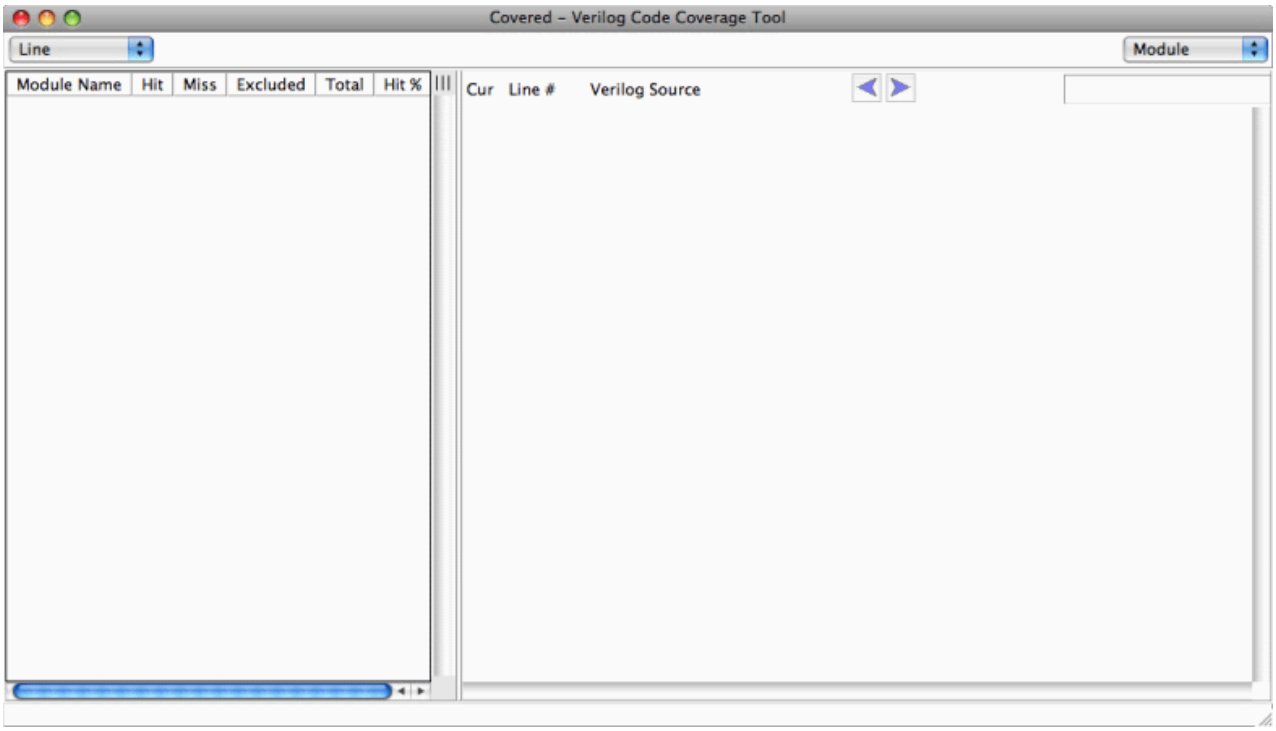
The GUI report viewer is invoked by specifying the **`-view`** option to the Covered report command. The report viewer is part of the report command and is, therefore, not used to parse/score a design. However, merging maytake place with the GUI as multiple CDD files may be read, merged (assuming all of the CDD files came from the same DUT) and saved in CDD file format.

The rest of the user manual pages describe the various widgets and windows of the GUI and their functionality. Additionally, this user manual contains information on how to get the most out of the Covered GUI report viewer.

4.2 Navigating the Main Window

Covered Main Window shows the main window prior to a CDD file being loaded. There are five main components of the main window. These components are the following (from top to bottom, left to right):

1. The menu bar
2. The coverage type bar
3. The module/instance summary listbox
4. The coverage file viewer
5. The information bar

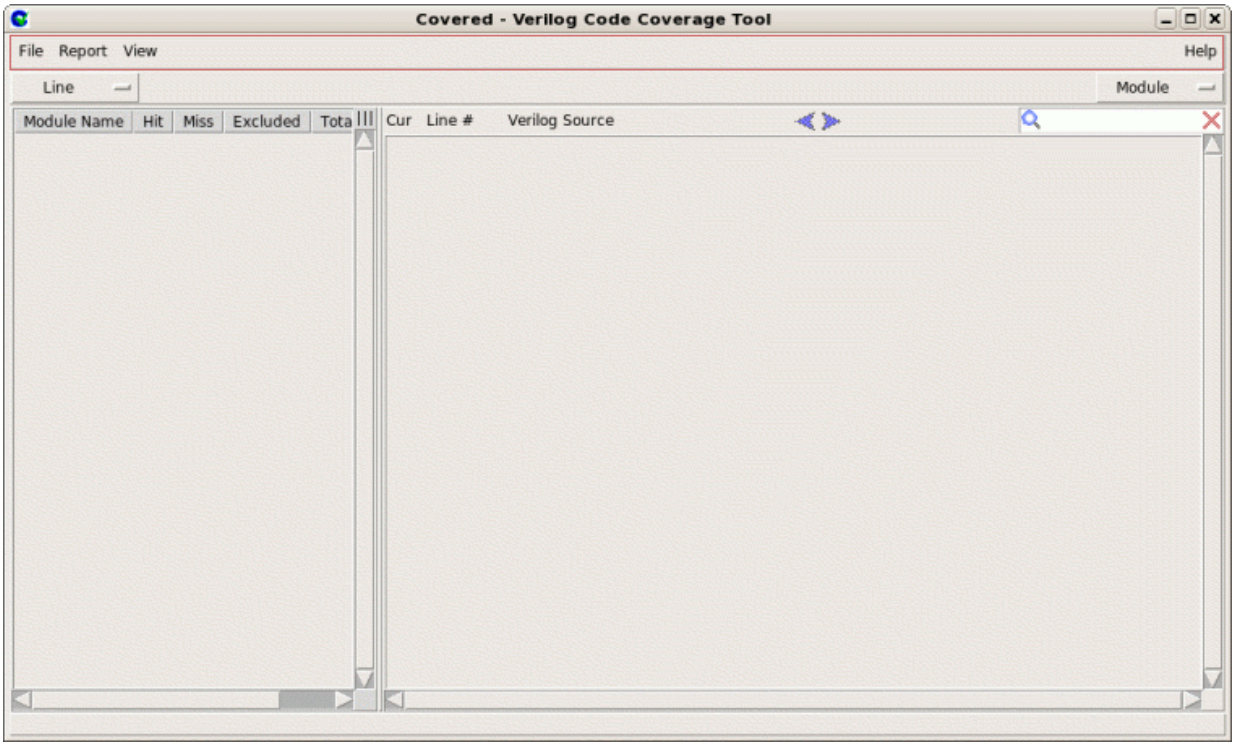


Covered Main Window

4.2.1 The Main Menu Bar

The Main Menu Bar consists of four menus at the top of the main window. Covered Main Window Highlighting the Menu Bar below shows the menu bar portion of the GUI bordered in red. These menus are the following:

1. File - Creates, opens, merges, saves and closes CDD files, generates reports, and exits the application.
2. Report - Specifies what type of report to show.
3. View - Provides additional viewing options.
4. Help - Provides access to online help manual and tool information.

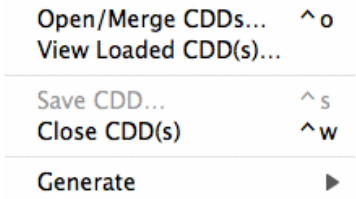


Covered Main Window Highlighting the Menu Bar

On Mac OS X, the main menubar does not exist in the application window. Like most Mac OS X applications, the main menubar exists at the top of the display.

4.2.1.1 The File Menu

The file menu is used to open a new CDD file, merge a related CDD file (derived from the same design as the currently opened CDD file, save the currently opened CDD file(s), close all opened CDD files, generate different types of output and view the names of all currently opened CDD files. Additionally, the user selects this menu bar to exit the Covered report viewer GUI. The File Menu shows the file menu contents within the main window.



The File Menu

4.2.1.1.1 Open/Merge CDDs...

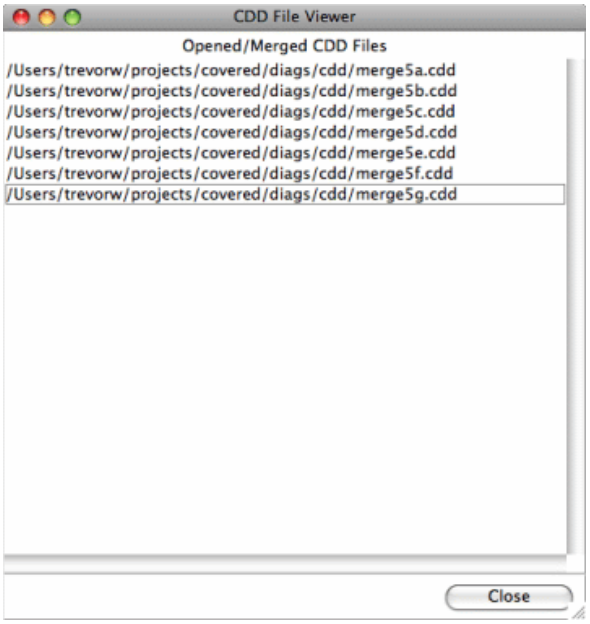
This menu option is used to either open a new CDD file or merge a related CDD file into the currently opened CDD file(s). This action needs to be performed before any coverage viewing can take place. When this option is selected, a standard file window is displayed allowing the user to select one (or possibly more depending on your version of Tcl/Tk) CDD file to open or merge. To select more than one CDD file with one call to this option, simply press and hold the Control key while selecting files with the left mouse button. If your version of Tcl/Tk supports this feature, you will see multiple files highlighted for selection; otherwise, only the last selected file will be selected. If no CDD files currently are opened, any CDD file may be opened. If one or more CDD files have been opened/merged, only CDD files generated from the same design may be merged.

Note that this option has an accelerator keystroke <Control-O>. Hitting this keystroke combination will also open new CDD file(s).

4.2.1.1.2 View Loaded CDD(s)...

This menu option is only available after an initial CDD file has been opened using the **Open/Merge CDDs...** file menu option. When this menu option is selected, a new window displaying all currently opened CDD files is displayed (see The CDD Opened File Viewer below). If another CDD file is merged and this window is currently displayed, it is automatically updated with the newly added file. If the **Close CDDs...** file menu option is selected while this window is opened, all filenames will be automatically removed to indicate this. The **View**

Loaded CDD(s)... menu option is useful for helping the user see exactly which files have been merged or not merged in the current viewer. Simply click on the **Close** button in the **Loaded CDD file viewer** window to close the window.



The CDD Opened File Viewer

4.2.1.1.3 Save CDD...

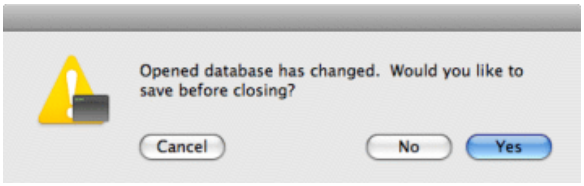
This menu option is only available after one or more CDD files have been opened/merged. It is used to save the opened or merged CDD files as a single, merged CDD file than can be viewed as a separate CDD file or can be merged with other CDD files. It is also used to save any changes that the user has made to exclude/include coverage cases within the GUI. If you would like to retain this information, please save the CDD file. By default, the name of the first loaded CDD file is placed in the "Save As" window when this option is selected. The user may choose to replace the old CDD file with the new one, or the user may save it under a different name. Once a CDD file has been saved, this option is deselected until either a new CDD file is merged or the user changes the exclude/include property of any coverage case within the GUI.

Note that this option has an accelerator keystroke <Control-s>. Hitting this keystroke combination will also save the current CDD.

4.2.1.1.4 Close CDDs

This menu option is used to close all opened/merged CDD files so that the user may view a different CDD file without needing to exit and re-enter the GUI. The user may open and close as many times as needed without needing to reinvoke the GUI. When CDD files are closed, any opened coverage windows are automatically closed and all necessary GUI components are restored to their original state as when the GUI is first invoked. If the currently opened CDD file(s) have not been saved, a window (shown in The Close Warning Window below) is displayed to ask the user if they would like to save prior to closing. To save the CDD file, simply click "Yes" and a "Save As" window will prompt you for a location/name to save it as. To close the CDD file without saving, simply click "No". To cancel the close operation, simply click **Cancel**. To view another CDD file after closing all CDD files, simply select the **Open/Merge CDDs...** file menu option.

Note that this option has an accelerator keystroke <Control-w>. Hitting this keystroke combination will also close all opened CDD files.



The Close Warning Window

4.2.1.1.5 Generate

This option, when selected, displays a submenu of items that can be generated by the GUI. The following options (with descriptions) are available (see The Generate Menu List).



The Generate Menu List

- **New CDD...**

  Selecting this submenu option will cause Covered to allow for the creation of a new CDD file (note: this can be accomplished on the command-line with the "score" command). When this option is selected, the new CDD creation wizard window is created. See Creating a New CDD for more information on the navigation of this window.

  Note that this option has an accelerator keystroke <Control-n>. Hitting this keystroke combination will also cause the new CDD creation window to be displayed.

- **ASCII Report...**

  Selecting this submenu option will cause Covered to generate an ASCII version of the coverage report to a specified file (note: this can also be accomplished with the "report" command of Covered's command-line interface). When this option is selected, the ASCII report generation wizard window is created. See Creating an ASCII report file for more information on the navigation of this window.

  Note that this option has an accelerator keystroke <Control-r>. Hitting this keystroke combination will also cause the ASCII report generator window to be displayed.

- **CDD Ranking Report...**

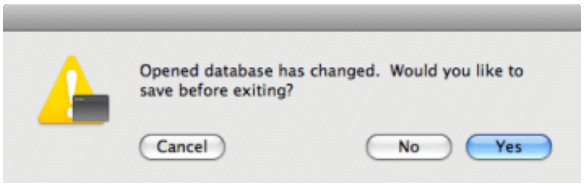
  Selecting this submenu option will cause Covered to generate a CDD ranking report to a specified file (note: this can also be accomplished on the command-line with the "rank" command). When this option is selected, the CDD ranking wizard window is created. See Creating a CDD Ranking Report for more information on the navigation of this window.

  Note that this option has an accelerator keystroke <Control-c>. Hitting this keystroke combination will also cause the CDD ranking window to be displayed.

4.2.1.1.6 Exit

This menu option will exit the entire GUI application, closing all opened windows created by this application. Use this option to exit the application. If the currently opened CDD file(s) have not been saved, a window (shown in The Exit Warning Window below) is displayed to ask the user if they would like to save prior to exiting. To save the CDD file, simply click **Yes** and a "Save As" window will prompt you for a location/name to save it as. To close the CDD file without saving, simply click **No**. To cancel the close operation, simply click **Cancel**.

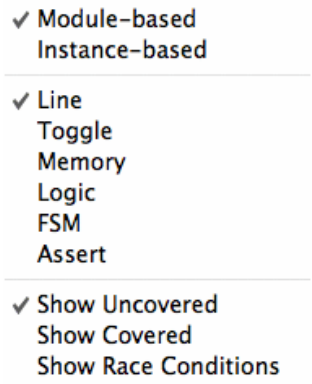
Note that this option has an accelerator keystroke <Control-x> (or <Command-Q> on Mac OS X). Hitting this keystroke combination will also close the entire application.



The Exit Warning Window

4.2.1.2 The Report Menu

The report menu allows the user to display the currently opened CDD file contents in a number of different ways for analyzing. The options specified in this menu closely imitate the options in the "report" command. The type of report can be changed at any point in time while a CDD is loaded. When a report menu option is selected, Covered will automatically convert all information in the GUI to represent this report type. The Report Menu below shows the contents of the report menu. Each menu item is described below the figure.



The Report Menu

4.2.1.2.1 Module-Based

When the diamond to the left of this option is selected, the generated report information will be in module-based representation. A module-based report gathers all of the module instances that instantiate the same module and merges their results based on the module name. This is useful for understanding what logic in a module has been tested or untested without care to individual instance contribution. Selecting this report type (the default) will cause the **Instance-based** menu option to be automatically deselected (the two report types are mutually exclusive).

4.2.1.2.2 Instance-Based

When the diamond to the left of this option is selected, the generated report information will be in instance-based representation. An instance-based report shows the coverage information for each module instance within the design. This allows the user to discern what logic is being covered in individual instances. Selecting this report type will cause the **Module-based** menu option to be automatically deselected (the two report types are mutually exclusive).

4.2.1.2.3 Line, Toggle, Memory, Logic, FSM, Assert

These menu options are directly tied to the coverage metric select menubutton in the coverage bar (see The Coverage Type View Bar). Changing the value in the menubutton will change the selection of this menu item and vice versa. Only one of these menu items will be selected at a given time.

4.2.1.2.4 Show Uncovered

When the square to the left of this option is selected (the default), all uncovered lines, toggles, logic, and FSM states/arcs within the file viewer window will be highlighted for the user to quickly see what code was monitored by Covered but determined to not be fully covered. This option may be selected along with the **Show Covered** menu option to see both covered and uncovered logic within the same file.

4.2.1.2.5 Show Covered

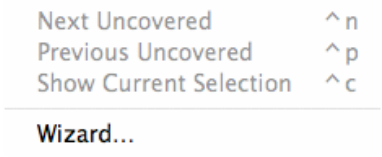
When the square to the left of this option is selected, all covered lines, toggles, logic, and FSM states/arcs within the file viewer window will be highlighted for the user to quickly see what code was monitored by Covered and determined to be fully covered. This option is useful for the user of Covered to understand what logic was monitored during simulation. This option may be selected along with the **Show Uncovered** menu option to see both covered and uncovered logic within the same file.

4.2.1.2.6 Show Race Conditions

When the square to the left of this option is selected, all logic lines that were found to contain a potential race condition and was, therefore, eliminated from coverage consideration by Covered will be highlighted by the colors as selected in the preference menu. This option is useful for the user of Covered to understand what logic was not considered for coverage information due to a detected potential race condition situation. Please see Race Condition Checking for more information on what Covered looks for in logic to consider it to be a possible race condition.

4.2.1.3 The View Menu

The View menu allows the user to select or change the current way the information in Covered is viewed. The View Menu shows the contents of the View menu.



The View Menu

4.2.1.3.1 Next Uncovered

Causes the line containing the next uncovered metric to be visible in the file viewer. If the next uncovered line is already visible, this menu item will have no visible effect. The search for the next uncovered line starts at (a) the top of the module (when the module is first displayed in the the Coverage Viewer), (b) from the last uncovered line searched for with the **Next Uncovered** or **Previous Uncovered** calls, or (c) from the last currently selected line (only valid for toggle and combinational logic views).

This menu item is only enabled when another uncovered line exists after the current uncovered line. It can also be accessed with the accelerator keystroke <Control-n> or the right arrow button in the Main Window.

4.2.1.3.2 Previous Uncovered

Causes the line containing the previous uncovered metric to be visible in the file viewer. If the previous uncovered line is already visible, this menu item will have no visible effect. The search for the previous uncovered line starts at (a) the top of the module (when the module is first displayed in the the Coverage Viewer), (b) from the last uncovered line searched for with the **Next Uncovered** or **Previous Uncovered** calls, or (c) from the last currently selected line (only valid for toggle and combinational logic views).

This menu item is only enabled when another uncovered line exists before the current uncovered line. It can also be accessed with the accelerator keystroke <Control-p> or the left arrow button in the Main Window.

4.2.1.3.3 Show Current Selection

This menu item is only available if the current metric mode is not "Line", the user has clicked on an uncovered segment of code, and the associated toggle, memory, logic, FSM or assertion verbose window exists. The currently selected line is visually identified in the file viewer area on the left-hand side of the line containing the uncovered code. If this line is not currently visible in the Coverage Viewer, selecting this menu item will cause it to be visible.

An accelerator keystroke <Control-c> will also invoke this menu item.

4.2.1.3.4 Wizard...

Displays the Covered wizard window which normally opens upon starting the Covered GUI. The Wizard window allows the user to easily create a new CDD file, open/ merge existing CDD files, rank existing CDD files and get user documentation. See Navigating the Wizard Window for more information.

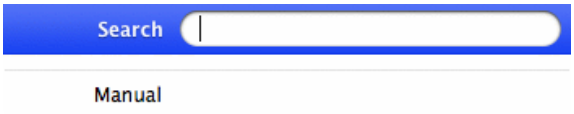
4.2.1.3.5 Preferences...

Causes the preferences window to be created if it currently does not exist or forces it into the foreground if it does exist.

On Mac OS X, this menu item does not exist in the "Report Menu" but rather in the Covered application menu to remain consistent with the Mac OS X environment.

4.2.1.4 The Help Menu

The help menu contains all of the user documentation for Covered. Additionally, information regarding the version of Covered used and other contact information for application problems, suggestions, etc. can be found in this menu.



The Help Menu

4.2.1.4.1 Manual

Selecting this menu item causes Covered to display the user manual in a web browser.

4.2.1.4.2 About Covered

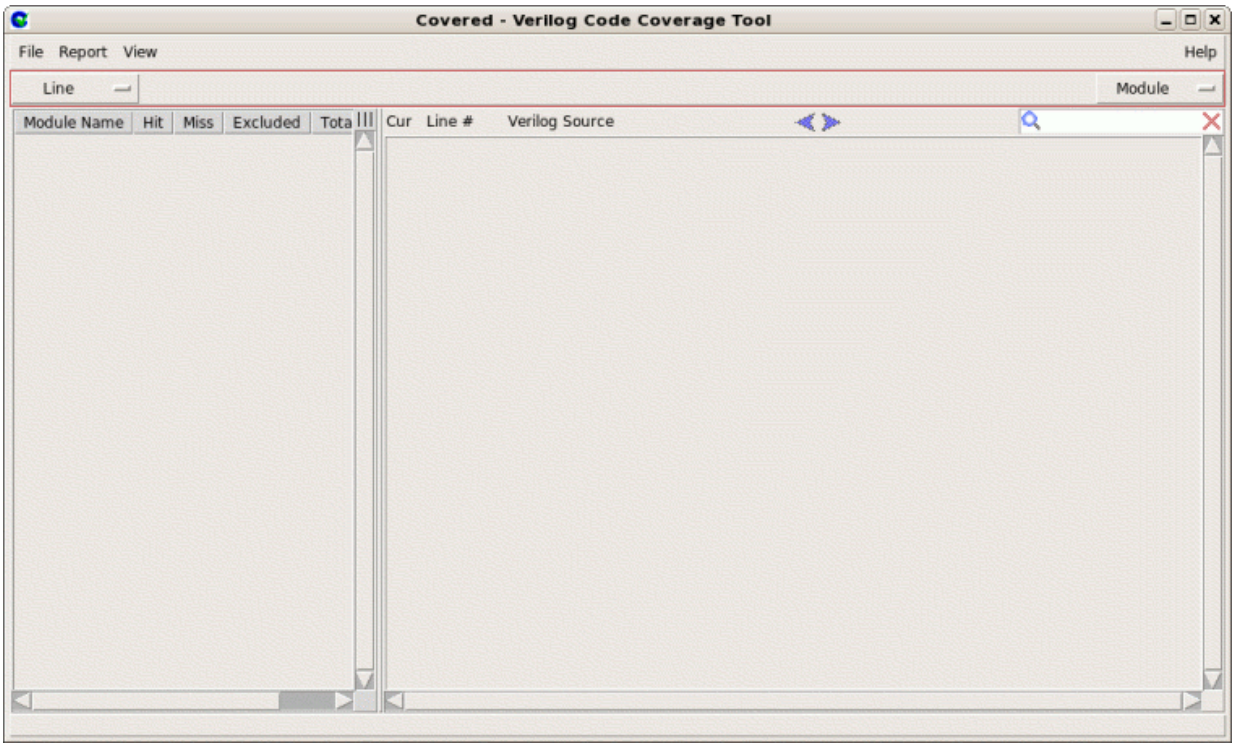
Selecting this menu item causes Covered to bring up a window showing the current release number, developer contact information and credits.

On Mac OS X, this menu item does not exist in the Help Menu but rather in the application's menu to remain consistent with the Mac OS X environment.

4.2.2 The Coverage Type View Bar

Covered Main Coverage Type View Bar shows the location of the coverage type view bar. The bar contains two selection buttons. The selection button on the left side of the bar shows the currently selected coverage metric. There are six main types of coverage metrics that Covered can provide: line, toggle, memory, logic (combinational logic), FSM (state and arc) and assert (assertion coverage). When the Covered GUI is first invoked, the line coverage metric is the default mode. To view a different metric, simply left click, select the desired coverage metric, and left click again. Doing so will automatically set the selection indicator to the new mode and display the new metric information in the Module/Instance listbox and file viewer.

The selection button on the right side of the bar shows the current coverage accumulation mode: module or instance. Module coverage takes all of the similar module instances and merges their coverages. Instance coverage displays each module instance individually. To select a different value than the one displayed, simply left click the selection button, highlight the desired value and left click again. This will repopulate the Module/Instance lisbox and clear the file viewer.



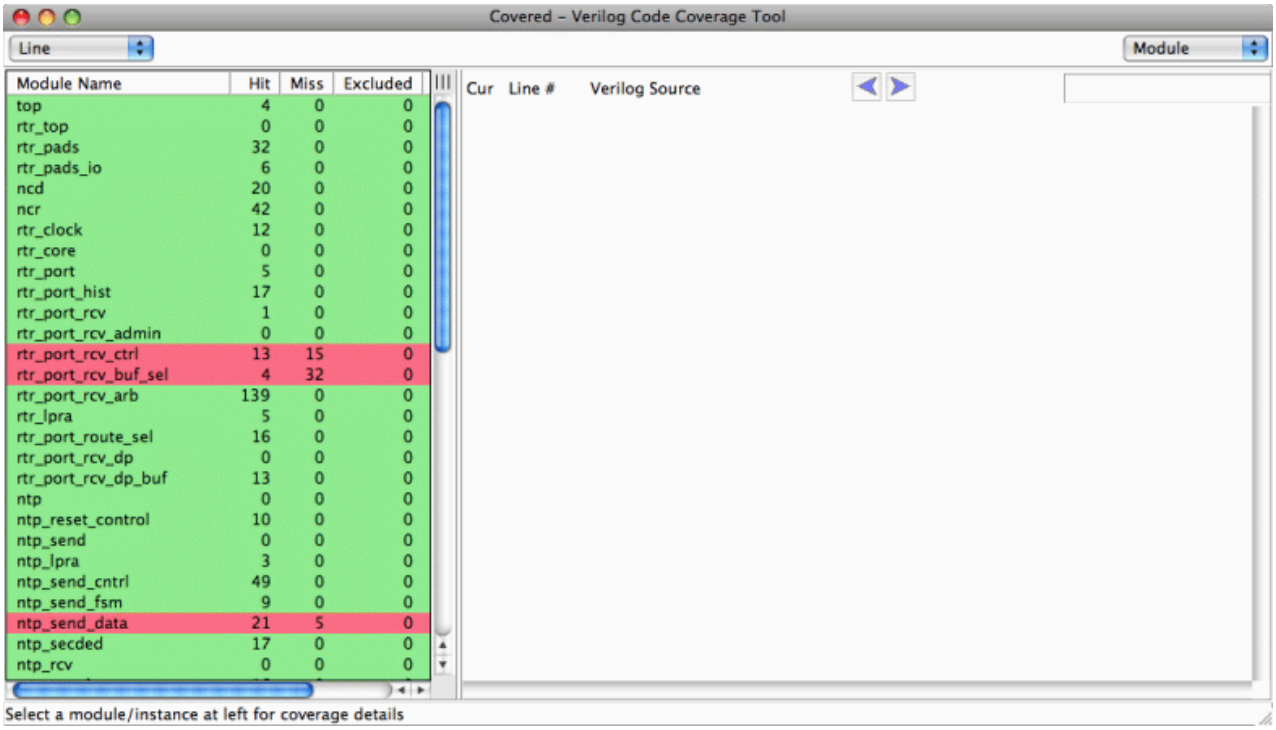
Covered Main Coverage Type View Bar

4.2.3 The Module/Instance Listbox

After a CDD file has been opened in the GUI, the module/instance selection box (shown in Populated Module/Instance Selection Box below) will contain either a list of all of the modules within the specified design or a list of all of the instances within the design, based on the type of report selected in the report menu. Left-clicking on a module/instance name within this box will cause that file to be output in the file viewer to the right of the selection box.

In addition to the names of the modules or instances of the design, each line also contains a summary view of the coverage for the currently selected metric for each module or instance. The summary coverage information includes the number of hits, number of misses, total number and hit percentage. Each column in the table can be resized by clicking on the separator bar between the column headers and sliding the column to the left or to the right. Additionally, columns may be hidden or shown in the module/instance selection box by clicking on the button at the right-hand side of the column header bar (and above the vertical scrollbar). When this button is left-clicked, a popup menu will be displayed showing which columns are currently shown (a check to the left of the column name) and which are hidden (no check). To change the display for a particular column, simply click on the column name in the popup window. This will cause the associated column to be hidden or shown and will close the popup window. If the popup window is displayed and the user does not wish to change the display values of any of the shown columns, simply click anywhere on the screen but on the popup window. This will cause the popup window to disappear and no column display values will be changed.

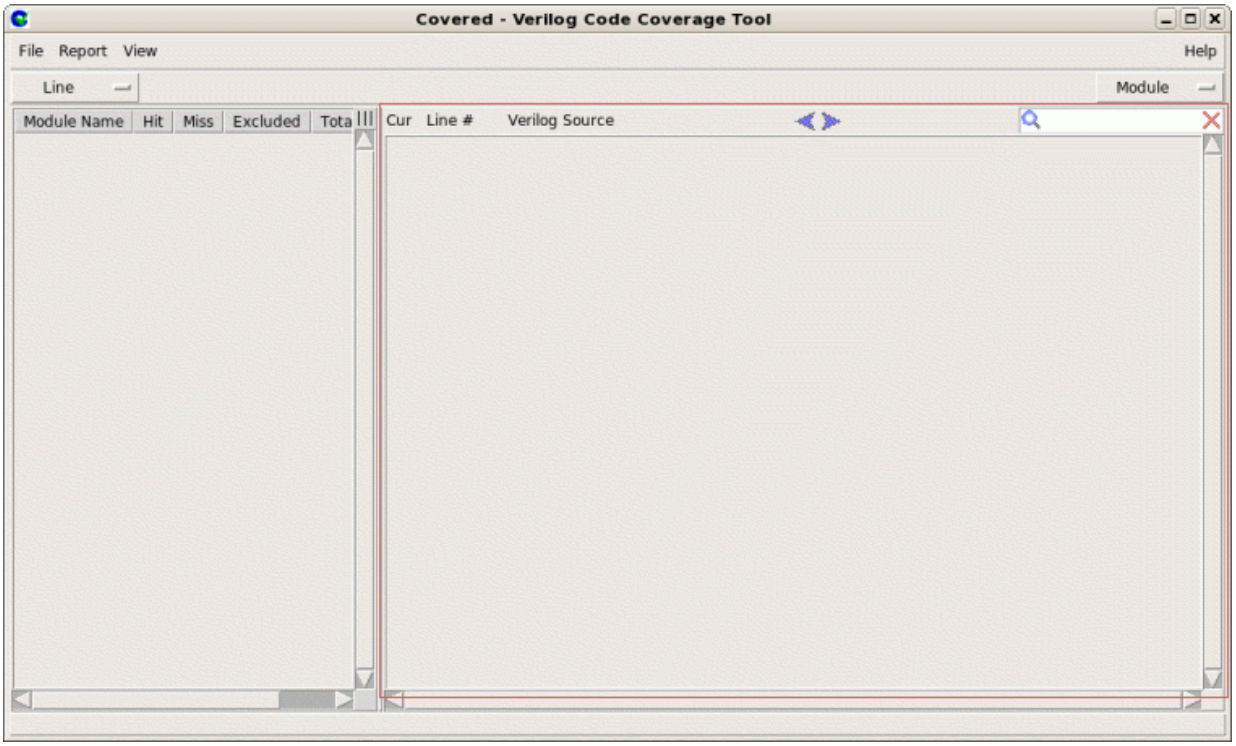
Depending on the selected coverage metric (specified in the coverage bar) and the coverage results for each module/instance, a module/instance item will be highlighted in the uncovered logic color if the corresponding module/instance was found to have uncovered logic within that module/instance. If the specified module/instance was found to be fully covered for the currently selected metric, it will be highlighted with the selected covered logic color. This can help the user quickly identify modules/instances that contain uncovered logic cases. Note that when a different coverage metric is selected, the highlights will be recalculated for that metric. Additionally, the currently selected module/instance will be displayed in a darker version of the covered/uncovered color.



Populated Module/Instance Selection Box

4.2.4 The Coverage File Viewer

Covered Main Coverage File Viewer shows the area of the main window where the coverage file viewer window is located.



Covered Main Coverage File Viewer

The coverage file viewer shows the currently selected module (indicated in the information bar) Verilog source. The coverage file viewer provides a read-only view of the original source code so all editing of the file must be done in a separate editor. Each source code line in the viewer window is numbered to the left of the line to aid in debugging and finding the lines in an editor.

Depending on the coverage metric currently selected in the coverage bar and the covered/uncovered/race condition selections in the report menu, the lines that are found to be uncovered, covered, and/or considered a race condition are highlighted in their respective color schemes in the coverage file viewer window. Using the scrollbars to the right and below the coverage file viewer, the user can find the missed/hit lines quickly for further analysis.

The coverage file viewer is the place where the user can invoke the toggle, memory,combinational logic, FSM and/or assertion detail windows. To view one of these windows, do the following:

1. Select either the **Toggle**, **Memory**, **Logic**, **FSM** or **Assert** coverage metric selection button in the coverage bar.
2. Select a module/instance that contains uncovered code in the listbox.
3. In the coverage file viewer, find code that is highlighted as being uncovered.
4. Left click on the uncovered piece of logic. Doing so will automatically bring up either the toggle, the memory, the logic, the FSM or the assertion verbose coverage windows.

Additionally, the logical blocks that have been disregarded by Covered due to a potential race condition existing are highlighted. By placing the cursor over one of these logical blocks, the reason for why the logical block was disregarded is output in the information bar.

To view a new module/instance, simply left click on one in the Module/Instance Listbox. The source code for the selected module/instance will be immediately loaded into the coverage file viewer in the currently selected coverage metric.

To view a new coverage metric, simply left click on one of the coverage metrics in the Coverage Type Bar. The currently selected module/instance will be immediately updated in the coverage file viewer, highlighting the uncovered/covered code for the current metric.

4.2.4.1 Searching for an Uncovered Line in the Coverage Viewer

To find the next or previous line containing uncovered logic for the current metric mode, click the left arrow (Previous) or right arrow (Next) buttons. Doing so will cause the next or previous uncovered line to be visible in the Coverage File Viewer. Clicking these buttons has the same effect as the "Next Uncovered" or "Previous Uncovered" menu items in the view menu.

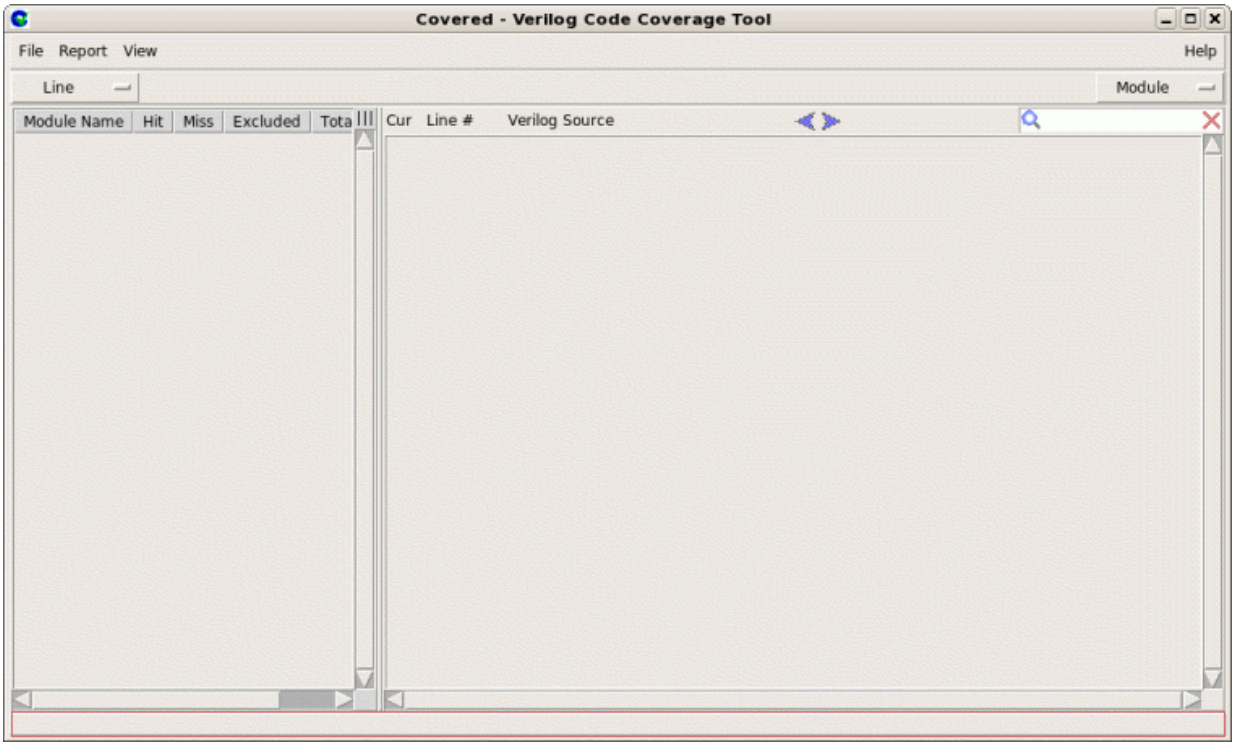
4.2.4.2 Searching for a String in the Coverage Viewer

The Coverage Viewer contains a find (magnifying lens) button, an entry field, and a clear (X) button for performing text searches in the Coverage File Viewer. The user may type in any string (wildcards and pattern matching is not supported) in the entry field. Hitting the "return" key or the find button will cause the string search to commence.

If a match to the search value is found, it is made visible in the Coverage File Viewer and highlighted. The user may search on the value again by clicking the find button. To clear the current value in the entry field, click on the clear button. If a match occurs, the specified value in the entry field will remain until either a new value is entered or the clear button is clicked. If a match was not found for the given string, a message window will pop-up specifying that the given string was not found. Clicking the **OK** button will cause this message window to close and the entry field will automatically be cleared.

4.2.5 The Information Bar

The information bar at the bottom of each window in the report viewer provides information about the current mode, file, or operation the user should take next. All information is state and context sensitive. If you are in doubt of what to do next, first see the information bar.



Covered Main Information Bar

4.3 Navigating the Line Coverage Window

Line coverage is the quickest way to determine what logic in your design is being exercised and which logic has not be exercised during simulation. If a line was found to not be simulated, this is an indication that there was some event that did not occur or did not occur at the proper time to cause a previous conditional expression to be evaluated to true or false. For example, if a block of code looked something like:

```
always @(posedge clock)  // Line 1
  if( a )                // Line 2
    b <= 1'b0;           // Line 3
  else                   // Line 4
    b <= 1'b1;           // Line 5
```

If line 3 was shown to be not covered, this would indicate that at the positive edge of the term "clock", the value of "a" never evaluated to a value of 1 (true). To make line 3 become covered, we would need to setup a situation in the diagnostic that caused "a" to become true at the positive edge of the term "clock".

On the other hand, if all lines 2 - 5 were found to be uncovered, this would indicate the the term "clock" never made a transition from some value to a 1 (true) value during simulation.

4.3.1 Viewing Uncovered Lines

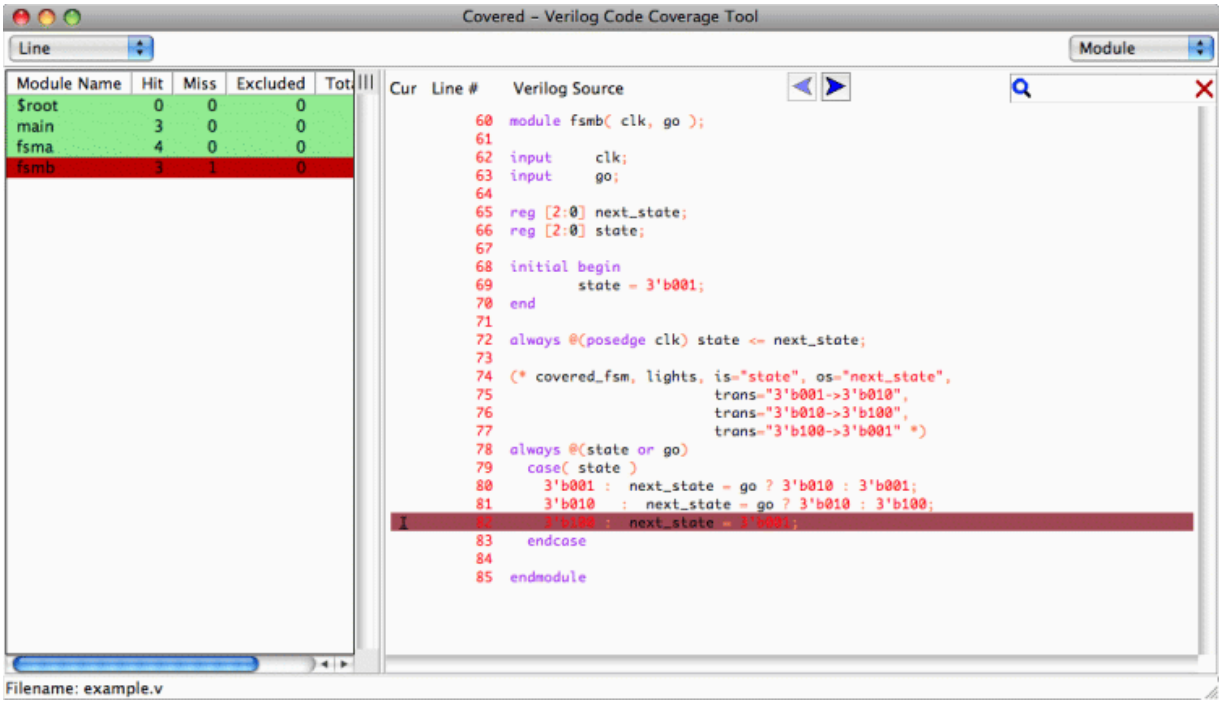
To view line coverage for all uncovered lines for a specific module/instance, do the following:

1. Make sure that the **Show Uncovered** menu item in the report menu is selected.
2. Select the line coverage metric in the coverage bar by left-clicking on selection button to the left of "Line".
3. Select one of the modules/instances that is highlighted in the uncovered color scheme (to change this color scheme, see Navigating the Preferences Window) in the listbox. This will cause the selected module/instance source code to be displayed in the file viewer.

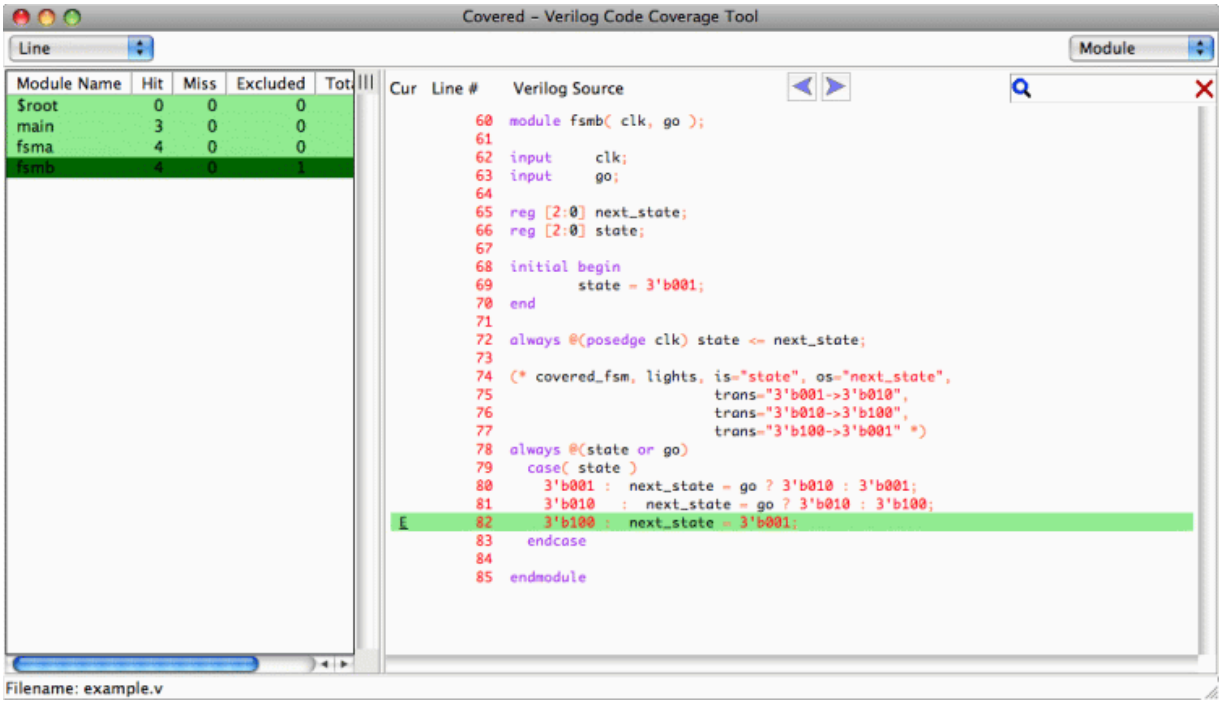
Once the selected module/instance has been displayed, scroll down in the Coverage File Viewer window (using the scroll bars to the right and below the Coverage File Viewer window) until you have reached a line that is highlighted in the uncovered color scheme. The line number of the highlighted line will be printed to the left of the source code line. This line represents a line that was found to not be hit during the simulation of this module/instance.

4.3.2 Excluding/Included Uncovered Lines from Line Coverage

Any uncovered line in the design may be included/excluded from coverage consideration. To the left of each uncovered line is an underlined "I" (line is currently included in coverage) or "E" (line is currently excluded in coverage). Clicking on an "I" will cause a exclusion reason popup window to be displayed, allowing you to specify a reason for excluding this line. Once a reason has been entered, the coverage exclusion property of the line will change to an "E" and be highlighted in the "covered" color scheme and the summary line coverage information will be recalculated and redisplayed in the listbox to reflect the current status of line coverage for the current module/instance. Clicking on an "E" will cause the current line to be displayed in the "uncovered" color scheme, the "E" will change to an "I", and the summary line coverage will be recalculated and redisplayed in the summary listbox. Example of uncovered line included for line coverage consideration shows an uncovered line that is included for coverage consideration. Example of uncovered line excluded for line coverage consideration shows the same line which is now excluded from coverage consideration.



Example of uncovered line included for line coverage consideration



Example of uncovered line excluded for line coverage consideration

In addition to changing the line coverage information, the combinational logic coverage information for an uncovered statement on that line will also be implicitly excluded from coverage (though the exclude properties for each subexpression of the statement will not change).

If the user excludes any line in the design, the **Save CDD...** option in the file menu will be enabled. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a

different time and retain the knowledge of which lines have been excluded/included for coverage. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded line information is preserved for the resulting merged CDD file.

If a line has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by right-click-and-hold on the "E" on the desired line. This will cause a tooltip-style window to be displayed specifying the reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply release the button.

4.3.3 Viewing Covered Lines

To view line coverage for all covered lines for a specific module/instance, do the following:

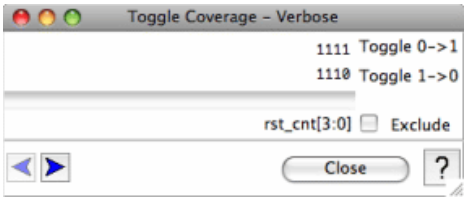
1. Make sure that the **Show Covered** menu item in the report menu is selected.
2. Select the line coverage metric in the coverage bar by left-clicking on the selection button to the left of "Line".
3. Select one of the modules/instances in the listbox that shows that at least one line was considered coverable by Covered.

Once the selected module/instance has been displayed, scroll down in the Coverage File Viewer window (using the scroll bars to the right and below the Coverage File Viewer window) until you have reached a line that is highlighted in the covered color scheme. The line number of the highlighted line will be printed to the left of the source code line. This line represents a line that was found to be hit during the simulation of this module/instance.

You may see both the covered and uncovered line coverage simultaneously by selecting both the **Show Uncovered** and **Show Covered** items in the **Report** menu. This is sometimes useful to see what logic Covered was not able to provide coverage support for (all lines which contain code that are not highlighted in either the uncovered or covered color schemes).

4.4 Navigating the Toggle Coverage Window

The verbose toggle window allows the user to see which bits for a given signal have toggled from a 0->1 or a 1->0 during simulation. To display this window, simply click on a highlighted/underlined uncovered signal in the file viewer of the main window. This will cause the verbose toggle viewer to be created for the specified signal as well as display a "-->" symbol in the Coverage Viewer on the same line as the selected signal. This symbol is simply meant to help the user identify in the Coverage Viewer which signal is currently displayed in the Toggle Viewer window. Verbose Toggle Window shows the Toggle Viewer loaded with a 4-bit signal called "err_vec" in which bits 3:2 have successfully toggled from a 0->1.



Verbose Toggle Window

4.4.1 Understanding the Layout

The bit vector displayed is a binary bit vector, each value (0 or 1) represents whether that bit in the signal either toggled (1) or did not toggle (0) during simulation. There are two bit vectors displayed for the signal, the top vector shows the 0->1 toggle information while the bottom vector shows the 1->0 toggle information.

The name of the signal along with its MSB:LSB information is displayed at the bottom of the window; however, if the mouse cursor is placed over a specific bit in the toggle box, the MSB:LSB will be changed to show the displayed bit, making it easy for the user to discern exactly which bit has toggled/not toggled. If the width of the signal exceeds the given space for the toggle window, a scrollbar will be allowed to slide left and right to view the rest of the toggle information.

To exit this window, simply click on **Close** button. To see this help page from the toggle window, click on the help (?) button.

4.4.2 Excluding/Including a Signal for Toggle Coverage

The user may exclude/include any uncovered signal in the design for toggle coverage by simply clicking on the **Excluded** checkbutton. If this checkbutton is checked, the exclusion reason popup window will be displayed, allowing the user to specify a reason for excluding this signal from toggle coverage. Once a reason has been specified, the summary coverage information will automatically be recalculated and redisplayed in the listbox and the background color of the underlined signal in the file viewer will change from the uncovered background color to the covered background color. However, the signal will still remain underlined, allowing the user to see the verbose toggle coverage

information, if necessary. If the **Excluded** checkbutton is unchecked, the summary coverage information and background colors will be readjusted to include the coverage information for this signal.

If the user excludes any signal for toggle coverage in the design, the **Save CDD...** option in the file menu will be enabled. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a different time and retain the knowledge of which signals have been excluded/included for toggle coverage. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded signal information is preserved for the resulting merged CDD file.

If a signal has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by right-click-and-holding on the **Excluded** checkbutton. This will cause a tooltip-style window to be displayed specifying the reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply release the button.

4.4.3 Displaying a New Signal for Toggle Coverage

If the Verbose Toggle Coverage window is currently displayed with a signal, the user may select a new signal in the file viewer of the Main window. When a new signal is selected, its value immediately replaces the currently selecting signal value. This feature allows many different signals to be viewed one after the other without having to create/destroy a new window for each signal.

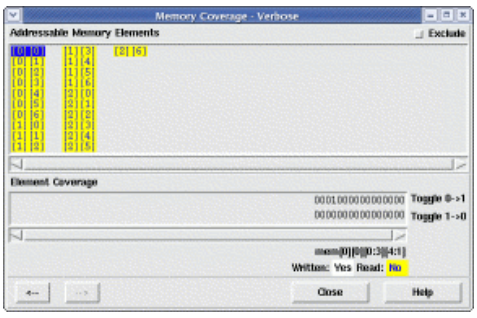
The user may also see the previous or next uncovered signal in the main viewer by clicking either the left arrow button (for previous signal viewing) or the right arrow button (for next signal viewing). Note that the left arrow button will be disabled if there is not a signal previous to the current signal in the given module and the right arrow button will be disabled if there is not a signal after the current signal in the given module.

4.5 Navigating the Memory Coverage Window

The verbose memory window allows the user to interactively view all of the coverage information for a selected memory, including the following for each addressable memory element:

1. Have all bits toggled from 0 -> 1 and from 1 -> 0?
2. Has the entry been written?
3. Has the entry been read?

To display this window, simply click on a highlighted/underlined uncovered memory in the file viewer. This will cause the verbose memory viewer to be created for the specified memory as well as display a "-->" symbol in the Coverage Viewer on the same line as the selected memory. This symbol is simply meant to help the user identify in the Coverage Viewer which memory is currently displayed in the Memory Viewer window. Figure 1 shows the Memory Viewer loaded with a 4-dimensional memory called "mem" in which two of the dimensions are unpacked (all combinations of unpacked dimensions make up the total number of addressable elements in the memory), creating a total of 18 addressable elements. The first addressable element "[0][0]" is shown in the lower frame in which only a single bit has been toggled from 0 to 1, the element has been written, but it has not been read.



Verbose Memory Window

4.5.1 Understanding the Layout

The Memory Coverage window is split up into three main frames stacked from top to bottom. These frames, in order from top to bottom, are the "Addressable Memory Elements" frame, the "Element Coverage" frame, and the button frame. Each are described in detail below.

4.5.1.1 Addressable Memory Elements Frame

The uppermost frame contains all of the addressable memory elements (AMEs) of the memory. An AME is a single memory entry that is addressed by specifying a unique combination of values for each of the unpacked dimensions of the memory. For example, consider the

following memory:

```
reg [15:0] foo[0:1][0:3];
```

This memory called "foo" contains three dimensions. One packed dimension which is a 16-bit little endian value and two unpacked dimensions, containing two entries of four entries (for a total of eight addressable memory elements). Each AME being 16 bits wide. The AMEs for this memory would be:

- foo[0][0]
- foo[0][1]
- foo[0][2]
- foo[0][3]
- foo[1][0]
- foo[1][1]
- foo[1][2]
- foo[1][3]

The upper frame will contain all of the AMEs for the selected memory. The uncovered AMEs will be highlighted in the uncovered color scheme (as specified in the preferences window) while all fully covered AMEs will remain unhighlighted. To view the coverage information for any uncovered AME, simply click on it and the information will be viewable in the "Element Coverage" frame below it.

4.5.1.2 Element Coverage Frame

The Element Coverage frame shows all of the coverage information for the currently selected AME including toggle coverage (which bits of the element toggled from 0->1 and 1->0), write coverage (was this entry written to) and read coverage (was this entry accessed).

The bit vector displayed in the toggle window is a binary bit vector, where each value (0 or 1) represents whether that bit in the signal either toggled (1) or did not toggle (0) during simulation. There are two bit vectors displayed for the AME, the top vector shows the 0->1 toggle information while the bottom vector shows the 1->0 toggle information.

The name of the memory along with its dimensional range information is displayed below the toggle window; however, if the mouse cursor is placed over a specific bit in the toggle box, the packed dimensional range information will be changed to show the displayed bit, making it easy for the user to discern exactly which bit has toggled/not toggled. If the width of the AME exceeds the given space for the toggle window, a scrollbar will be allowed to slide left and right to view the rest of the toggle information.

Below the memory name are two other pieces of coverage information about the AME. The value to the right of the **Written:** label will be set to either "Yes" or "No". If the value is "Yes", this indicates that the currently selected AME was written during simulation and value will be highlighted with the covered color scheme (as selected in the preferences window). If the value is "No", this indicates that the currently selected AME was not written during simulation and the value will be highlighted with the uncovered color scheme.

To the right of the **Written** information is the **Read** label which will also be set to either a value of "Yes" or "No". If the value is "Yes", this indicates that the currently selected AME was accessed during simulation and the value will be highlighted with the covered color scheme. If the value is "No", this indicates that the currently selected AME was not accessed during simulation and the value will be highlighted with the uncovered color scheme.

4.5.1.3 Button Frame

The button frame contains buttons for closing the Memory Coverage window, displaying this help page, or moving to the next/previous uncovered memory in the same module/instance.

To exit this window, simply click on **Close** button. To see this help page from the toggle window, click on the **Help** button.

4.5.2 Excluding/Including a Memory for Memory Coverage

The user may exclude/include any uncovered memory in the design for memory coverage by simply clicking on the **Excluded** checkbutton in the upper right-hand portion of the Memory Coverage window. If this checkbutton is checked, the exclusion reason popup window will be displayed automatically, allowing the user to specify a reason for excluding the memory. Once a reason has been entered, the summary coverage information will automatically be recalculated and redisplayed in the listbox and the background color of the underlined memory in the file viewer will change from the uncovered background color to the covered background color. However, the memory will still remain

underlined, allowing the user to see the verbose memory coverage information, if necessary. If the **Excluded** checkbutton is unchecked, the summary coverage information and background colors will be readjusted to include the coverage information for this memory.

If the user excludes any memory for memory coverage in the design, the **Save CDD...** option in the file menu. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a different time and retain the knowledge of which memories have been excluded/included for memory coverage. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded memory information is preserved for the resulting merged CDD file.

If a memory has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by right-click-and-holding on the **Excluded** checkbutton. This will cause a tooltip-style window to be displayed specifying the reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply release the button.

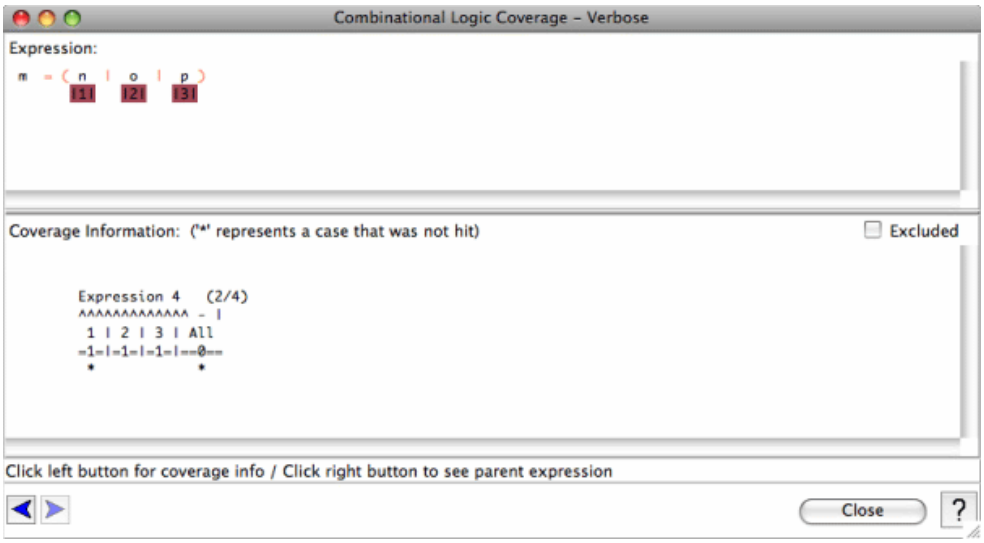
4.5.3 Displaying a New Memory for Memory Coverage

If the Verbose Memory Coverage window is currently displayed with a memory, the user may select a new memory in the file viewer of the Main window. When a new memory is selected, its value immediately replaces the currently selecting memory value. This feature allows many different memories to be viewed one after the other without having to create/destroy a new window for each memory.

The user may also see the previous or next uncovered memory in the main viewer by clicking either the left arrow button (for previous memory viewing) or the right arrow button (for next memory viewing). Note that the left arrow button will be disabled if there is not a memory previous to the current memory in the given module and the right arrow button will be disabled if there is not a memory after the current memory in the given module.

4.6 Navigating the Combinational Logic Window

The Verbose Combinational Logic viewer allows the user to look at the subexpressions of a selected statement to determine exactly why a statement did not achieve 100% coverage. The window is split up into two text frames. The top frame displays the selected statement, underlining and numbering subexpressions that were found to not achieve full coverage. The bottom frame displays which combinations of a selected subexpression were hit/missed, allowing the user to discern which combinations of logic are still in need of verification. Figure 1 shows the verbose combination logic window.



Example of the verbose combinational logic window

4.6.1 Navigating a Statement

To understand what subexpressions of a statement caused it to not achieve 100% coverage, the viewer window allows the user to traverse uncovered subexpressions, displaying the coverage information for each subexpression in the lower frame. To see an uncovered subexpression in the lower frame, simply click the left mouse on the underlined subexpression. If the subexpression has one or more children subexpressions, these will be individually underlined and numbered. If the subexpression is a leaf subexpression of the statement, only its coverage information will be displayed in the lower pane. To traverse upward in a subexpression, simply click the right mouse button on the underlined subexpression. This will cause the parent subexpression to be underlined and its parent (if one exists) will have its coverage information displayed in the lower frame.

To exit this window, simply click on **Close** button. To see this help page for the combinational logic window, click on the help (?) button.

4.6.2 Understanding the Coverage Information

The coverage information for a selected subexpression can be one of four different types:

4.6.2.1 Event Subexpression

These subexpressions do not have any children expressions. Their coverage is in terms of:

- Did this event occur?

The following operators are considered events:

- **At-event operators**:

```
@(posedge expression)
@(negedge expression)
@(expression)
@*
```

- **Event trigger operator**:

```
-> event_variable
```



Example Coverage Output for an Uncovered Event Subexpression

In this diagram, we see that a certain `@(posedge clock)` expression never occured.

4.6.2.2 Unary Subexpression

These subexpressions do not have any children expressions. Their coverage is in terms of:

- Did this expression evaluate to a value of 0?
- Did this expression evaluate to a non-zero value?

The following operators are considered unary subexpressions:

- **Signal/Bit Selectors**:

```
signal_value
signal_value[expression]
signal_value[value:value]
signal_value[expression+:value]
signal_value[expression-:value]
```

- **Function call**:

```
function_name( ... )
```

- **Relational operators**:

```
!expression
expression  <    expression
expression  >    expression
expression  <=   expression
expression  >=   expression
expression  ==   expression
expression  !=   expression
expression  ===  expression
expression  !==  expression
```

- **Shift operators**:

```
expression <<  value
expression >>  value
expression <<< value
expression >>> value
```

- **Conditional operator**:

```
expression ? expression : expression
```

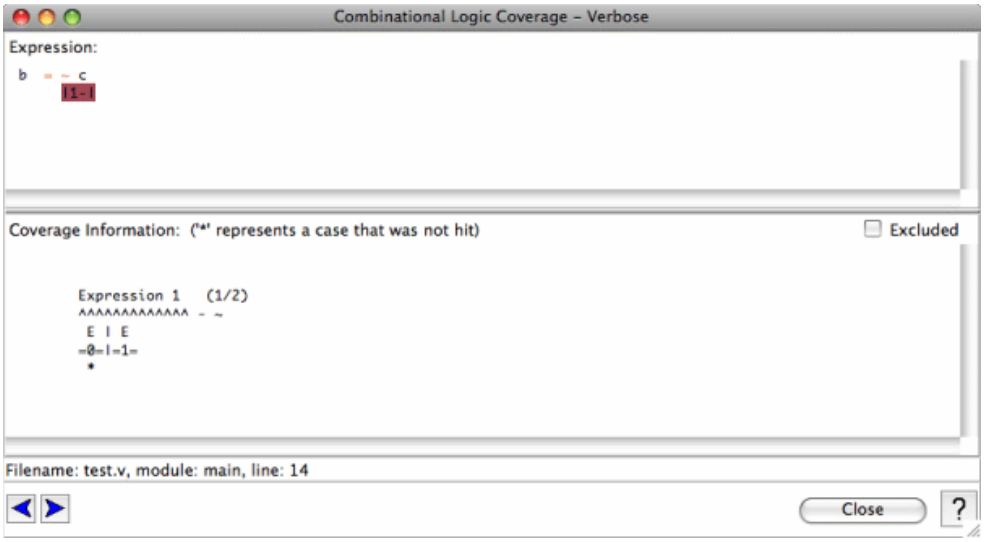
- **Unary operators**:

```
expression
&expression
|expression
^expression
~&expression
~|expression
~^expression
```

- **Concatenation operators**:

```
{value{expression} }
{expression, expression, ... }
```

- **Case equality**



Example Coverage Output for an Uncovered Unary Subexpression

4.6.2.3 Simple Combinational Subexpression

These subexpressions have two child expressions (L=left and R=right). There are three flavors of simple combinational subexpressions: AND-type, OR-type and other. The differences are explained below.

4.6.2.3.1 Simple AND-type Combinational Subexpressions

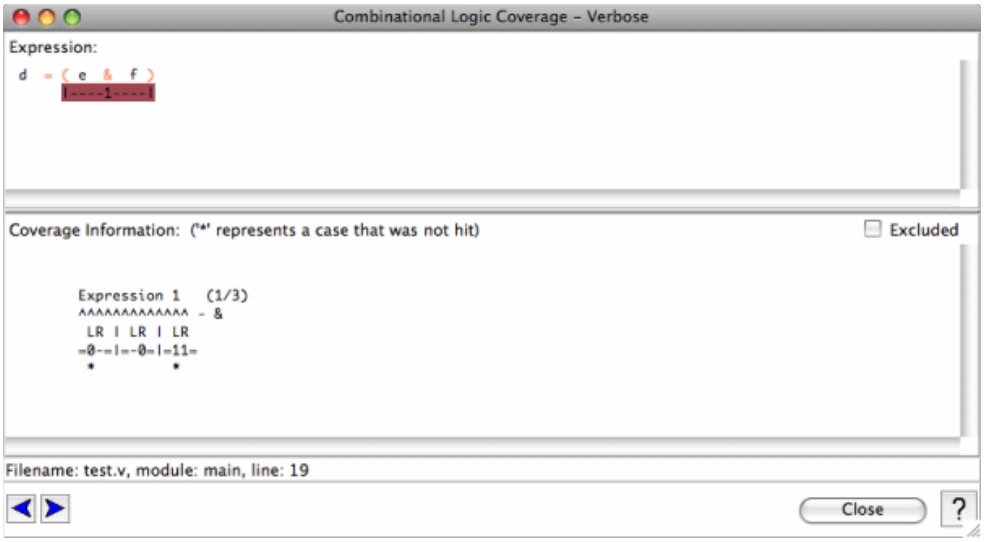
These subexpressions have three possible combinational outcomes:

- Did the left expression evaluate to 0?
- Did the right expression evaluate to 0?
- Did the left expression evaluate to 1 when the right expression evaluated to 1?

The following operators are considered simple AND-like combinational subexpressions:

- **Logical operators**:

```
expression  &   expression
expression  ~&  expression
expression  &&  expression
```



Example Coverage Output for an Uncovered Simple AND-type Combinational Subexpression

4.6.2.3.2 Simple OR-type Combinational Subexpressions

These subexpressions have three possible combinational outcomes:

- Did the left expression evaluate to 1?

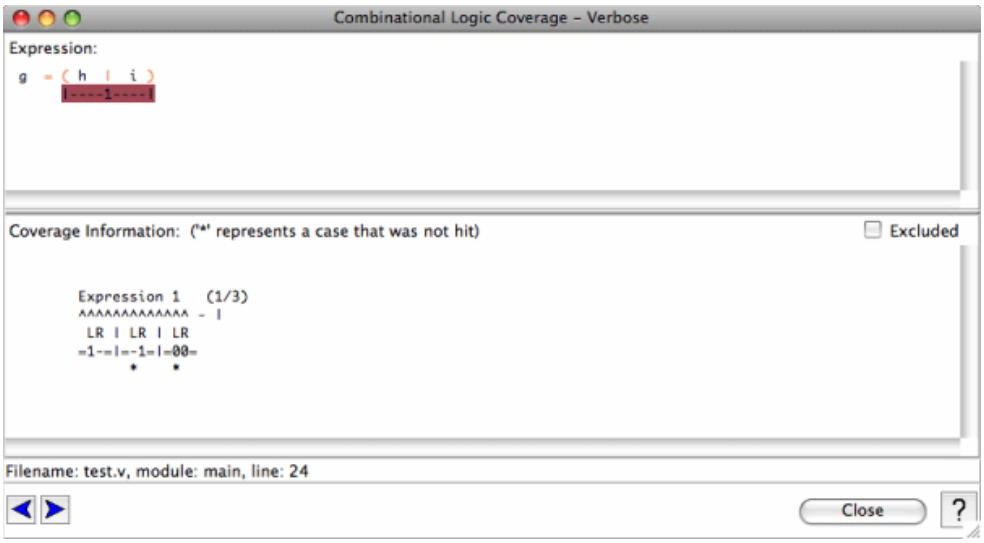
  Did the right expression evaluate to 1?

  Did the left expression evaluate to 0 when the right expression evaluated to 0?

The following operators are considered simple OR-like combinational subexpressions:

- **Logical operators**

- ```
  expression  |   expression
  expression  ~|  expression
  expression  ||  expression
  ```
  ```



Example Coverage Output for an Uncovered Simple OR-type Combinational Subexpression

4.6.2.3.3 Simple Other-type Combinational Subexpressions

These subexpressions have four possible combinational outcomes:

- Did the left expression evaluate to 0 when the right expression evaluated to 0?
- Did the left expression evaluate to 0 when the right expression evaluated to 1?
- Did the left expression evaluate to 1 when the right expression evaluated to 0?
- Did the left expression evaluate to 1 when the right expression evaluated to 1?

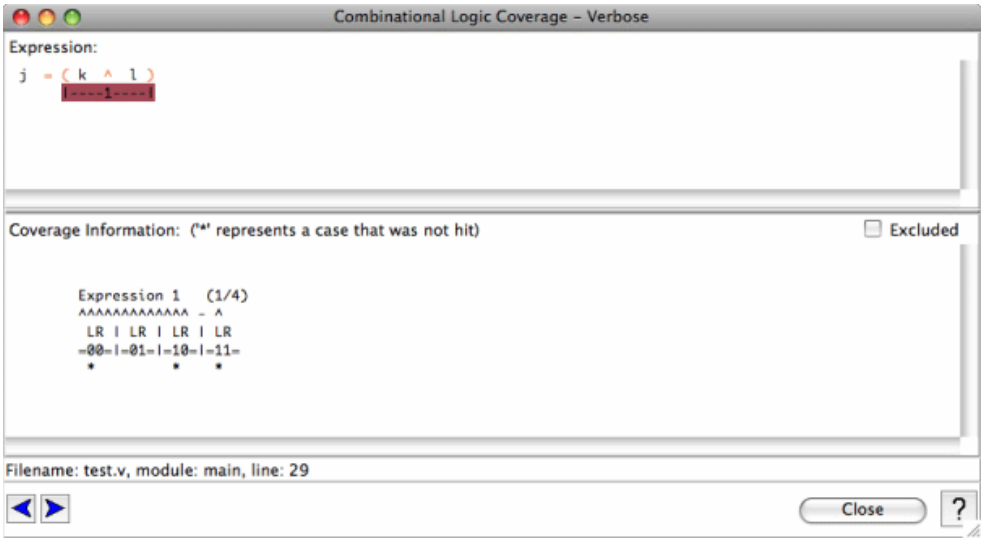
The following operators are considered simple combinational subexpressions:

- **Mathematical operators**:

```
expression + expression
expression – expression
```

- **Logical operators**:

```
expression ^  expression
expression ~^ expression
```



Example Coverage Output for an Uncovered Simple Combinational Subexpression

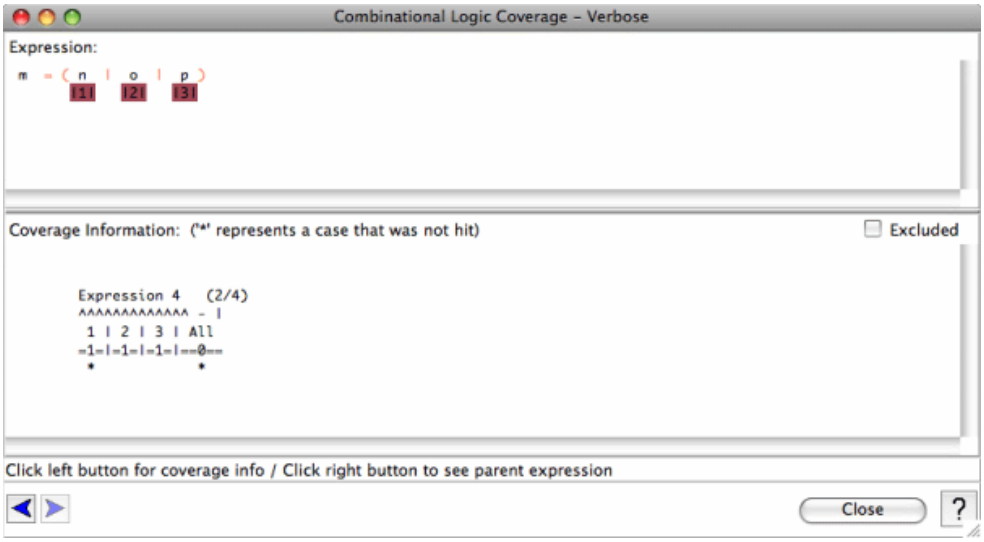
4.6.2.4 Complex Combinational Subexpression

These subexpression have three or more child expressions that all are connected by the same operator. For example, the statement `a = (b & c & d)` would contain the complex combinational subexpression of `(b & c & d)` where b, c and d are the three child expressions all combined using the bitwise AND operator.

The effect of each subexpression on the value of the entire subexpression as well as the combination of each child toward the parent is displayed. For example, if the combinational operator is a bitwise OR, each child is checked to see if it evaluated to TRUE (in an OR operation, only one subexpression needs to evaluate to TRUE for the entire subexpression to be TRUE). Additionally, all child subexpressions are checked to see if all evaluated to 0 simultaneously (this is the only way to get the whole subexpression to evaluate to 0).

The unique identifier below each subexpression is used in the displayed coverage information for these types of subexpressions for identifying which subexpression did not behave correctly.

All simple combinational subexpressions can be output as complex combinational subexpressions if there are three or more child subexpressions to the operation.

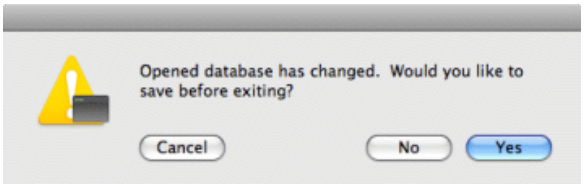


Example Coverage Output for an Uncovered Complex Combinational Subexpression

In this example, subexpression 1 never evaluated to a value of TRUE (which would have made the entire expression TRUE) and all three subexpressions 1, 2, and 3 were never a value of 0 simultaneously (which would have made the entire expression FALSE). However, subexpressions 2 and 3 both evaluated to TRUE at some point.

4.6.3 Excluding/Including a Subexpression

For any displayed subexpression within a statement, the user may choose to disregard the coverage information for that subexpression by clicking on the **Excluded** checkbutton on the right side of the window. If this option is checked, Covered will display a exclusion reason popup window to allow the user to specify a reason for excluding the subexpression. Once a reason has been entered, the combinational logic summary coverage information will be recalculated for the current module/instance by adding to the "Exclude" count the number of uncovered combinational logic values for the given subexpression (and all subexpressions below this expression within the expression tree). This new summary information will be redisplayed on the listbox. The subexpression background color in the top frame will also change from the uncovered background color to the covered background color. However, the subexpression will still contain an underline that may be clicked on at any time to view its coverage information or the coverage information for any subexpressions that are uncovered. To include a previously excluded subexpression, simply uncheck the **Excluded** checkbutton. This will automatically cause the summary information and background color to be updated for the given subexpression. Example of an excluded subexpression below shows an example of a subexpression that has been excluded from coverage consideration.



Example of an excluded subexpression

If an expression tree has been entirely excluded from coverage (all subexpressions have the **Excluded** checkbutton checked), the background color in the file viewer will change from an uncovered background color to the covered background color to indicate at the top-level that this entire statement has been excluded from coverage. However, the statement will still be underlined in the Main Coverage File Viewer to allow the user to view the statement, if necessary.

If the user excludes any subexpression in the design, the **Save CDD...** option in the file menu will be enabled. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a different time and retain the knowledge of which subexpressions have been excluded/included for coverage. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded subexpression information is preserved for the resulting merged CDD file.

If a subexpression has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by selecting the subexpression in the top window (so that it is displayed in the lower window and right-click-and-holding on the **Excluded** checkbutton. This will cause a tooltip-style window to be displayed specifying the reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply release the button.

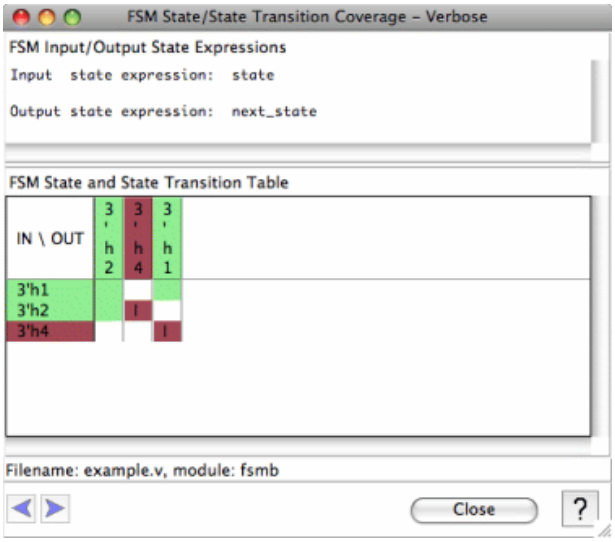
4.6.4 Displaying a New Statement

If the Verbose Combinational Logic Coverage window is currently displayed with a statement, the user may select a new statement in the file viewer of the Mainwindow. When a new statement is selected, its value immediately replaces the currently selected statement. This feature allows many different statements to be viewed one after the other without having to create/destroy a new window for each.

The user may also see the previous or next uncovered statement in the main viewer by clicking either the left arrow button (for previous signal viewing) or the right arrow button (for next signal viewing). Note that the left arrow button will be disabled if there is not a signal previous to the current signal in the given module and the right arrow button will be disabled if there is not a signal after the current signal in the given module.

4.7 Navigating the FSM State/State Transition Coverage Window

The Verbose FSM State/State Transition Coverage window allows the user to see which states and state transitions of a currently selected FSM have been covered or are uncovered. To view this window, simply select the "FSM" coverage metric mode in the coverage bar in the main window and select an underlined FSM state variable in the file viewer. This will cause the FSM Coverage window to be displayed with the currently selected FSM information loaded. Verbose FSM Coverage Window below shows an example of this window.



Verbose FSM Coverage Window

4.7.1 Navigating the FSM Window

The Verbose FSM window is split into two main frames. The top frame contains the expressions that make up the input and output states of the selected FSM. The bottom frame contains a two-dimensional table showing the covered and/or uncovered states and state transitions.

The state/state transition table is a two-dimensional table with all possible values of the input states in the left-most column, the output states in the upper-most row, and the state transitions in the rest of the table.

To determine which states have been uncovered in the table, simply find all state values in the input column or output row (both show the same information) which are highlighted with the "Uncovered" color selection specified in the preferences window. To determine which states have been covered in the table, simply find all state values in the input column or output row which are highlighted with the "Covered" color selection. All other state values which are not highlighted are either impossible values (if all of the state transitions were specified in the Verilog file) or are unknown for coverage (if not all of the state transitions were specified in the Verilog file).

To determine which state transitions have been uncovered in the table, simply find all empty boxes in the table that are highlighted with the "Uncovered" background color. If one of these exist, it specifies that the transition of "input state" (the value specified in the left-most

column in the same row) to "output state" (the value specified in the upper-most row in the same column) did not occur. In the example above, we can see that the following state transitions did not occur:

- 2 -> 4
- 4 -> 1

Follow the same procedure for finding all covered state transitions in the table except that these boxes will be highlighted with the "Covered" background color. In the example above, we can see that the following state transitions occurred:

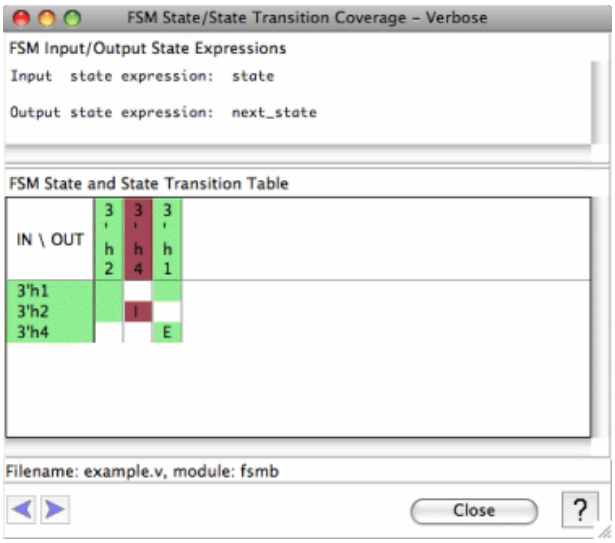
- 1 -> 2
- 1 -> 1 (loopback case)
- 2 -> 2 (loopback case)

All other state transition boxes that are not highlighted are either impossible state transitions (if all state transitions were specified in the Verilog source code) or are unknown in their coverage (either not hit or impossible transition if all state transitions were not specified in the Verilog source code).

To exit this window, simply click on **Close** button. To see this help page for the FSM window, click on the help (?) button.

4.7.2 Excluding/Including a State Transition

Uncovered state transitions can be excluded/included for FSM coverage consideration. This is done by clicking on the "I" (state transition is currently included in coverage results) or "E" (state transition is currently excluded from coverage results) character in the middle of a state transition square. Changing a state transition from an "I" to an "E" will automatically cause a exclusion reason popup window to be displayed, allowing you to specify the reason for excluding this state transition. After a reason has been entered, the state transition summary coverage information in the listbox to be updated and the background color of the state transition box will change from the uncovered color to the covered color. Changing a state transition from an "E" to an "I" will automatically cause the state transition summary coverage information in the Main Window Summary Bar to be updated and the background color of the state transition box will change from the covered color to the uncovered color. Example of excluded state transitions below shows an example of the Verbose FSM Coverage Window state machine coverage information with a couple of state transitions excluded from coverage.



Example of excluded state transitions

If all of the state transitions for a state machine have been excluded for coverage consideration, the output state variable in the file viewer will change its background from the uncovered to the covered background color. However, the underline will remain under the variable to allow the user to still examine the verbose FSM information for that state machine.

If the user excludes any state transition in the design, the **Save CDD...** option in the file menu will be enabled. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a different time and retain the knowledge of which state transitions have been excluded/included for coverage. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded state transition information is preserved for the resulting merged CDD file.

If a state transition has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by moving the cursor over the state transition. This will cause a tooltip-style window to be displayed specifying the state transition and the

reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply move the cursor off of the state transition cell.

4.7.3 Displaying a New FSM

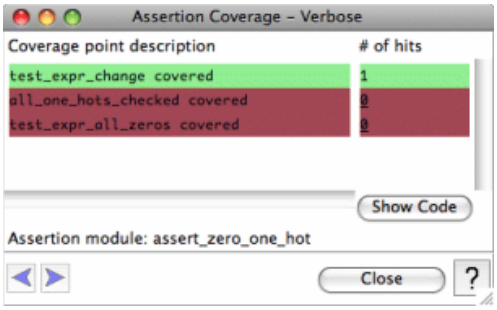
If the Verbose FSM Coverage window is currently displayed, the user may select a new FSM state variable in the file viewer of the Main window. When a new FSM is selected, its information immediately replaces the currently selected FSM. This feature allows many different FSMs to be viewed one after the other without having to create/destroy a new window for each.

The user may also see the previous or next uncovered FSM in the main viewer by clicking either the left arrow button (for previous signal viewing) or the right arrow button (for next signal viewing). Note that the left arrow button will be disabled if there is not a signal previous to the current signal in the given module and the right arrow button will be disabled if there is not a signal after the current signal in the given module.

4.8 Navigating the Assertion Coverage Window

The Verbose Assertion Coverage window allows the user to see exactly which coverage points were hit or missed for a selected assertion instantiation in the design and, if hit, how many times the assertion coverage point (ACP) was hit during simulation. Additionally, this window allows you to bring up the verbose assertion window to view the internal assertion module code for the currently selected assertion instance. This lets the user view the code to determine what logically needs to occur to hit a given ACP.

To view this window, select the "Assert" mode in the coverage bar in the main window and click on an underlined assertion instance name in the file viewer. Note that both covered and uncovered assertion instance names can be selected. Doing so will cause the Verbose Assertion Coverage Window to be displayed for the selected assertion instantiation. Verbose Assertion Coverage Window shows an example of a zero_one_hot assertion that had the "test_expr_change covered" ACP covered (hit twice during simulation) and the other two ACPs uncovered during simulation.



Verbose Assertion Coverage Window

4.8.1 Navigating the Assertion Coverage Window

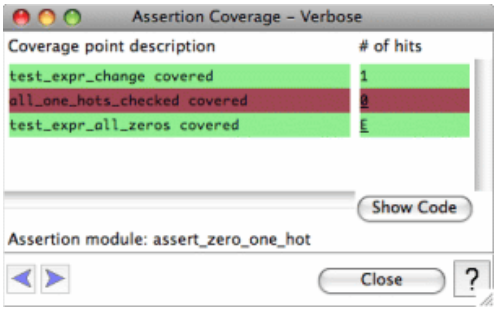
The window is made up of a simple table which contains the coverage point names within the selected assertion on the left and the number of times each of these coverage points were hit during simulation on the right. If a coverage point has not been hit, both the coverage point name and the number of hits (0) will be highlighted with the "uncovered" color selection as selected in the preferences window. If a coverage point has been hit at least once, both the coverage point name and the number of hits will be highlighted with the "covered" color selection as selected in the Preferences window.

Below the table the name of the currently selected assertion type is named. To view the source code for this assertion, simply click on the **Show Code** button at the bottom right-hand portion of the coverage table. This will cause the source code for this assertion to be displayed in a new assertion source viewer.

To exit this window, simply click on **Close** button. To see this help page for the assertion coverage window, click on the help (?) button.

4.8.2 Excluding/Including Assertion Coverage Points

Individual ACPs can be excluded/included for coverage consideration using the verbose assertion coverage window. This is accomplished by clicking on the "0" (ACP is currently included in coverage) or "E" (ACP is current excluded from coverage) value in the "# of hits" column. Changing an ACP from "0" to "E" (excluding the ACP from coverage consideration) will cause the exclusion reason popup window to be displayed, allowing you to specify a reason for excluding the ACP. Once a reason has been entered, the summary assertion coverage information to be recalculated and redisplayed in the listbox and the background color of the ACP will change from the uncovered to the covered background color. Changing an ACP from "E" to "0" (including the ACP for coverage consideration) will cause the summary assertion coverage information to be recalculated and redisplayed in the Main Window Summary Bar and background color of the ACP will change from the covered to the uncovered background color. Example of excluded ACP below shows the above assertion instance with the last coverage point excluded from coverage.



Example of excluded ACP

If all ACPs for a given assertion instance have been excluded from coverage consideration, the file viewer will automatically change the background color of the assertion instance name from the uncovered to the covered background color. The assertion instance name will still be underlined, allowing the user to still see the coverage information for that given assertion instance.

If the user excludes any ACP in the design, the **Save CDD...** option in the file menu will be enabled. This allows the user to save the excluded property either back to the CDD file (or to a newly named CDD file). Doing so will allow the user to load the CDD file at a different time and retain the knowledge of which ACPs have been excluded/included for coverage along with any specified reasons for exclusion. Additionally, if a saved CDD file with coverage exclusion is merged with another CDD file, the excluded ACP information is preserved for the resulting merged CDD file.

If an ACP has been excluded from coverage and an exclusion reason was specified, you can view the reason for exclusion by right-click-and-holding on the "E" next to the ACP. This will cause a tooltip-style window to be displayed specifying the reason for exclusion. The background color of the tooltip will be the same color as the "covered" background color to help distinguish it from a normal tooltip. To cause the exclusion reason tooltip to disappear, simply release the button.

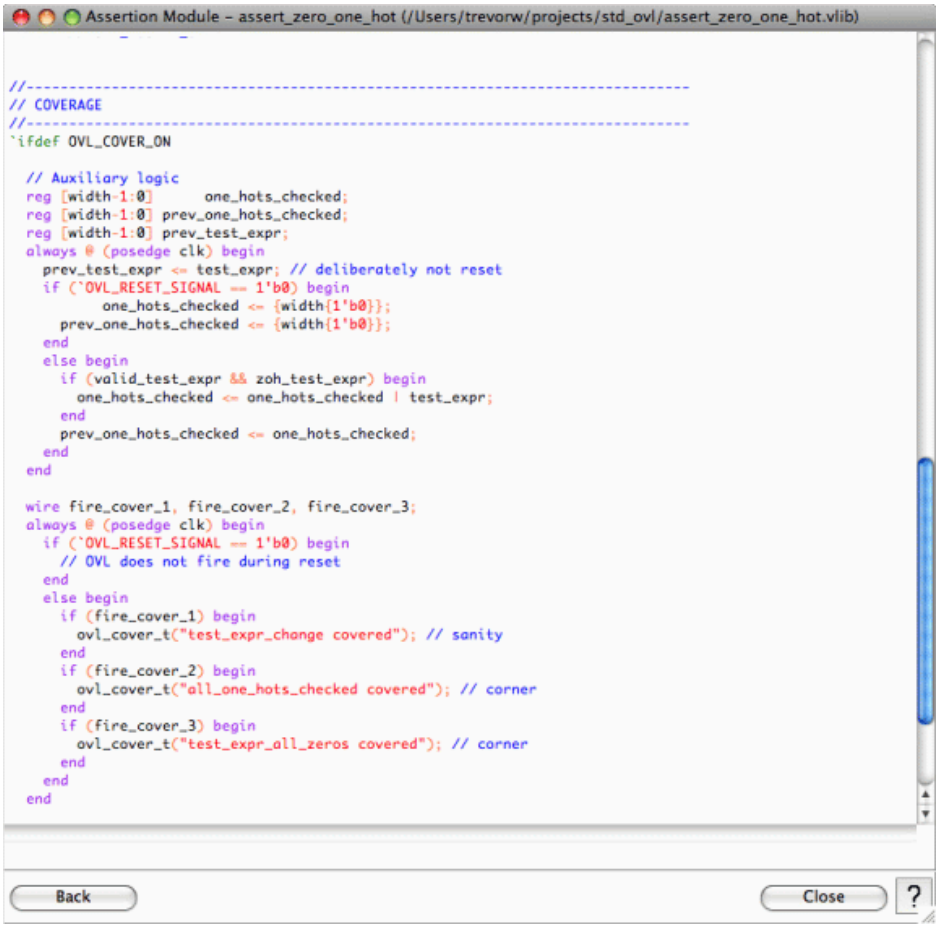
4.8.3 Displaying a New Assertion Instance

If the Verbose Assertion Coverage window is currently displayed, the user may select a new assertion in the file viewer of the Main window. When a new assertion is selected, its information immediately replaces the currently selected assertion. This feature allows many different assertions to be viewed one after the other without having to create/destroy a new window for each.

The user may also see the previous or next uncovered assertion in the main viewer by clicking either the left arrow button (for previous signal viewing) or the right arrow button (for next signal viewing). Note that the left arrow button will be disabled if there is not a signal previous to the current signal in the given module and the right arrow button will be disabled if there is not a signal after the current signal in the given module.

4.9 Navigating the Assertion Source Code Window

The Assertion Source Viewer window allows the user to see the source code for the current assertion in the verbose assertion window. To view this window, simply click the **Show Code** button in the Verbose Assertion Coverage window. Assertion Source Viewer Window shows a zero_one_hot OVL assertion code with syntax highlighting applied.



Assertion Source Viewer Window

4.9.1 Navigating the Source Code

The source code for the current assertion will be automatically displayed in the text viewer when the window is created. However, the source code will be displayed as it is found in the file system, without preprocessing applied to it. This means that included files will not be displayed in an in-lined fashion. To view included files, simply click on the included filename (it will be underlined). This will cause the included file to replace its includer in the text viewer. An infinite number of includes may be traversed in this manner. To go back to the file that included the current file, simply click on the **Back** button located in the bottom left-hand corner of the window. This will cause the current source code to be replaced with its "parent" source code.

Note that the **Back** button will be disabled when the top-level assertion file is currently displayed.

To exit this window, simply click on **Close** button. To see this help page for the assertion source code window, click on the help (?) button.

4.10 Creating a New CDD

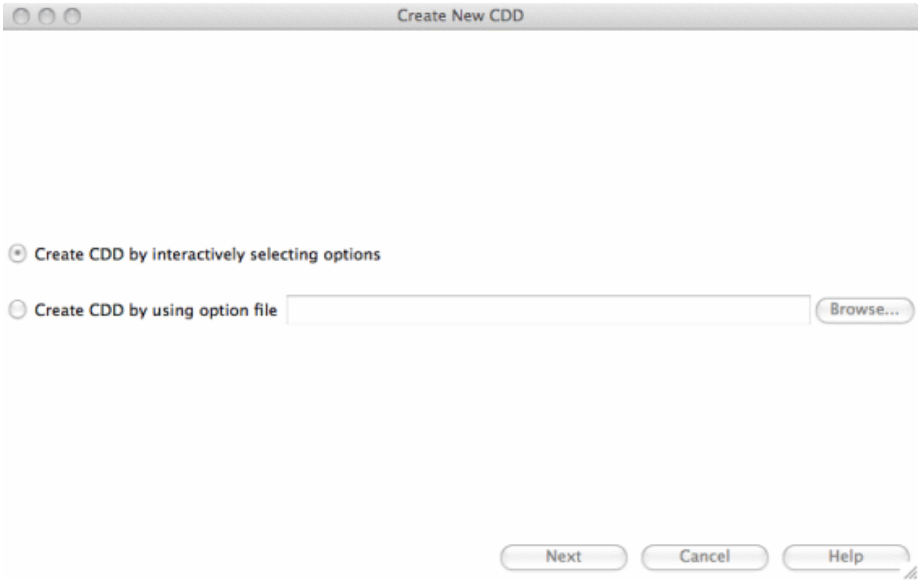
Creating a new CDD file from the GUI can be accomplished by either clicking on the **Generate New CDD File** button in the Wizard window (see Navigating the Wizard Window for more details on this window), by selecting the `File -> Generate -> New CDD...` menu option, or using the shortcut "Shift-Control-N". This file creation "wizard" allows the user to create a new CDD file that can be opened/merged into the GUI for coverage analysis. The methods that the CDD creation GUI allows for creating a new CDD file are quite flexible and are completely compatible with the command-line commands.

The following subsections describe the various windows used to create a CDD. Note that the CDD creation wizard uses several "pages" to describe the various options to set for creating a new CDD file, using **Back** and **Next** buttons on each page for navigation. Clicking on the **Back** button will not cause previously filled in pages to be reverted to their default values but rather will remain populated with the user's data. Similarly, if a page has been filled in with data and the **Back** button is clicked and then the **Next** button is clicked, the page contents are retained. The only way to clear out fields is to click the **Cancel** button at the bottom of each page and startup the CDD creation wizard again.

Clicking the **Cancel** button at any time will cause the CDD creation wizard to immediately exit. Clicking on the help (?) button at any time will bring up the user documentation for the current page (context sensitive help).

4.10.1 Navigating the New CDD Creation Option Selection Page

Figure CDD Creation Option Selection Page shows the look of this page of the CDD creation wizard.



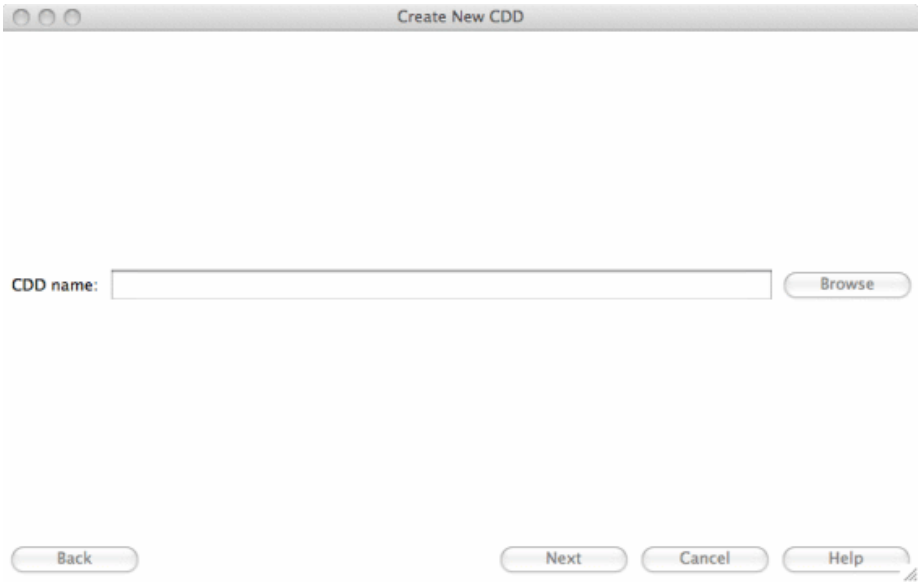
CDD Creation Option Selection Page

The first page of the CDD creation window allows the user to select whether all options should be interactively selected within the GUI or if a file should be read in which contains command-line options to the score command to use. If the latter is chosen, the contents of that file are read in and are populated in the GUI, but the CDD creation wizard will continue in the same fashion as the first option, allowing the user to change, remove or add to the options found in that file.

If the **Create CDD by interactively selecting options** entry is currently selected and the user wishes to not use a command-line file to populate the rest of the CDD creation pages, simply click on the **Next** button at the bottom of the page to continue. If the **Create CDD by using option file** entry is currently selected, the user must either type a valid filename or select one by clicking the **Browse...** button to the right of the file entry area and select a valid file. Once a valid filename is selected, the **Next** button will be enabled and the user may then click it to continue to the next page. Note that any filename can be selected (whether it's a valid command file or not). Only valid score command options found in that file are used to populate the wizard entries. No error message will be displayed if the file does not contain this information.

4.10.2 Navigating the New CDD Creation CDD Name Page

This page is used to specify the name of the output CDD file. CDD Creation Output Name Page shows the display of this page.



CDD Creation Output Name Page

The output CDD name must be specified and, as such, the **Next** button is disabled until a name is specified in the entry field. Any filename is allowed. If the user wishes to make use of a "Save As" style window for creating a filename, click the **Browse...** button and use the window for creating a filename. When the **Save** button is clicked, the specified filename is placed in the entry field and the **Next** button is enabled.

4.10.3 CDD Creation Type Page

This page is used to specify what type of CDD file to create. The four types of CDD files that can be created are:

1. An unscored CDD file that was created by parsing the design. This CDD file can later be scored via a VCD, LXT2 or FST dumpfile.

   This option can be chosen by selecting the **Parse Design Only** radiobutton.

2. A scored CDD file created from the design and from a previously created VCD, LXT2 or FST dumpfile.

   This option can be chosen by selecting the **Dumpfile:** option. If this option is chosen, a VCD, LXT2 or FST dumpfile is required to be specified before the next option page can be used. Either specify this dumpfile name by typing in an existing dumpfile name in the entry field or use the **Browse...** button to select a dumpfile.

3. An unscored CDD file, a top-level Verilog file containing a PLI call to run Covered in VPI mode, and a PLI tab file that can be used by some simulators for compilation purposes. See What is needed for VPI scoring? for details on how to use these files for simulation/scoring purposes.

   This option can be chosen by selecting the **VPI Module:** radiobutton. If this option is chosen, the name specified will be the name of the module file created that is used to allow Covered to run in VPI mode within a simulation. The basename of the module file will be used for the top-level module.

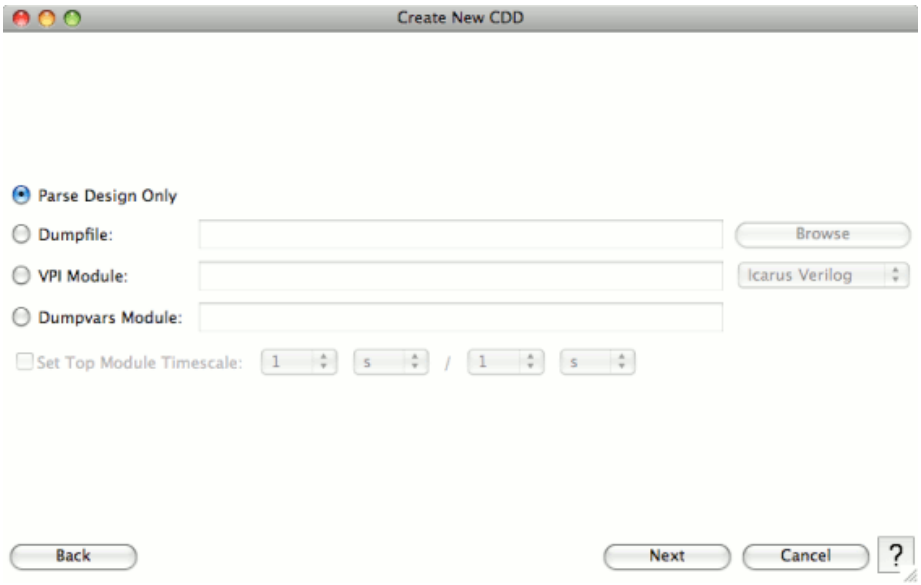
   In addition to providing a module name when this option is selected, the simulator type that will be used when scoring can be specified as well as the timescale information that can, optionally, be specified at the top of the generated module.

4. An unscored CDD file and a top-level Verilog file containing the necessary $dumpfile/$dumpvars calls to score the design. This module can be compiled as a top-level module along with the design to produce a VCD/LXT2/FST dumpfile to be used for coverage scoring. See Selecting What to Dump for details on how to use these files for simulation/scoring purposes.

   This option can be chosen by selecting the **Dumpvars Module:** radiobutton. If this option is chosen, the name specified will be the name of the module file created (the basename will be the name of the module and the basename of the VCD/LXT2/FST dumpfile created from the simulation) that will contain the $dumpfile/$dumpvars calls needed to score the design.

The timescale specifiers will be enabled if either the "VPI Module" or "Dumpvars Module" buttons are selected. Selecting the "Set Top Module Timescale" checkbutton will allow you to specify a timescale that will be added to the top of the VPI or dumpvars module file (setting the timescale in top-level modules is sometimes necessary for compilation purposes). If the checkbutton is not selected, no timescale information will be added to the generated top-level module file.

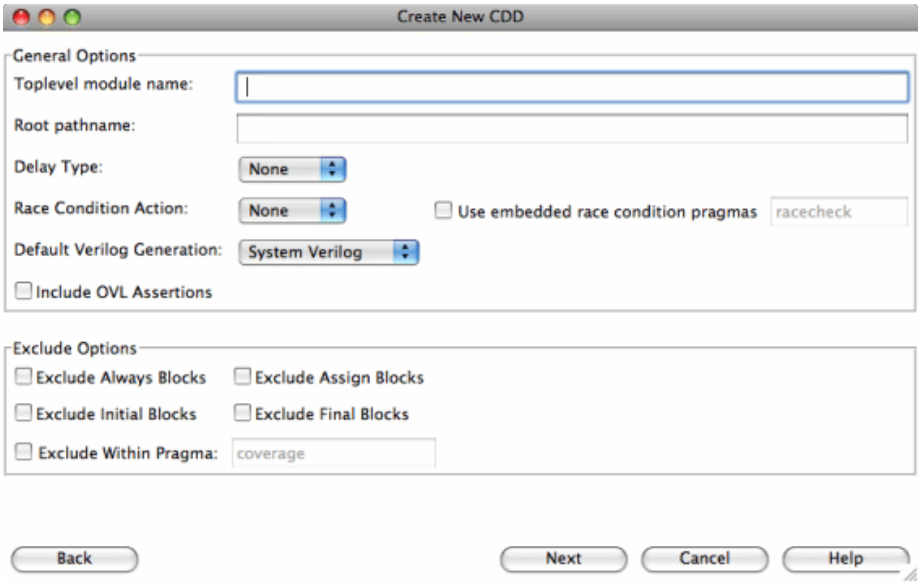
Figure CDD Creation Type Page displays the contents of this page. When all necessary fields have been specified, the **Next** button will be enabled and clicking it will lead to the next option page.



CDD Creation Type Page

4.10.4 CDD Creation Options1 Page

This page is used to specify several options to Covered's score command. See figure CDD Creation Options1 Page for the layout of this window.



CDD Creation Options1 Page

The following is a description about each of the CDD creation options available on this page.

4.10.4.1 General Options

- **Toplevel module name**

  Required? *Yes*

  The name of the top-level module must be specified for a given design to be parsed and a CDD generated from that source. Simply type in the name of the top-most module in the provided entry field. Once a name is provided, the **Next** button will be enabled to allow traversal to the next option page.

- **Root pathname**

  Required? *No*

  This entry field is an optional one to fill. This value is only necessary if the top-most module to be scored is not the top-most module of the simulator. If this is true, then the value that must be provided in the associated entry field must be the hierarchical reference to the module to be scored.

  An example of a valid root pathname would be something like: `top.foo.bar.dut`

  See Specifying What to Cover for more information.

- **Delay Type**

  This value specifies which delay should be used when a delay type of #(delay_max:delay_typ:delay_min) is encountered in the scored design. If this type of delay specification is not encountered, there is no need to change this value in the GUI. If the delay specification is encountered in the design and this value has not been specified, the default of *typ* will be used. Otherwise, the user can specify which delay value should be chosen by changing the displayed type to either *max*, *typ*, or *min*.

- **Race Condition Action**

  After the design has been parsed and elaborated, Covered performs a static race condition check on the design to verify that no potential race conditions exist. If a race condition exists in the design, this would cause the coverage results to potentially be errant.

  This parameter allows the user to tell Covered what to do if a race condition is found in the design. The valid values are:

  - *Silent*

    Specifies that no output should be generated. Any blocks with race conditions existing within them will be automatically removed from coverage consideration.

  - *Warning*

Specifies that race condition should be output and those blocks containing the race condition will be automatically removed from coverage consideration.

- *Error*

  Causes Covered to display the race condition violations and stop scoring after all race condition checking has occurred.

- *Ignore*

  Causes Covered to skip race condition checking completely. This option can be used if the user is confident that no real race conditions exist in the design but Covered's static race condition checker is displaying potential problems. If no race conditions actually exist, ignoring race condition checking results allows more of the design to be considered for coverage.

- **Use embedded race condition pragmas**

  If this option is selected, a user-specified pragma ("racecheck" by default) is parsed and any potential race conditions found between the off/on pragma pair are ignored by the race condition checker.

  See Avoiding Race Condition Checking for more details.

- **Default Verilog Generation**

  Covered's parser supports parsing for Verilog-1995, Verilog-2001 and SystemVerilog, defaulting its parser to support SystemVerilog. However, for older designs which may contain names that are keywords in the newer versions, it may be necessary to change the parser of Covered for the entire design. This option specifies the version of the parser which is used for the entire design.

  For information regarding the ability to specify a certain version of the parser for a specific module, see Module Generation....

- **Include OVL Assertions**

  If this option is selected, any OVL assertion modules found within the scored design will have assertion coverage information extracted from them.

4.10.4.2 Exclude Options

- **Exclude Always Blocks**

  If this option is selected, all `always` code blocks will be excluded from coverage consideration.

- **Exclude Assign Blocks**

  If this option is selected, all `assign` code blocks will be excluded from coverage consideration.

- **Exclude Initial Blocks**

  If this option is selected, all `initial` code blocks will be excluded from coverage consideration.

- **Exclude Final Blocks**

  If this option is selected, all `final` code blocks will be excluded from coverage consideration.

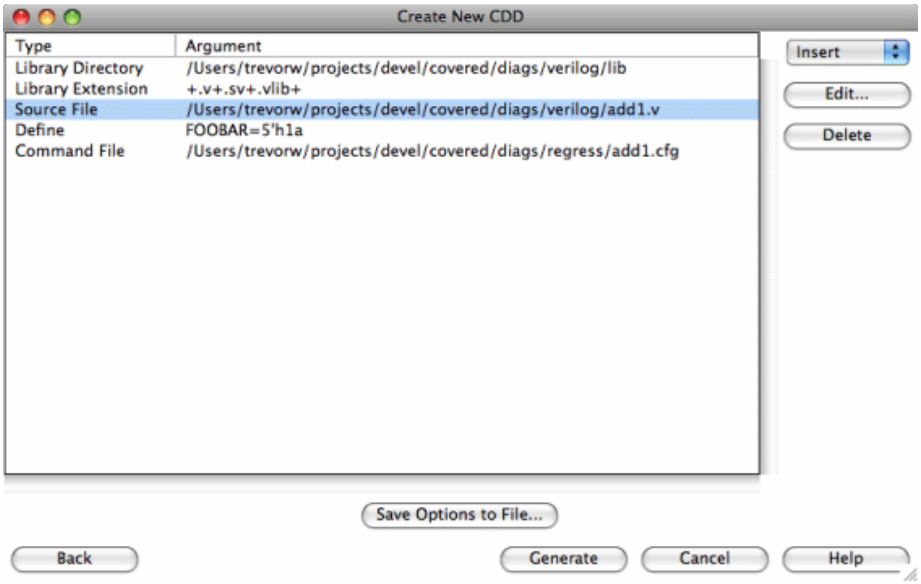
- **Exclude Within Pragma**

  If this option is selected, all code blocks will be excluded between the exclusion off/on pragmas. The name of the exclusion pragma is defaulted to "coverage" but can be changed within the associated entry field.

  See Using the -ep Option for Exclusion for more information.

4.10.5 CDD Creation Options2 Page

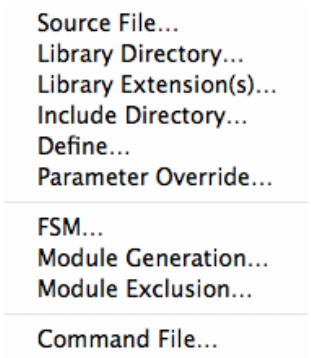
The second CDD creation options page contains a listbox which will contain the various user-specified options that have been inserted via the **Insert...** menubutton. Any options that exist within this listbox can be edited by selecting the option and clicking on the **Edit...** button or deleted by clicking on the **Delete** button. Additionally, options may be reorganized within the listbox by simply clicking on an option and dragging it into the needed position. See figure CDD Creation Options2 Page for the layout of this option page.



CDD Creation Options2 Page

To add an option to the listbox, simply click on the **Insert...** menubutton which will display a list of options. Select one of these options to display a window where the contents of that option can be specified. Clicking the **OK** button in one of those windows will cause the option to be added to the option listbox. Clicking **Cancel** in one of those windows will cause the option to not be added to the listbox. Clicking help (?) will display context-sensitive help information to be displayed about the current window.

Figure CDD Creation Insert Menubutton shows the contents of the **Insert...** menubutton. The following sections describe each option and its corresponding entry window.



CDD Creation Insert Menubutton

4.10.5.1 Source File...

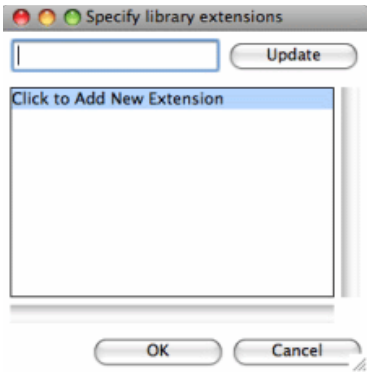
Selecting this insert option will cause an open file selector window to be displayed. Use it to choose a single file that will be parsed as a library file (i.e., the -v option to most Verilog compilers).

4.10.5.2 Library Directory...

Selecting this insert option will cause an open directory selector window to be displayed. Use it to choose a single directory that will be used as a library directory (i.e., the -y option to most Verilog compilers).

4.10.5.3 Library Extension(s)...

Selecting this insert option will cause a window to be displayed that will allow the user to add library file extensions to the library file parser (i.e., the +libext+ option to most Verilog compilers). CDD Creation Library Extension Window is a display of this window.



CDD Creation Library Extension Window

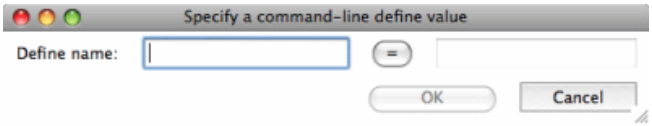
To add a new extension, select the **Click to Add New Extension** item in the extension box (selected by default), type in the name of the extension in the upper entry box and either hit the "Enter" key or click on the **Update** button. If an incorrect extension was input, you may edit it by clicking on the incorrect item in the extension listbox. This will cause the item to be listed in the upper entry box where it can be edited or removed. Click the **OK** button to accept the specified library extensions and add these to the main window listbox. Click the **Cancel** button to ignore the recent updates in this window.

4.10.5.4 Include Directory...

Selecting this insert option will cause an open directory selector window to be displayed. Use it to choose a single directory that will be used as an include directory (i.e., the +incdir+ option to most Verilog compilers).

4.10.5.5 Define...

Selecting this insert option will cause a window to be displayed that will allow the user to add a command-line variable definition to be added to the score command (i.e., the +define+ option to most Verilog compilers).



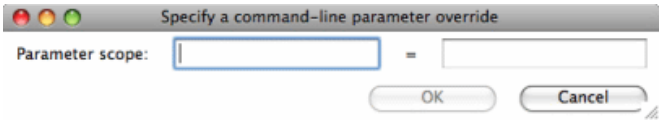
CDD Creation Define Window

To add a new define, enter the name of the definition variable in the entry box to the right of "Define name:". If the variable does not require a value, simply click on the **OK** button to submit it to the main window option listbox. If the variable requires a value, click on the **=** button, enter the value in the entry box to the left of that button, and click on the **OK** button. To abort the addition of the definition, simply click on the **Cancel** button.

The OK will only be enabled when the user has entered a value in the "Define name:" entry box.

4.10.5.6 Parameter Override...

Selecting this insert option will cause a window to be displayed that will allow the user to add a parameter overload option to the score command. Because Verilog "defparam" is not used by the Covered parser, this option is the only way to override parameter values.



CDD Creation Parameter Override Window

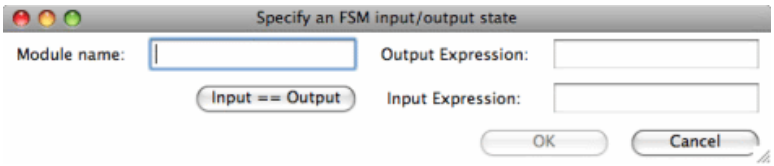
To add a new parameter override option, enter the full hierarchical path to the parameter to override in the left entry box and add the override value in the right entry box. See Overriding Parameters (-P option) for details on how to specify an override value. After both values have been input, click the **OK** button to add the parameter override option to the main window listbox. To abort the parameter override addition, click on the **Cancel** button.

The OK button will only be enabled if both values are input in the entry boxes.

4.10.5.7 FSM...

Selecting this insert option will cause a window to be displayed that will allow the user to add an FSM input/output state on the score command line. This option allows the user to specify the location of FSMs within the design without inlining this information with the use

of pragmas for the purposes of gaining FSM coverage information.



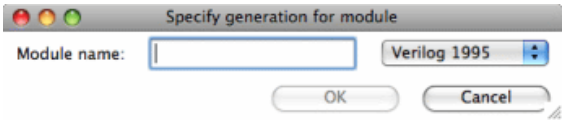
CDD Creation FSM Specification Window

To add a new FSM option, enter the name of the module containing the FSM in the upper-left entry box, the description of the output state in the upper-right entry box (see Scoring FSMs for information on properly specifying the output/input state information). If the input state is the same as the output state, simply click on the **Input == Output** button which will fill in the input state with the same value as the output state value. If the input state differs from the output state, enter the input state in the lower-right entry box. When all entry fields have been filled in, click on the **OK** button to add the FSM description to the main window listbox. To abort the addition, click on the **Cancel** button.

The OK button will only be enabled if all entry fields have been filled in.

4.10.5.8 Module Generation...

Selecting this insert option will cause a window to be displayed that will allow the user to specify a Verilog generation (Verilog 1995, Verilog 2001 or SystemVerilog) for a particular module. This generation value will override the universal generation value (see the "Default Verilog Generation" item in General Options) for the design.



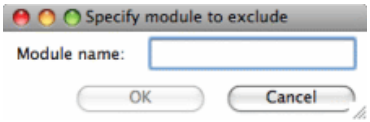
CDD Creation Module Generation Window

To specify a Verilog generation to be used for a specific module, enter the name of the module in the left entry box and select one of the Verilog generations in the drop-down menu on the left. Once this information has been specified, click on the **OK** button to add this option to the main window listbox. To abort the module generation addition, click on the **Cancel** button.

The OK button will only be enabled if a module name is specified in the left entry box.

4.10.5.9 Module Exclusion...

Selecting this insert option will cause a window to be displayed that will allow the user to specify a module to exclude from parsing and coverage consideration. More information on the purpose for excluding modules is discussed in Specifying What to Cover.



CDD Creation Module Exclusion Window

To specify a Verilog module to exclude from the score command parser, enter the name of the module in the entry box to exclude. Click on the **OK** button to add the module exclusion option to the main window listbox. Click on the **Cancel** button to abort the option addition.

The OK button will only be enabled if a value is specified in the entry box.

4.10.5.10 Command File...

Selecting this insert option will cause an open file selector window to be displayed which will allow the user to find a file that contains Covered score command-line options within it. Select a file and click on the **Open** button to cause the selected file to be added to the main window listbox. Click on the **Cancel** button to abort this action.

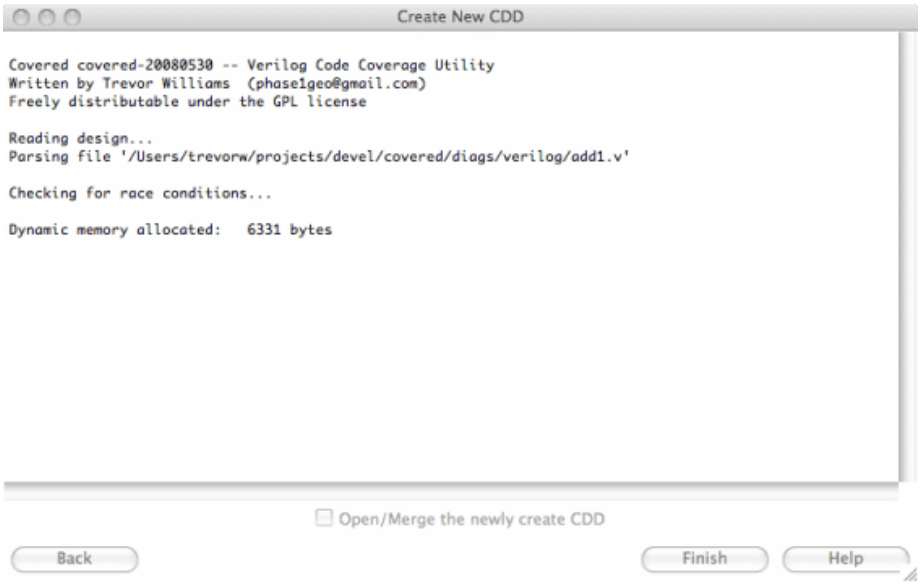
4.10.5.11 Other Notes

After all options have been specified, the **Save Options to File...** button can be clicked to cause all previously specified options to be saved to a file which can later be loaded by the new CDD creation window. Simply click the button and specify a filename to save the options under and click the **Save** button.

When all options have been setup, click the **Generate** button to create the new CDD file. This button will advance to another page containing the output from Covered's score command.

4.10.6 CDD Creation Output Pane

The final pane in the CDD creation window wizard is the output window. After the **Generate** button is pressed, the user-supplied options are gathered and executed within a system shell and the output of that command is sent to this pane's textbox. If the score command run was a success, the **Open/Merge the newly created CDD** checkbutton will be enabled, allowing the user to open or merge the new CDD file into the GUI immediately. To exit the CDD creation process, click on the **Finish** button. If the score command failed for any reason, the user may click on the **Back** button to change the options specified to the score command or simply click the **Finish** button to exit the new CDD creation wizard.



CDD Creation Output Window

4.11 Creating an ASCII report file

Though Covered's graphical coverage analyzer is very useful for examining coverage, there may still be times when a textual report is necessary. For that purpose, the GUI has a menu item for generating an ASCII coverage report, running Covered's "report" command on the currently loaded coverage information.

The ASCII report generator options are only available if one or more CDD files are currently loaded in the GUI.

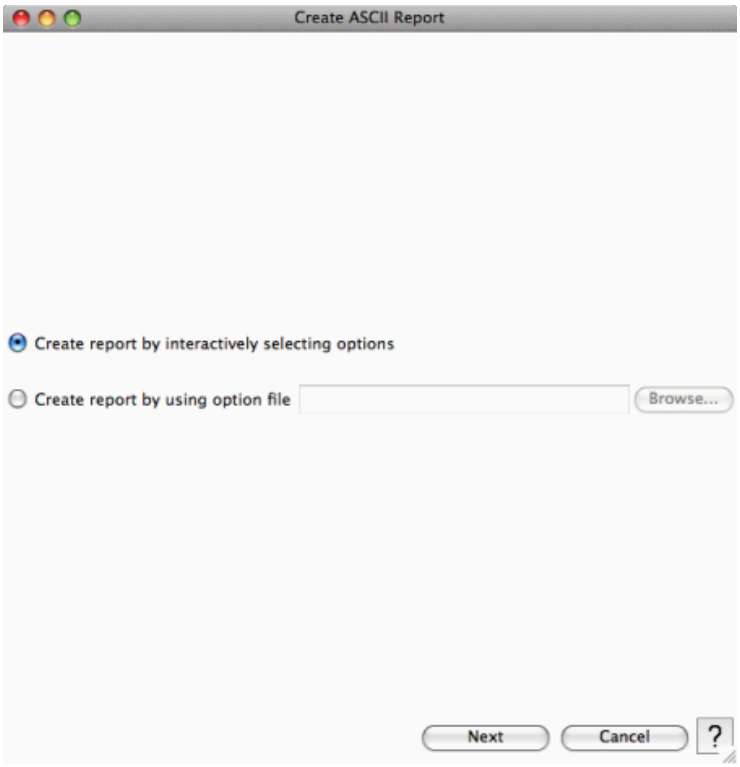
When the `File -> Generate -> ASCII Report` menu option is selected (or the "Cntrl-Shift-R" shortcut is used), the ASCII Report Generator Window is created to allow the user to specify the current report output options and generate the report file. Once the report has been generated, it can be viewed within the GUI itself or it can be viewed with any type of file viewer/editor available.

The following subsections describe the various windows used to create a report file. Note that the report generation wizard uses several "pages" to describe the various options to set for creating a new report file, using **Back** and **Next** buttons on each page for navigation. Clicking on the **Back** button will not cause previously filled in pages to be reverted to their default values but rather will remain populated with the user's data. Similarly, if a page has been filled in with data and the **Back** button is clicked and then the **Next** button is clicked, the page contents are retained. The only way to clear out fields is to click the **Cancel** button at the bottom of each page and startup the report generation wizard again.

Clicking the **Cancel** button at any time will cause the report generation wizard to immediately exit. Clicking on the **?** button at any time will bring up the user documentation for the current page (context sensitive help).

4.11.1 Navigating the Report Generator Selection Page

Report Generation Option Selection Page shows the look of this page of the report generation wizard.



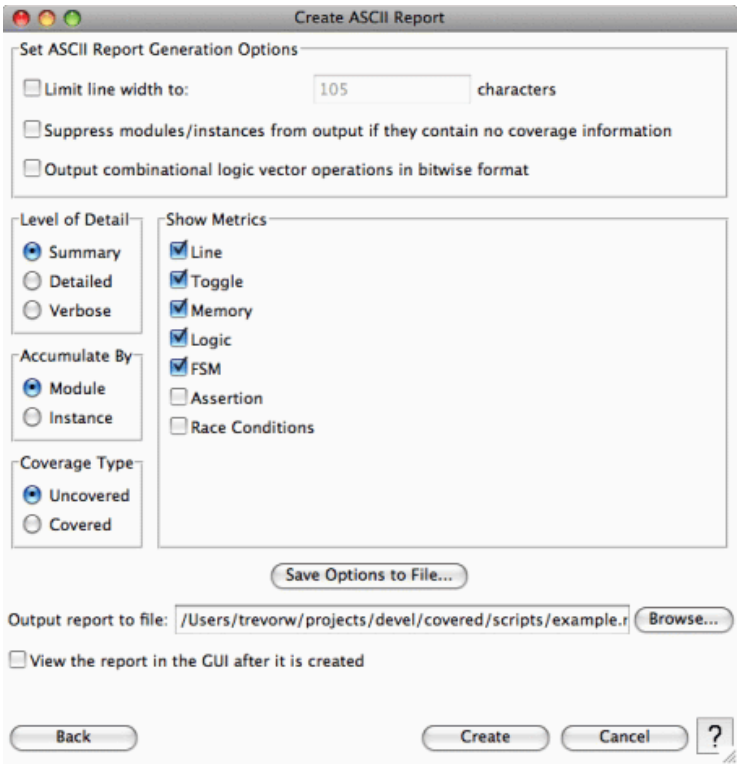
Report Generation Option Selection Page

The first page of the report generation window allows the user to select whether all options should be interactively selected within the GUI or if a file should be read in which contains command-line options to the report command to use. If the latter is chosen, the contents of that file are read in and are populated in the GUI, but the report generation wizard will continue in the same fashion as the first option, allowing the user to change, remove or add to the options found in that file.

If the **Create report by interactively selecting options** entry is currently selected and the user wishes to not use a command-line file to populate the rest of the report generation pages, simply click on the **Next** button at the bottom of the page to continue. If the **Create report by using option file** entry is currently selected, the user must either type a valid filename or select one by clicking the **Browse...** button to the right of the file entry area and select a valid file. Once a valid filename is selected, the **Next** button will be enabled and the user may then click it to continue to the next page. Note that any filename can be selected (whether it's a valid command file or not). Only valid report command options found in that file are used to populate the wizard entries. No error message will be displayed if the file does not contain this information.

4.11.2 Report Generation Options Page

This wizard page allows you to setup the options to create the needed report. Figure Report Generation Options Page shows the contents of this page.



Report Generation Options Page

The options page is split into several different categories of options:

- Set ASCII Report Generation Options
- Level of Detail
- Accumulate By
- Coverage Type
- Show Metrics

4.11.2.1 Set ASCII Report Generation Options

- **Limit line width to X characters**

  If this option is checked, the number of characters that will be displayed per line when outputting design logic will be limited to the number of characters specified in the entry box. If this option is unchecked, the length of the output design logic will be determined by the way the logic was formatted in the design file.

  This option can be useful for printing purposes where a logic design file contains logic that is formatted to exceed the number of characters than can be printed on a page or viewed on a screen.

- **Suppress modules/instances from output if they contain no coverage information**

  If this option is checked, any modules (or instances) that were considered for code coverage but did not contain any coverable code will be excluded from the report. If this option is unchecked, all modules/instances will be reported.

  This option may help make the report files easier to view if there are many modules/instances that do not contain coverage points.

- **Output combinational logic vector operations in bitwise format**

  Outputs combinational vector logic expressions in bitwise format instead of treating the vector as a 0 or "not 0" value. This can create quite a bit of extra output when creating verbose reports, but can provide the information for a more thorough analysis for these expression types. This option has the same effect as the **-b** option to the report command.

4.11.2.2 Level of Detail

These radiobuttons control the level of detail that is output in the generated report. Only one value can be selected for the three available detail values:

- **Summary**
- **Detailed**
- **Verbose**

See Summary Vs. Detailed Vs. Verbose for information on the difference between these three options.

4.11.2.3 Accumulate By

These radiobuttons control whether coverage information is accumulated by instance or by module. Only one value can be selected for the two available options. See Module Vs. Instance for information on the difference between these two options.

4.11.2.4 Coverage Type

These radiobuttons control whether covered or uncovered coverage points are output in the report. Only one value can be selected for the two available options. See Covered Vs. Uncovered for information on the difference between these two options.

4.11.2.5 Show Metrics

These checkbuttons control which coverage metric information is output in the generated report. You may select any number (or combinations) of these options. All of the metrics available for viewing in the GUI can be output to a report file, including:

- **Line**
- **Toggle**
- **Memory**
- **Combinational Logic**
- **FSM**
- **Assertion**
- **Race Conditions**

  This item, though it is not a coverage metric, can be useful for understanding what logic was not considered for coverage due to violating the static race condition checks and what check was violated for each block.

4.11.2.6 Save Options to File...

Once the above options have been specified, it can be useful to save those parameters to a file which can be loaded at a later time in the Navigating the Report Generator Selection Page page of the GUI report generation wizard. This same file can also be fed "as is" into the Covered's command-line report command via the **-f** option. Click on this button to open a "Save As" window where you can create a filename to save this information into. Click on the **Save** button of this window to exit the window and save the information. Click on the **Cancel** button to abort this action.

4.11.2.7 Output report to file frame

This frame of the ASCII Report window allows the user to specify a filename/location for the generated output file. By default, the report name is the same as the first CDD file loaded, substituting the .cdd extension for a .rpt extension. By default, the directory location to store the generated report is the same directory that the first CDD file exists in. To change this name/location, the user may either enter a value in the entry field or click on the **Browse...** button to display a "Save As" type window.

4.11.2.8 View the report in the GUI after it is created

Selecting this option will cause a window to open after the report has been generated which will contain the contents of the generated report in a read-only view. You can search in this window and scroll to help view the entire document.

Once you have completed setting everything up as you need, click on the **Create** button to generate the report file. If everything worked, the report generation window will close and, if enabled via the View the report in the GUI after it is created selector, a file viewer window will

be displayed with the contents of the newly generated report file. Click on the "X" button in the window manager to close this window when you have finished with it.

4.12 Creating a CDD Ranking Report

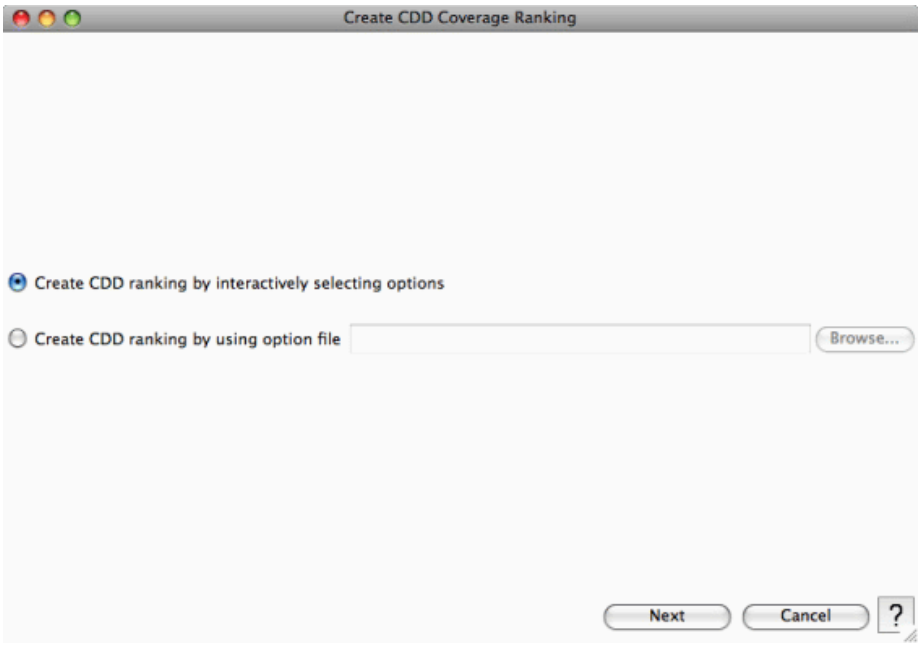
Generating a ranking report (see The rank Command for more information) can be accomplished in the GUI by either selecting the **Rank CDD Coverage** button in the Wizard window (see Navigating the Wizard Window for details), selecting the `File -> Generate -> CDD Ranking Report...` menu option, or using the keyboard shortcut Shift-Cntrl-C. Any of these actions will cause the CDD ranking report wizard window to be started.

The following subsections describe the various windows used to create a CDD ranking report file. Note that the CDD ranking wizard uses several "pages" to describe the various options to set for creating a CDD ranking report file, using **Back** and **Next** buttons on each page for navigation. Clicking on the **Back** button will not cause previously filled in pages to be reverted to their default values but rather will remain populated with the user's data. Similarly, if a page has been filled in with data and the **Back** button is clicked and then the **Next** button is clicked, the page contents are retained. The only way to clear out fields is to click the **Cancel** button at the bottom of each page and startup the CDD ranking wizard again.

Clicking the **Cancel** button at any time will cause the CDD ranking wizard to immediately exit. Clicking on the help (?) button at any time will bring up the user documentation for the current page (context sensitive help).

4.12.1 CDD Ranking Option Selection Page

Figure CDD Ranking Option Selection Page shows the look of this page of the CDD ranking wizard.



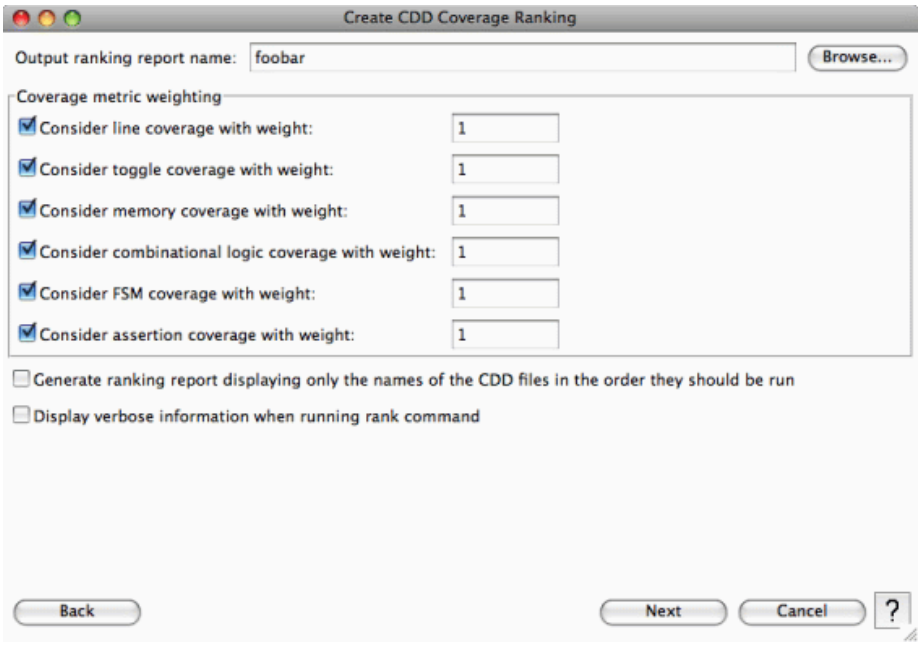
CDD Ranking Option Selection Page

The first page of the CDD ranking wizard allows the user to select whether all options should be interactively selected within the GUI or if a file should be read in which contains command-line options to the rank command to use. If the latter is chosen, the contents of that file are read in and are populated in the GUI, but the CDD ranking wizard will continue in the same fashion as the first option, allowing the user to change, remove or add to the options found in that file.

If the **Create CDD ranking by interactively selecting options** entry is currently selected and the user wishes to not use a command-line file to populate the rest of the CDD ranking pages, simply click on the **Next** button at the bottom of the page to continue. If the **Create CDD ranking by using option file** entry is currently selected, the user must either type a valid filename or select one by clicking the **Browse...** button to the right of the file entry area and select a valid file. Once a valid filename is selected, the **Next** button will be enabled and the user may then click it to continue to the next page. Note that any filename can be selected (whether it's a valid command file or not). Only valid rank command options found in that file are used to populate the wizard entries. No error message will be displayed if the file does not contain this information.

4.12.2 CDD Ranking Option Page

This page of the CDD ranking wizard allows the user to specify certain options to the ranking command when it is run. The following subsections document each option within this window.



CDD Ranking Options Page

4.12.2.1 Output ranking report name

This option is a required field to fill in before the rank command can be run. Simply specify a filename in the entry box or use the **Browse...** button to help create a filename to save the ranking report file to.

The Next button will not be enabled until a filename has been specified in the entry box.

4.12.2.2 Coverage metric weighting

The checkboxes within this frame allow the user to specify an importance value for each coverage metric. The allowed values within the entry box for each metric is a value greater than zero, where all values are relative to each other. In other words, if the weight for line coverage is set to a value of 10 and the weight of toggle coverage is set to a value of 1, it indicates that line coverage should be considered 10x more important for coverage selection purposes than toggle coverage. Changing the various weight values can have an impact on which CDD files are chosen to be included in the "need to run" list and/or the order in which CDD files should be run to maximize the coverage obtained.

By default, all metrics are included (except for assertion coverage) and are given a weight value of 1 (all coverage metrics are considered to be equally important for ranking purposes). To remove a metric from ranking consideration, simply uncheck the checkbox to the left of the metric line.

4.12.2.3 Generating only names

When a CDD ranking report is generated, the report contains header information and a columnar list of accumulated and per CDD ranking information. If the user is only interested in creating a report that includes the names of the CDD files that are needed to run (and the order to run these CDD files in), checking this option will produce just that, allowing for easier usage in Makefiles or other commands needed to run regression runs.

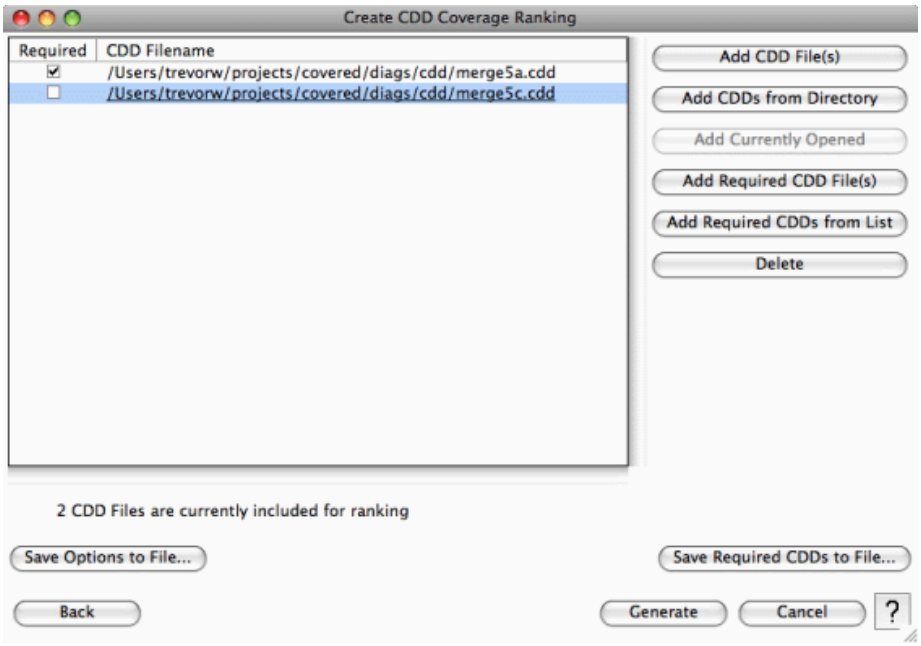
4.12.2.4 Displaying verbose information when running rank command

When the ranking order is being generated, there is output from the rank command concerning the filenames that are being read and when the command is writing to the output file; however, if you would like to know more information about what's going on "under the hood", enabling this option will cause more verbose information to be reported in the output screen, including the amount of time spent executing each phase of the rank process, the number of files selected for ranking in each phase, the number of coverage points hit by selected CDD files for each phase and the number of files excluded from ranking. Leave this box unchecked if this information is not important/interesting to you.

4.12.3 CDD Ranking Input File Selection Page

This page allows the user to add all of the CDD files to rank. Since the GUI CDD ranking generation command can be run at any time (even when no CDD files have currently been loaded), the input files to the rank command can be any valid CDD file that can be merged with each other.

As can be seen in the following figure, the window is comprised of a filename listbox in the upper left-hand corner of the page, several buttons that allow for various file selection strategies on the right-hand side of the page, a current file count information bar beneath the listbox, a button to allowing saving the currently setup ranking options to be saved to a file, a button to save any selected "required" CDD files to a file, and a **Generate** button in the bottom navigation bar.



CDD Ranking Input File Selection Page

4.12.3.1 Adding CDD files

Adding CDD files to be ranked is performed by clicking on one or more of the buttons on the right-hand side of the page. When files are added to the input file listbox, the current number of CDD files in the listbox are reported in the information bar beneath the listbox.

Next to each filename specified in the input file listbox is a checkbutton that specifies if the associated CDD file should be considered a mandatory CDD file to be ranked (i.e., it cannot be excluded from the list of CDD files that should be run during regression to achieve full coverage). This checkbutton can be set/cleared at any time for any CDD file in the listbox.

The following are the various ways that CDD files can be added to or removed from the input file listbox.

- **Add File(s)**

  Clicking this button will bring up a file selection box that will allow the user to select one or more files that will will be added to the input file listbox with the "Required" field left unchecked. This option is useful for selecting certain files within one or more directories to rank.

- **Add CDDs from Directory**

  Clicking this button will bring up a directory selection box that will allow the user to specify a directory to load CDD files from. Only files with the ".cdd" extension that exist in this directory will be added to the input file listbox.

- **Add Currently Opened**

  Clicking this button will add any CDD files that are currently opened within the GUI into the input file list.

- **Add Required CDD File(s)**

  Clicking this button will bring up a file selection box that will allow the user to select one or more files that will be added to the input file listbox with the "Required" field checked.

- **Add Required CDDs from List**

  Clicking this button will bring up a file selection box that will allow the user to select a file that contains a list of CDD files to be required for ranking (files will be added into the input file listbox with the "Required" field checked).

- **Delete**

When one or more CDD files are selected in the input file listbox, the **Delete** button will be enabled. Clicking this button will remove the selected files from the input file listbox. The information bar displaying the number of currently selected CDD files will be updated accordingly.

If desired, any CDD files listed in the input file listbox can be moved to change the order of the input files. However, it should be noted that the input file order will not necessarily have any impact on the CDD ranking order list.

4.12.3.2 Saving the CDD ranking options

If the user wishes to save the current option/file list setup, click on the **Save Options to File...** button. This will cause a file save window be displayed where a filename can be chosen and saved to by clicking on the **Save** button in this window. To cancel the option save, click on the **Cancel** button in that window.

The generated options file can be used in the CDD Ranking Option Selection Page window or in the command-line call to the rank command's -f option.

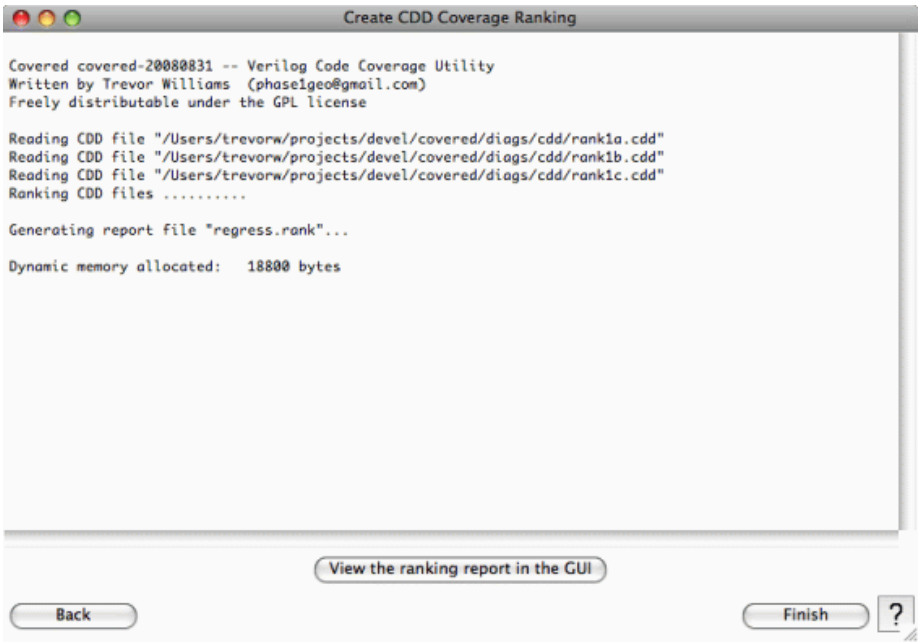
4.12.3.3 Save Required CDDs to File...

If one or more files are labeled as "Required" in the input file listbox, this button will be enabled. Clicking on this button will bring up a "Save As" file selection box. Specifying a filename and clicking on the "Save" button will cause a file by that name to be created that contains a list of all of the CDD files listed as "Required". This file may be used at a later time via the -required-list option to the rank command (if using the rank command on the command-line) or by clicking on the **Add Required CDDs from List** button in the rank command wizard input file selection page.

4.12.3.4 Generating the CDD ranking report

After all of the input files have been included (you must include more than one to create a CDD ranking report), the **Generate** button will be enabled, click on this button to advance to the next page in the CDD ranking wizard where the CDD ranking report command will be run and its output displayed.

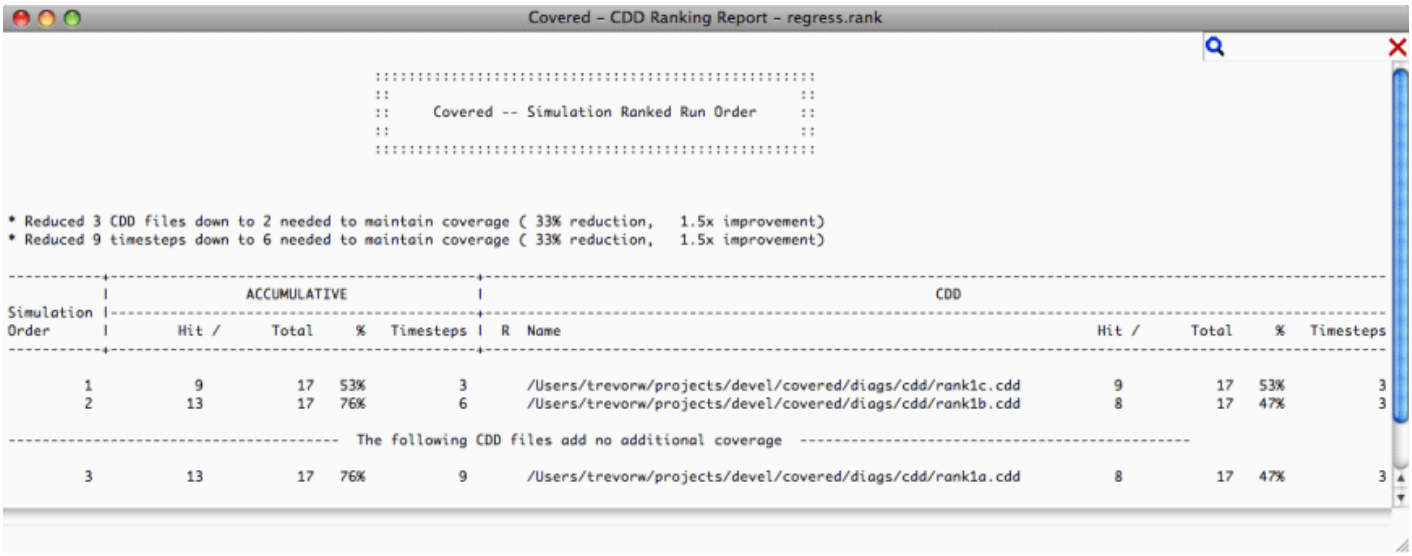
4.12.4 CDD ranking Command Output Page



CDD Ranking Command Output Page

The above figure shows an example of output displayed to the CDD ranking command output page. The purpose of this page is to show you the results of the rank command run. If any errors occurred during this run, the output from those errors will be displayed within this page. If the command ran successfully, the **Finish** button will be enabled at the bottom of the screen.

Additionally, the **View the ranking report in the GUI** button will be enabled. When clicked, this button will display the created report file in a file viewer which may be read and searched. Click on the window manager "X" button to close the file viewer window. The following figure shows an example of the file viewer window and an example of the report output.

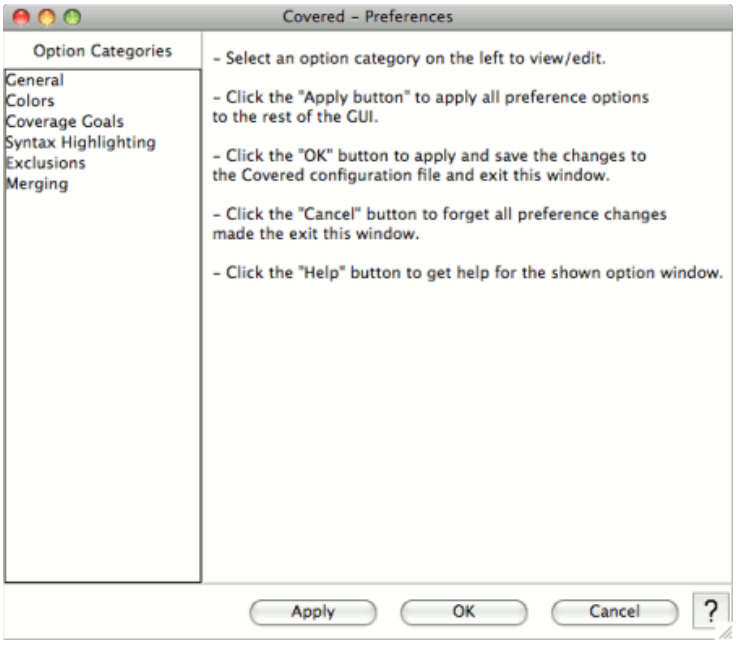


CDD Ranking Report Page

When the user has successfully generated a CDD ranking report, clicking the **Finish** button will close the CDD ranking report window.

4.13 Navigating the Preferences Window

The preferences window allows the user to adjust the view of some of the GUI elements for the purpose of more easily analyzing coverage information for a specific user. Main Preferences Window shows the preferences window that is created when the user selects the **`View > Preferences...`** menu option from the main window.



Main Preferences Window

4.13.1 Option Categories

On the left side of the preferences window is a list of option categories that can be modified. Selecting any one of the items in the listbox will display the modifiable options to the frame to the right of the listbox. The following option categories exist:

- Preferences General Pane
- Preferences Color Pane
- Preference Coverage Goals Pane
- Preferences Syntax Highlighting Pane
- Preferences Exclusion Pane

- Preference Merging Pane

4.13.2 Preference Buttons

4.13.2.1 Apply Button

Clicking on the **Apply** button will cause the currently selected preference options to be immediately updated in Covered but will not update the preference option values in the .coveredrc file. This has the effect of changing the preferences for the current application only. Clicking on this button will not close the Preferences window.

4.13.2.2 OK Button

Clicking on the **OK** button will cause the currently selected preference options to be immediately updated in Covered and will update the .coveredrc file with the new values. This also causes the Preferences window to close.

4.13.2.3 Cancel Button

Clicking on the **Cancel** button will cause the the Preferences window to close without updating Covered with the currently selected values and without updating the .coveredrc file.

4.13.2.4 Help (?) Button

Clicking on the help (?) button will cause the help page for the currently selected'Option Category' to be displayed.

4.13.3 Preferences Configuration File

User preferences are loaded from the ".coveredrc" file that is searched for in the following order:

1. Current directory that the 'covered report -view' command was executed from.
2. The user's home directory.
3. The installation directory (by default this is /usr/local/share/covered).

If user preferences are changed in this window and the **OK** button is pressed, the settings are immediately saved back to a configuration file and the GUI is immediately updated to reflect the new user changes. If the **Cancel** button is pressed, all changes are thrown away and the configuration file remains unchanged. The location of the stored configuration file depends on which configuration file location was initially loaded. The following table describes this correlation.

Location of Written .coveredrc File According to the Location of Read .coveredrc File	
Loaded config file location	**Stored config file location**
Current directory	Current directory
Home directory	Home directory
Install directory	Current directory
None found	Current directory

4.13.4 Preferences General Pane

The general preferences pane allows the user to control Covered's settings that are not specific to an individual task. The following subsections describe the controls available within this pane.

4.13.4.1 Show wizard window on startup

This checkbutton selects if Covered's wizard popup window will be displayed upon starting Covered. Simply check this button if you would like to see the wizard at the beginning of each startup or uncheck it to cause the wizard window to not be displayed on startup. On the wizard window itself, there is an option to change this behavior as well.

Note that you can always display the wizard window by selecting its menu option in the view menu.

4.13.4.2 Save state of GUI when exiting the application

This checkbutton determines if certain GUI element (i.e., window placement/sizing, module/instance listbox column arrangement/width/state, etc.) information is saved when the GUI exits. If this checkbutton is selected, the state is saved to the configuration file on application exit and will be used when the application is started. If the checkbutton is deselected, the state will not be saved and the default GUI element behaviors will be used.

This setting allows the user to setup the various GUI elements to their liking and retains this information whenever the application is restarted, saving the user the hassle of getting the environment setup again.

While the application is being used, any GUI state information will be remembered for the duration of the application run, regardless of the setting of this preference item.

4.13.4.3 Show tooltips

This checkbutton specifies if GUI widget tooltips should be displayed if the user hovers the cursor over a GUI element that has a tooltip associated with it. To disable tooltips from being used, uncheck this option and click either the **Apply** or **OK** buttons.

4.13.5 Preferences Color Pane

The color preference pane allows the user to adjust the foreground and background colors for all code that is considered covered, uncovered, or in a race condition state.

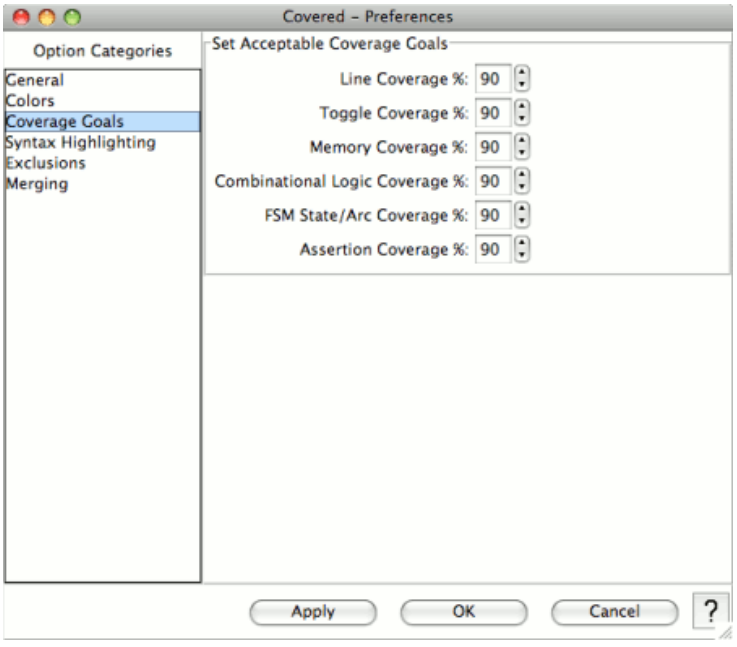
4.13.5.1 Set Highlight Color

This frame contains six buttons that affect the foreground and background colors used to highlight code in the file viewer portion of the main window. The currently selected foreground/background colors are displayed to the left of the corresponding buttons to allow the user to see the selections prior to committing to them by clicking the **Apply** or **OK** buttons at the bottom of the Preferences window.

The "F" button controls the foreground color while the "B" button controls the background color. Clicking one of these buttons will bring up a color selection window that you can use to modify the current color to one of your choosing.

4.13.6 Preference Coverage Goals Pane

The coverage goals preference pane allows the user to change the acceptable coverage goals for each coverage metric.



Coverage Goals Window

4.13.6.1 Set Acceptable Coverage Goals Frame

This frame contains four spinner boxes that contains values between 0 and 100. These values correspond to a coverage percentage for each coverage metric that the user deems to be "good enough". These values are used by the listbox to help visually identify modules/instances that are meeting or exceeding these goals and which modules are not meeting these goals. If a coverage percentage for a module/instance is less than this value, the module is colored red. If the coverage percentage is greater than or equal to this value, the module/instance is colored yellow (or green which indicates that the module/instance reached 100% coverage for a given metric).

To increase the specified percentage value, simply click on the "Up" button to the right of the percentage until the desired value is displayed. To decrease the specified percentage value, simply click on the "down" button to the right of the percentage until the desired value is displayed.

- **Line Coverage %**

  Sets the coverage goal of line coverage to the specified percentage to the right. If line coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

- **Toggle Coverage %**

  Sets the coverage goal of toggle coverage to the specified percentage to the right. If toggle coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

- **Memory Coverage %**

  Sets the coverage goal of memory coverage to the specified percentage to the right. If memory coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

- **Combinational Logic Coverage %**

  Sets the coverage goal of combinational coverage to the specified percentage to the right. If combinational logic coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

- **FSM State/Arc Coverage %**

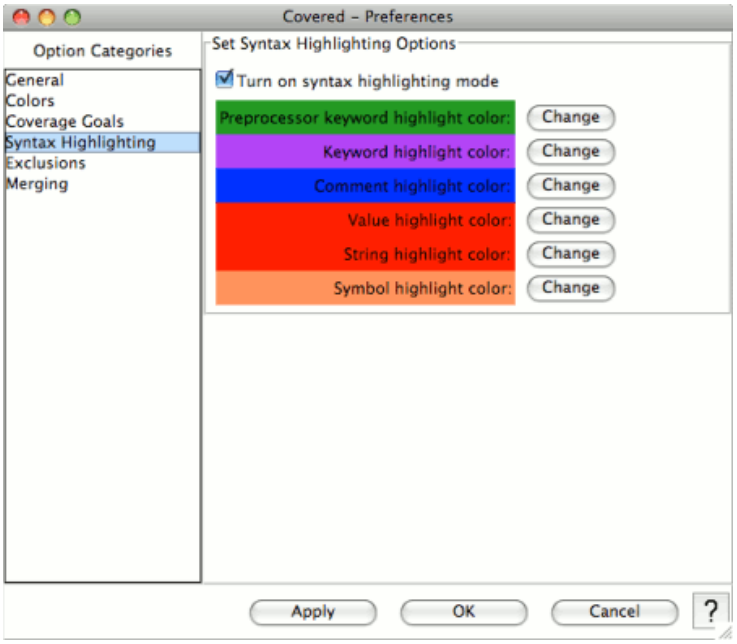
  Sets the coverage goal of FSM state/arc coverage to the specified percentage to the right. If FSM state/arc coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

- **Assertion Coverage %**

  Sets the coverage goal of assertion coverage to the specified percentage to the right. If assertion coverage for a given module/instance meets or exceeds this percentage value, it will be displayed yellow in the Module/Instance listbox on the Main window.

4.13.7 Preferences Syntax Highlighting Pane

The syntax highlighting preferences pane allows the user to adjust the syntax highlighting mode and colors for any displayed Verilog source code.



Syntax Highlighting Preferences Pane

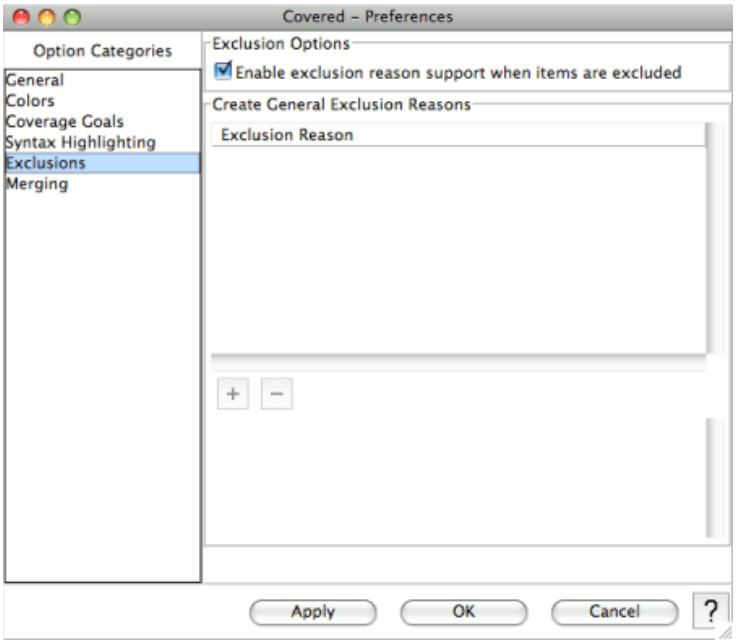
4.13.7.1 Set Syntax Highlighting Options

This frame allows the user to specify if syntax highlighting should be performed in all Verilog source viewers or not. When the square to the left of the **Turn on syntax highlighting mode** is selected, syntax highlighting will occur; otherwise, it will not perform syntax highlighting. The various colored buttons below the selection mode allow the user to select the foreground color of the text associated with its specified syntax type (Comments, Preprocessor Keywords, Verilog Keywords, Strings, Constant Values, and Operation Symbols). Simply click on one of the buttons to change its color. Doing so will cause a color selection window (see Syntax Highlighting Preferences Pane above) to be displayed.

If the syntax highlighting mode is disabled by the user, the color selection buttons will be disabled.

4.13.8 Preferences Exclusion Pane

The exclude preferences pane allows the user to control whether or not the coverage exclusion reason window automatically pops up when a coverage point is excluded, and it allows the user to create some general purpose exclusion reasons. The reasons created in this pane can be selected in the exclusion reason window's drop-down table and can help to expedite the exclusion process when many coverage points are being excluded for the same reason. Exclusion Preferences Pane shows the layout of this preference pane.



Exclusion Preferences Pane

4.13.8.1 Enable exclusion reason support when items are excluded

If this option is enabled, whenever the user clicks on a coverage point to cause it to be excluded from coverage, a popup window will be displayed, allowing for an exclusion message to be associated with the excluded coverage point.

If this option is not enabled, the coverage point will still be excluded but no exclusion message will be associated with it.

If a coverage point was excluded and is selected to be included and the coverage point had an exclusion message associated with it, the exclusion message will be forgotten/deleted.

4.13.8.2 Create General Exclusion Reasons

This frame allows the user to add/delete general purpose exclusion reasons that can be selected from within the exclusion reason popup window's drop-down table. Having these reasons specified can help to speed up coverage point exclusion when multiple coverage points need to be excluded for the same reason.

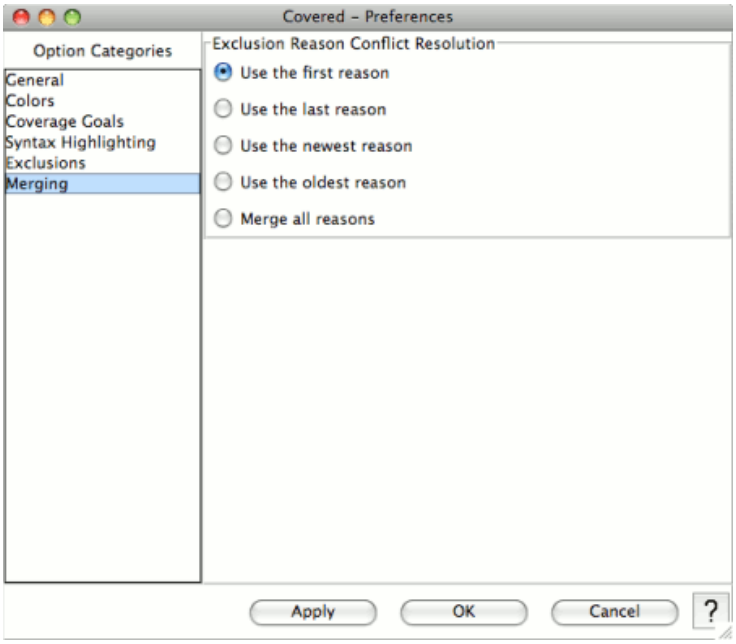
To add a new general purpose exclusion reason, simply type in the general purpose exclusion reason in the bottom textbox (note that any formatting characters such as newlines, tabs, extra spaces will be removed from the exclusion message) and click the "+" button. You will see the exclusion reason get added to the table when this occurs. Note that the "+" button will only be enabled when you have entered at least one character in the textbox.

To remove a general purpose exclusion reason from the table, simply click on the reason in the table and click the "-" button. Note that the "-" button will only be enabled once an entry has been selected in the table.

To help find an exclusion reason more easily within the exclusion reason popup window, you may also rearrange the order of the exclusions within the table by simply clicking on an entry and dragging it to the desired location. The order of the exclusion reasons specified in this table will be the initial order of the exclusion reasons in the popup window.

4.13.9 Preference Merging Pane

The merging pane is needed to specify to Covered how to resolve conflicts that are found when merging CDD files. Although coverage points and exclusions are simply merged (ORed) together, things such as exclusion reasons can exist for the same coverage point but contain different information. When two exclusion reasons are given for the same coverage point, Covered must decide how to resolve this information and this preference pane allows the user to set what this behavior will be.



Merging Preferences Pane

4.13.9.1 Exclusion Reason Conflict Resolution Frame

This frame allows the user to specify the way in which exclusion reason conflicts should be resolved. The following options are provided:

- **Use the first reason**

  Specifies if two or more exclusion reasons are found for the same coverage point, Covered should use the exclusion reason found in the first CDD file merged. The rest of the exclusion reasons are thrown away.

- **Use the last reason**

  Specifies if two or more exclusion reasons are found for the same coverage point, Covered should use the exclusion reason found in the last CDD file merged. The rest of the exclusion reasons are thrown away.

- **Use the newest reason**

  Specifies if two or more exclusion reasons are found for the same coverage point, Covered should use the newest exclusion reason (when an exclusion is created, a timestamp is saved along with the coverage reason). The rest of the exclusion reasons are thrown away.

- **Use the oldest reason**

  Specifies if two or more exclusion reasons are found for the same coverage point, Covered should use the oldest exclusion reason. The rest of the exclusion reasons are thrown away.

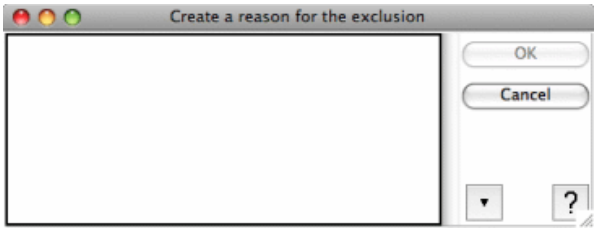
- **Merge all reasons**

  Specifies if two or more exclusion reasons are found for the same coverage point, Covered should merge all of them. No information is lost in this case.

4.14 The Exclusion Reason Popup Window

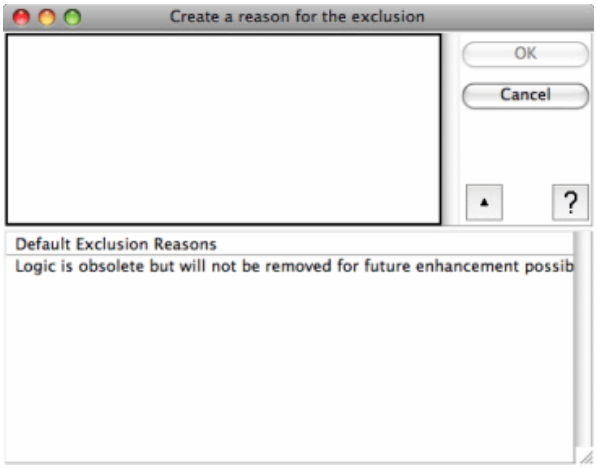
When an uncovered coverage point needs to be excluded for various reasons, a user-supplied message may be associated with the exclusion, specifying the reason for the exclusion. This information can be useful documentation for a coverage analysis and is possible to achieve using the Covered GUI. This task is accomplished with the exclusion reason popup window which is automatically popped up when a

coverage point is marked for exclusion (assuming that the capability is enabled in the exclusion preferences pane). The Exclusion Reason Popup Window shows what this popup window looks like.



The Exclusion Reason Popup Window

To specify an exclusion reason, you may do either enter the exclusion reason into the textbox located on the left side of this window or you may select a pre-created, general purpose exclusion reason in the drop-down table. To view this table, click on the down arrow button at the bottom of this window (see Full View of the Exclusion Reason Popup Window). To hide this table, click on the same button (which will be changed to an up arrow button). If a general-purpose exclusion reason is selected from the table, it will be displayed in the top textbox where it may be optionally edited for the current coverage point.



Full View of the Exclusion Reason Popup Window

Once the exclusion reason has been entered as desired in the textbox (please note that any formatting characters such as newlines, tabs and/or multiple spaces will be automatically removed and replaced with a single space character), simply click on the **OK** button which will associate the exclusion reason with the excluded coverage point and close the popup window. Note that the **OK** button will only be enabled if at least one character is entered in the textbox.

Editing a general purpose reason in the popup window only has an effect for this coverage point; it does not change the general purpose reason value itself (this can only be done in the preferences window).

To not specify an exclusion reason with an excluded coverage point, simply click on the **Cancel** button.

4.15 Navigating the Wizard Window

The Wizard window starts up automatically at the start of the Covered GUI. It allows the user to quickly perform some of the more regular tasks within the GUI. The window is not necessary for performing these tasks, but for the new user it makes figuring out how to perform these tasks a bit easier. Covered GUI Wizard Window shows the layout of this window.



Covered GUI Wizard Window

The buttons at the top of the window are shortcuts to performing the tasks displayed on the button. Clicking the associated button will cause a window to open to perform the given task. The following are the available shortcut buttons:

- **Generate New CDD File**

  Allows the user to create a new CDD file for a given design by specifying the various scoring options needed to create the CDD file.

- **Open/Merge CDD File(s)**

  Opens a CDD file for coverage viewing. If one or more CDD files have previously been opened, this option opens and merges one or more CDD files with the coverage results of the previous CDD files for coverage viewing.

- **Rank CDD Files**

  Allows the user to specify CDD files that will be compared with one another and ranked in the order that they should be run in a regression to achieve the most coverage in the least amount of time. It also indicates which CDD files do not add any interesting coverage information and could, therefore, be excluded from regression for the purposes of code coverage.

At the bottom of the wizard window, there is a checkbutton option that allows the user to not display the wizard window on startup. This option can be changed in the Preferences General Pane at any time.

When the wizard window is closed, it can be redisplayed by clicking on the **`View > Wizard...`** menu item. The window will automatically be closed when a button on the wizard window is clicked or when the user closes the window with the window manager's "X" button.

5 FAQ

5.1 FAQ

This page contains a list of accummulated Frequently Asked Questions. If you are having difficulties using Covered or have questions that are not answered in the User's Guide, please check this list for answers. If you do not find the information that you are looking for in the User's Guide or this FAQ, please send an e-mail to:

phase1geo@gmail.com

Q: Is a CDD file generated from a newer version of Covered compatible with a CDD file generated from an older version?

A: The answer to this question is, "Well, that depends...". Since the format of the CDD file is only used by Covered, the contents and format of this file may be changed to suit the needs of Covered. This means that it is possible for a CDD file created from an older version of Covered to be incompatible with a newer version. Additionally, the developers of Covered will not make any attempts to make sure that older CDD files can be properly read by a newer version of Covered. It is suggested that any CDD files generated from a particular version of Covered be merged and/or reported on by that same version, and if Covered is upgraded, new CDD files should be generated.

This being said, it is also a possibility that between versions of Covered a "backwards compatibility" may be maintained if a change to this file's format is not required.

Q: When I run the score command, Covered seems to take a long time to run. Is there anything that I can do to speed up scoring?

A: While Covered is continuously being enhanced to provide better performance for all commands (especially the score command), it may still take a while to complete scoring if one or more of the following is true:

1. The design being scored is sufficiently large
2. The VCD/LXT2/FST dumpfile is sufficiently large
3. The VCD/LXT2/FST dumpfile contains dump information for a part of the design not being scored
4. When Covered was configured, the --enable-debug and/or --enable-profiling options were specified

If reason (1) is true, speeding up each run can be achieved by one of the following suggestions:

- Reduce the scored design in size by eliminating modules or constructs from its design tree. (See Specifying What to Cover for more information on how to accomplish this)
- Split up the design into smaller parts along module boundaries and generate coverage for those parts.

  It is not currently possible to append these modules into one file. Merging and reporting must be done on these smaller pieces independently from one another.

If reason (2) is true, nothing can be done except to shorten the run time of the diagnostics to produce shorter dumpfiles.

If reason (3) is true, the $dumpvars call in the simulator should be modified to only generate VCD/LXT2/FST dump information for the modules included in scoring. If there are modules not being scored which are included in the $dumpvars calls, please remove these unnecessary modules from dumpfile output.

If reason (4) is true, Covered should be reconfigured without these options specified. The debugging and profiling facilities are enormous performance degraders and immediate simulation performance enhancement will be seen if Covered is rebuilt without these options specified.

If you believe that you have a situation which is void of these prevailing reasons and Covered is still running slowly, please send an e-mail to me. I will consider these problems to be of lower priority than actual bugs but will look into the situation to see where code can be optimized.

Q: I get an assertion error when running Covered, what should I do?

A: Covered uses C assertions to make sure that internal pointers are not be referenced when NULL and that certain internal situations do not arise. If you receive some type of assertion error when running Covered, it means that something went wrong internally in Covered. The error is NOT due to user error. Please submit a bug report containing the assertion error message, file and line number. Additionally, run the Covered command with the -D global option (covered -D *command*) and provide the output from doing this with the bug report (please only specify tail of output if the output is too lengthy).

If Covered provides any other type of error message (something other than a core dump), Covered has found a user error that must be fixed by the user. Please do not submit bug reports if these errors are encountered, unless you wish to add a question about it to the FAQ.

Q: Covered is giving me a parser error for Verilog code that seems to be syntactically correct. What is wrong?

A: If Covered displays a syntax (parse) error during the score command and the Verilog code is written correctly, it is because Covered's parser is incomplete. Please submit a bug or send an e-mail containing a code snippet of the unsupported Verilog.

Q: Is Covered's Verilog parser Verilog-2001 compliant?

A: This is currently being worked on. The parser should properly parse all code; however, not all Verilog-2001 constructs are considered for coverage due to lack of support for them in Covered's core. If there is code that will not parse that should, please submit a bug report for this.

Q: What is the difference between the stable release and the development release?

A: Covered's development consists of two active "branches": the stable branch and the development branch. When a new stable branch is created (ex. 0.4 or 0.5), it represents the latest version of the development branch. As Covered's development branch is worked on (adding new features, changes to the core, etc.), the stable branch remains in a feature frozen stage. Only bug fixes, documentation updates or minor enhancements (changes that will not affect the core code) are made to the stable branch. When a number of these changes have been accumulated on the stable branch, a snapshot of the stable branch is made available for public download (ex. 0.4.1). User's of the stable release should expect no major feature changes from minor rev to minor rev and should expect a somewhat polished version of the project (few if any bugs and correct user documentation).

User's of the development releases should expect to see more bugs and fewer documentation consistencies (although an attempt is made to minimize both) but should expect lots of feature additions, optimizations, improvements, etc. from release to release. Bug findings found in the stable release are applied to the development branch when applicable. It is important to note that both the stable and development releases contain a regression testbench that must fully pass before either release is made. This should minimize bugs in both releases and give user's of either branch a level of confidence that the release is usable.

6 Epilogue

6.1 Epilogue

6.1.1 Author

Covered is programmed and documented by Trevor Williams (phase1geo@gmail.com)

6.1.2 Special Thanks

I would like to acknowledge certain individuals for their help in making Covered the tool that it is today.

- Ed Spittles - Thank you for your tireless efforts in logging bugs, providing suggestions on enhancements/optimizations (including the tip on using of FIFOs to replace VCD/LXT2/FST file usage), your input into scoring optimizations for vectors and CDD ranking, and for allowing me access to needed information for fixing those "hard to reproduce" bugs.

- Clifford E. Cummings - provided race condition checking rules and clarifications about those rules.
- Stephen Williams - Thanks for helping me understand some of the ins and outs of what's required "under the hood" to handle some of Verilog's constructs.
- Daniel McMahill - testing and patches for 64-bit compatibility

6.1.3 Reporting Bugs

If you have read this user's guide, chances are that you might be a Covered user. As such, it is encouraged that any bugs that you might find while using Covered get reported so that this project may become more stable for yourself and for other users. To submit a bug report or a request for enhancement, you may either send an e-mail to Trevor Williams (see Author or, if you have a SourceForge login, can report a bug by going to:

http://sourceforge.net/projects/covered

and clicking on the "Bugs" link.

6.1.4 Mailing List

The Covered project has a user mailing list. Whenever there are stable releases being made of the project, an e-mail will get sent out to this list stating the new release is available. Additionally, the mailing list is a great way to bring up usability issues with the tool and, of course, bugs. The mailing list can also be useful if you have a question about using Covered -- there is probably someone else who has encountered the same problem that might be able to provide you with some help.

To subscribe to this e-mail list, please visit the following URL, enter the prescribed information and click on the "Submit" button.

http://lists.sourceforge.net/lists/listinfo/covered-users

6.1.5 Homepage

The Covered project homepage is established on SourceForge at the following URL address.

http://covered.sourceforge.net

6.1.6 Copyright and Licensing

This program is distributed under the GNU Public License GPL.

6.1.6.1 GNU GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc. 675 Mass Ave, Cambridge, MA 02139, USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

6.1.6.1.1 Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software-to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Library General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

6.1.6.1.2 Terms and Conditions for Copying, Distribution, and Modification

- [0.] This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The ``Program'', below, refers to any such program or work, and a ``work based on the Program'' means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term ``modification''.) Each licensee is addressed as ``you''.

  Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

- [1.] You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

  You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

- [2.] You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

  - [a.] You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.

  - [b.] You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.

  - [c.] If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

  These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

  Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

  In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

- [3.] You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

  - [a.] Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

  - [b.] Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

  - [c.] Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such

an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

- [4.] You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

- [5.] You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

- [6.] Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

- [7.] If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

  If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

  It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

  This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

- [8.] If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

- [9.] The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

  Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and ``any later version'', you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

- [10.] If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

  NO WARRANTY

- [11.] BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM ``AS IS'' WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY

AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

- [12.] IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

  END OF TERMS AND CONDITIONS

6.1.6.1.3 Appendix: How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the ``copyright'' line and a pointer to where the full notice is found.

> Copyright © 20yy This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
>
> This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.
>
> You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.